

1.1 ROS简介

ROS诞生背景

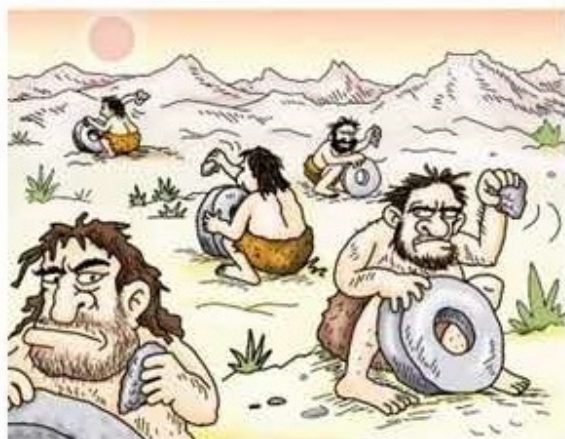
机器人是一种高度复杂的系统性实现，机器人设计包含了机械加工、机械结构设计、硬件设计、嵌入式软件设计、上层软件设计....是各种硬件与软件集成，甚至可以说机器人系统是当今工业体系的集大成者。



机器人体系是相当庞大的，其复杂度之高，以至于没有任何个人、组织甚至公司能够独立完成系统性的机器人研发工作。

一种更合适的策略是：让机器人研发者专注于自己擅长的领域，其他模块则直接复用相关领域更专业研发团队的实现，当然自身的研究也可以被他人继续复用。这种基于"复用"的分工协作，遵循了**不重复发明轮子**的原则，显然是可以大大提高机器人的研发效率的，尤其是随着机器人硬件越来越丰富，软件库越来越庞大，这种复用性和模块化开发需求也愈发强烈。

在此大背景下，于 2007 年，一家名为 **柳树车库 (Willow Garage)** 的机器人公司发布了 **ROS**(机器人操作系统)，ROS是一套机器人通用软件框架，可以提升功能模块的复用性，并且随着该系统的不断迭代与完善，如今 ROS 已经成为机器人领域的事实标准。



传统模式

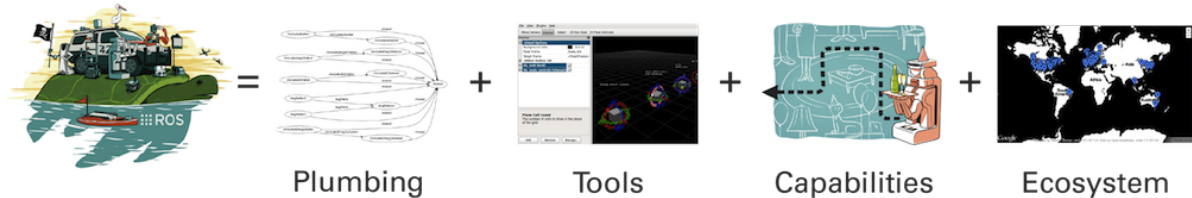


现代模式

1.1.1 ROS概念

ROS全称Robot Operating System(机器人操作系统)

- ROS是适用于机器人的**开源**元操作系统
- ROS集成了大量的工具，库，协议，提供类似OS所提供的功能，简化对机器人的控制
- 还提供了用于在**多台计算机**上获取，构建，编写和运行代码的工具和库，ROS在某些方面类似于“机器人框架”
- ROS设计者将ROS表述为“ROS = Plumbing + Tools + Capabilities + Ecosystem”，即ROS是通讯机制、工具软件包、机器人高层技能以及机器人生态系统的集合体



1.1.2 ROS设计目标

机器人开发的分工思想，实现了不同研发团队间的共享和协作，提升了机器人的研发效率，为了服务“分工”，ROS主要设计了如下目标：

- **代码复用**: ROS的目标不是成为具有最多功能的框架，ROS的主要目标是支持机器人技术研发中的代码**重用**。
- **分布式**: ROS是进程（也称为**Nodes**）的分布式框架,ROS中的进程可分布于不同主机，不同主机协同工作，从而分散计算压力
- **松耦合**: ROS中功能模块封装于独立的功能包或元功能包，便于分享，功能包内的模块以节点为单位运行，以ROS标准的IO作为接口，开发者不需要关注模块内部实现，只要了解接口规则就能实现复用,实现了模块间点对点的松耦合连接
- **精简**: ROS被设计为尽可能精简，以便为ROS编写的代码可以与其他机器人软件框架一起使用。
ROS易于与其他机器人软件框架集成：ROS已与OpenRAVE，Orocos和Player集成。
- **语言独立性**: 包括Java，C++，Python等。为了支持更多应用开发和移植，ROS设计为一种语言弱相关的框架结构，使用简洁，中立的定义语言描述模块间的消息接口，在编译中再产生所使用语言的目标文件，为消息交互提供支持，同时允许消息接口的嵌套使用
- **易于测试**: ROS具有称为**rotest**的内置单元/集成测试框架，可轻松安装和拆卸测试工具。
- **大型应用**: ROS适用于大型运行时系统和大型开发流程。
- **丰富的组件化工具包**: ROS可采用组件化方式集成一些工具和软件到系统中并作为一个组件直接使用，如RVIZ（3D可视化工具），开发者根据ROS定义的接口在其中显示机器人模型等，组件还包括仿真环境和消息查看工具等
- **免费且开源**: 开发者众多，功能包多

1.1.3 ROS发展历程

- ROS是一个由来已久、贡献者众多的大型软件项目。在ROS诞生之前，很多学者认为，机器人研究需要一个开放式的协作框架，并且已经有不少类似的项目致力于实现这样的框架。在这些工作中，斯坦福大学在2000年年中开展了一系列相关研究项目，如斯坦福人工智能机器人（STanford AI Robot, STAIR）项目、个人机器人（Personal Robots, PR）项目等，在上述项目中，在研究具有代表性、集成式人工智能系统的过程中，创立了用于室内场景的高灵活性、动态软件系统，其可以用于机器人学研究。
- 2007年，柳树车库（Willow Garage）提供了大量资源，用于将斯坦福大学机器人项目中的软件系统进行扩展与完善，同时，在无数研究人员的共同努力下，ROS的核心思想和基本软件包逐渐得到完善。

- ROS的发行版本 (ROS distribution) 指ROS软件包的版本, 其与Linux的发行版本 (如Ubuntu) 的概念类似。推出ROS发行版本的目的在于使开发人员可以使用相对稳定的代码库, 直到其准备好将所有内容进行版本升级为止。因此, 每个发行版本推出后, ROS开发者通常仅对这一版本的bug进行修复, 同时提供少量针对核心软件包的改进。
 - 版本特点: 按照英文字母顺序命名, ROS 目前已经发布了ROS1 的终极版本: noetic, 并建议后期过渡至 ROS2 版本。noetic 版本之前默认使用的是 Python2, noetic 支持 Python3。
建议版本: noetic 或 melodic 或 kinetic
-

Distro	Release date	Poster	Tuturtle, turtle in tutorial	EOL date
RDS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
RDS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
RDS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)
ROS Jade Turtle	May 23rd, 2015			May, 2017
ROS Indigo Igloo	July 22nd, 2014			April, 2019 (Trusty EOL)
ROS Hydro Medusa	September 4th, 2013			May, 2015
ROS Groovy Galapagos	December 31, 2012			July, 2014
ROS Fuerte Turtle	April 23, 2012			--
ROS Electric Emys	August 30, 2011			--
ROS Diamondback	March 2, 2011			--
ROS C Turtle	August 2, 2010			--
ROS Box Turtle	March 2, 2010			--

1.2 ROS安装

我们使用的是 ROS 版本是 Noetic，那么可以在 ubuntu20.04、Mac 或 windows10 系统上安装，虽然一般用户平时使用的操作系统以 windows 居多，但是 ROS 之前的版本基本都不支持 windows，所以当前我们选用的操作系统是 ubuntu，以方便向历史版本过渡。ubuntu 安装常用方式有两种：

- 实体机安装 ubuntu (较为常用的是使用双系统，windows 与 ubuntu 并存)；
- 虚拟机安装 ubuntu。

两种方式比较，各有优缺点：

- 方案1可以保证性能，且不需要考虑硬件兼容性问题，但是和 windows 系统交互不便；
- 方案2可以方便的实现 windows 与 ubuntu 交互，不过性能稍差，且与硬件交互不便。

在 ROS 中，一些仿真操作是比较耗费系统资源的，且经常需要和一些硬件(雷达、摄像头、imu、STM32、arduino...)交互，因此，原则上建议采用方案1，不过如果只是出于学习目的，那么方案2也基本够用，且方案2在 windows 与 ubuntu 的交互上更为方便，对于学习者更为友好，因此本教程在此选用的是方案2。当然，具体采用哪种实现方案，请按需选择。

如果采用虚拟机安装 ubuntu，再安装 ROS 的话，大致流程如下：

1. 安装虚拟机软件(比如：virtualbox 或 VMware)；
2. 使用虚拟机软件虚拟一台主机；
3. 在虚拟主机上安装 ubuntu 20.04；
4. 在 ubuntu 上安装 ROS；
5. 测试 ROS 环境是否可以正常运行。

虚拟机软件选择上，对于我们学习而言 virtualbox 和 VMware 都可以满足需求，二者比较，前者免费，后者收费，所以本教程选用 virtualbox。

1.2.1 安装虚拟机软件

1. 下载 virtualbox

安装 virtualbox 需要先访问官网，下载安装包，官网下载地址：<https://www.virtualbox.org/wiki/Downloads>



The screenshot shows the VirtualBox website's download page. On the left, there is a navigation menu with links: About, Screenshots, Downloads (highlighted with a red box and an arrow), Documentation, End-user docs, Technical docs, Contribute, and Community. The main content area has the heading 'VirtualBox Download VirtualBox'. Below this, it says 'Here you will find links to VirtualBox binaries and its source code.' There are two sections: 'VirtualBox binaries' and 'VirtualBox 6.1.10 platform packages'. The 'VirtualBox binaries' section includes a link to 'VirtualBox 6.0 builds' and 'VirtualBox 5.2 builds'. The 'VirtualBox 6.1.10 platform packages' section lists four options: Windows hosts, OS X hosts, Linux distributions, and Solaris hosts. The 'Linux distributions' option is highlighted with a red box and labeled '按需选择' (Select as needed). Below this, there is a section for 'VirtualBox 6.1.10 Oracle VM VirtualBox Extension Pack' with a link to 'All supported platforms', which is also highlighted with a red box and labeled '扩展插件也一并下载，后期会使用' (Download the extension pack as well, it will be used later). At the bottom, there is a section for 'VirtualBox 6.1.10 Software Developer Kit (SDK)' with a link to 'All platforms'.

2.安装virtualbox

virtualbox 安装比较简单，如果没有特殊需求，双击安装文件，一直 "下一步" 即可。



自定安装

选择你要安装的功能。

请选择以下选项：

- ☒ 添加系统菜单条目
- ☒ 在桌面创建快捷方式
- ☒ 在快速启动栏创建快捷方式
- ☒ 注册文件关联

Version 6.0.8

< 上一步(B)

下一步(N) >

取消(C)



警告：
网络界面

安装 Oracle VM VirtualBox 6.0.8 网络功能将重置网络连接并暂时中断网络连接。

立即安装吗？

Version 6.0.8

是(Y)

否(N)

准备好安装

安装向导准备好进行 自定义 安装。

单击“安装”开始安装。如果你要检查或更改任何安装设置，单击“上一步”。单击“取消”结束向导。

Version 6.0.8

< 上一步(B)

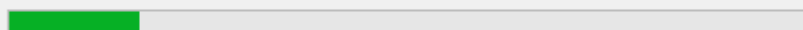
安装(I)

取消(C)

Oracle VM VirtualBox 6.0.8

在安装向导安装 Oracle VM VirtualBox 6.0.8 时请稍候。这需要几分钟时间。

Status: 正在更新组件注册



Version 6.0.8

< 上一步(B)

下一步(N) >

取消(C)

Would you like to install this device software?



Name: Oracle Corporation Universal Serial Bus ...
Publisher: Oracle Corporation

☒ Always trust software from "Oracle Corporation".

Install**Don't Install**

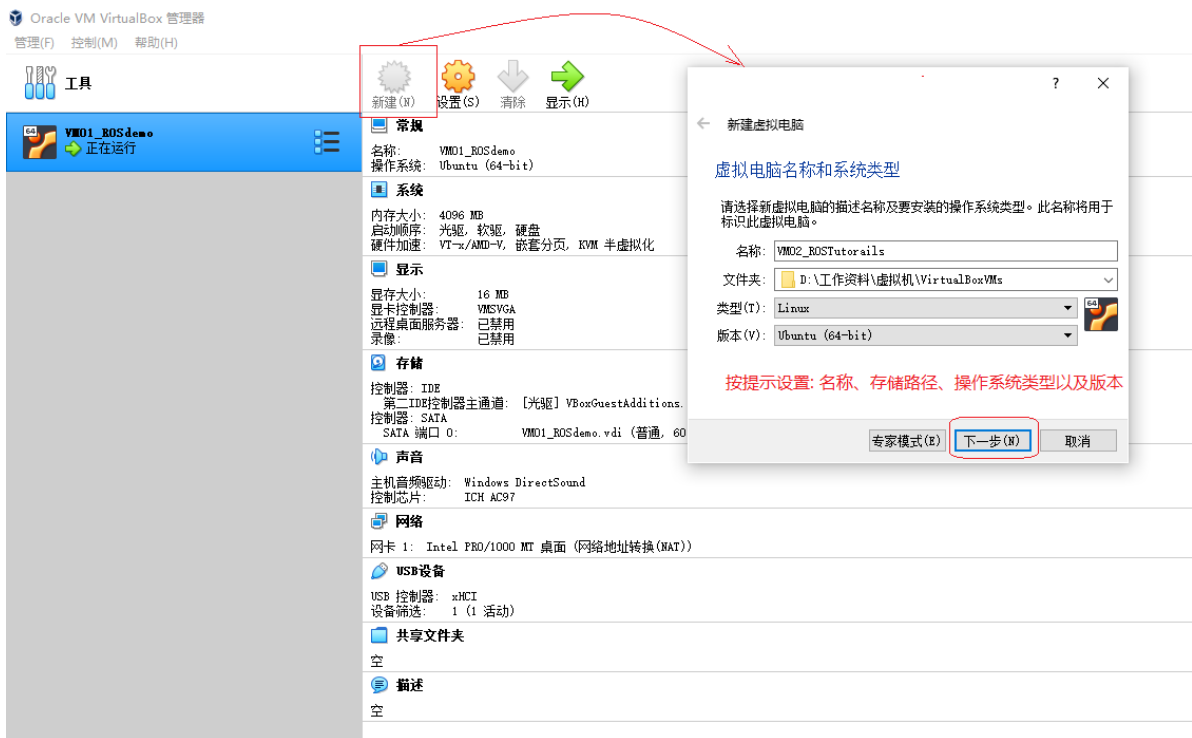
! You should only install driver software from publishers you trust. [How can I decide which device software is safe to install?](#)



安装完毕后，虚拟机已经可以正常启动了，接下来需要使用其虚拟出一台计算机

1.2.2 虚拟一台主机

使用 virtual 虚拟计算机的过程也不算复杂，只需要按照提示配置其相关参数即可



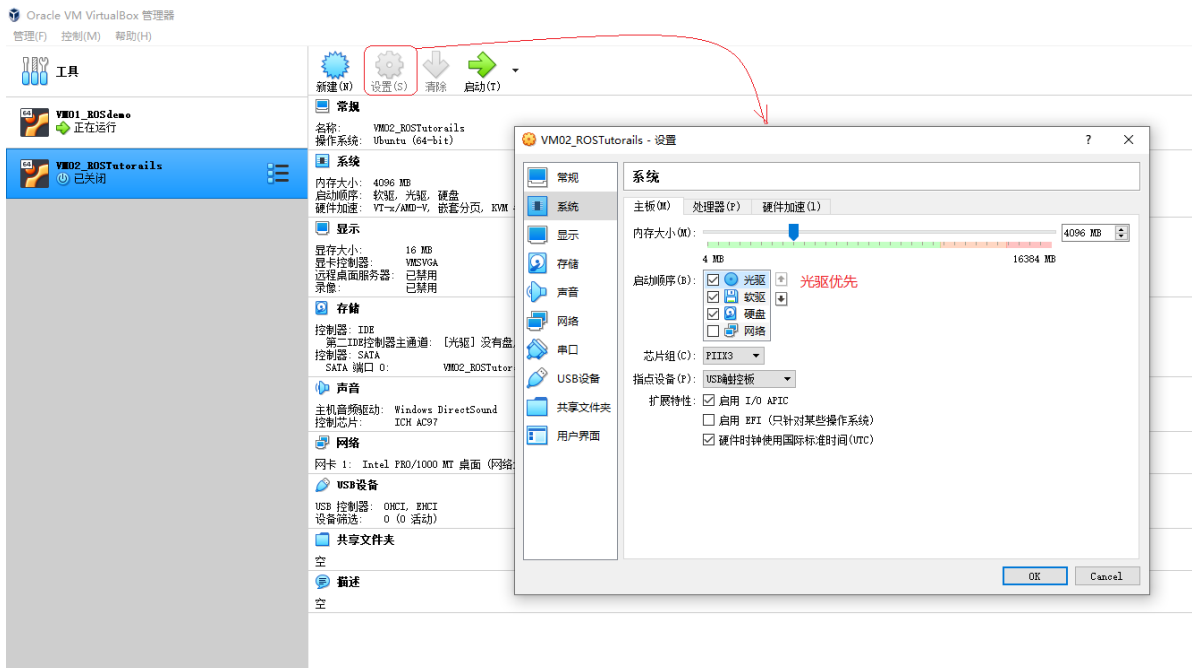


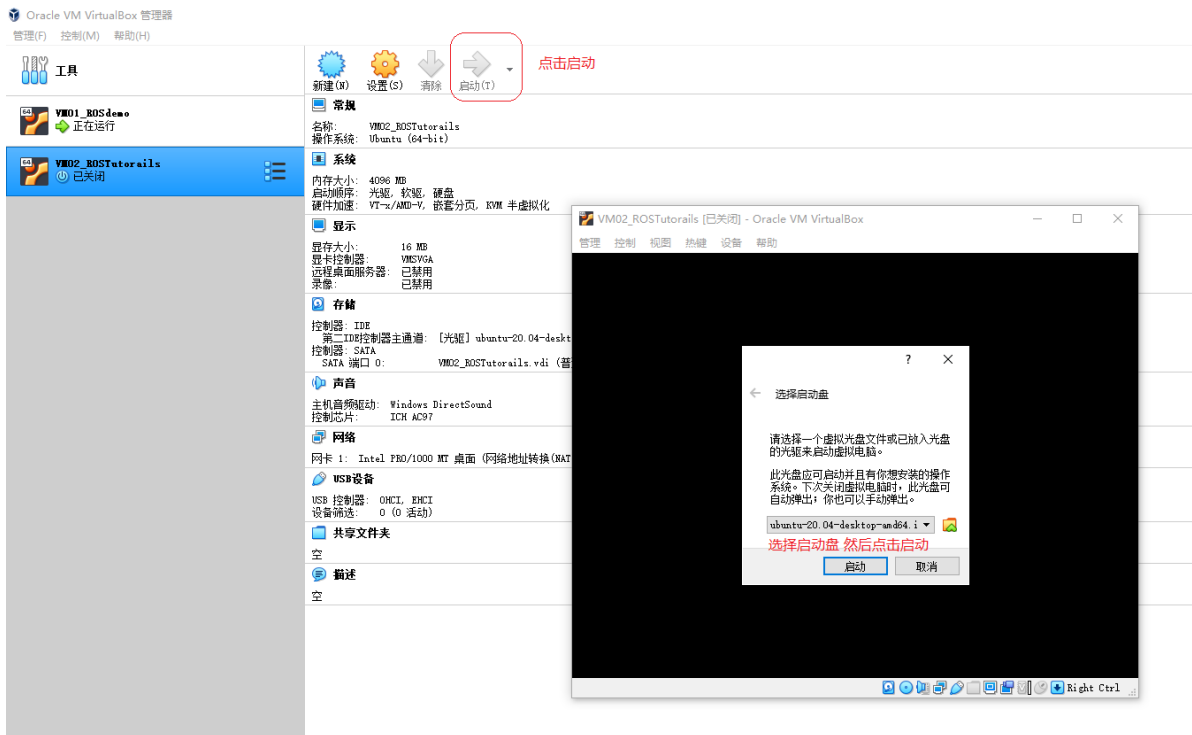
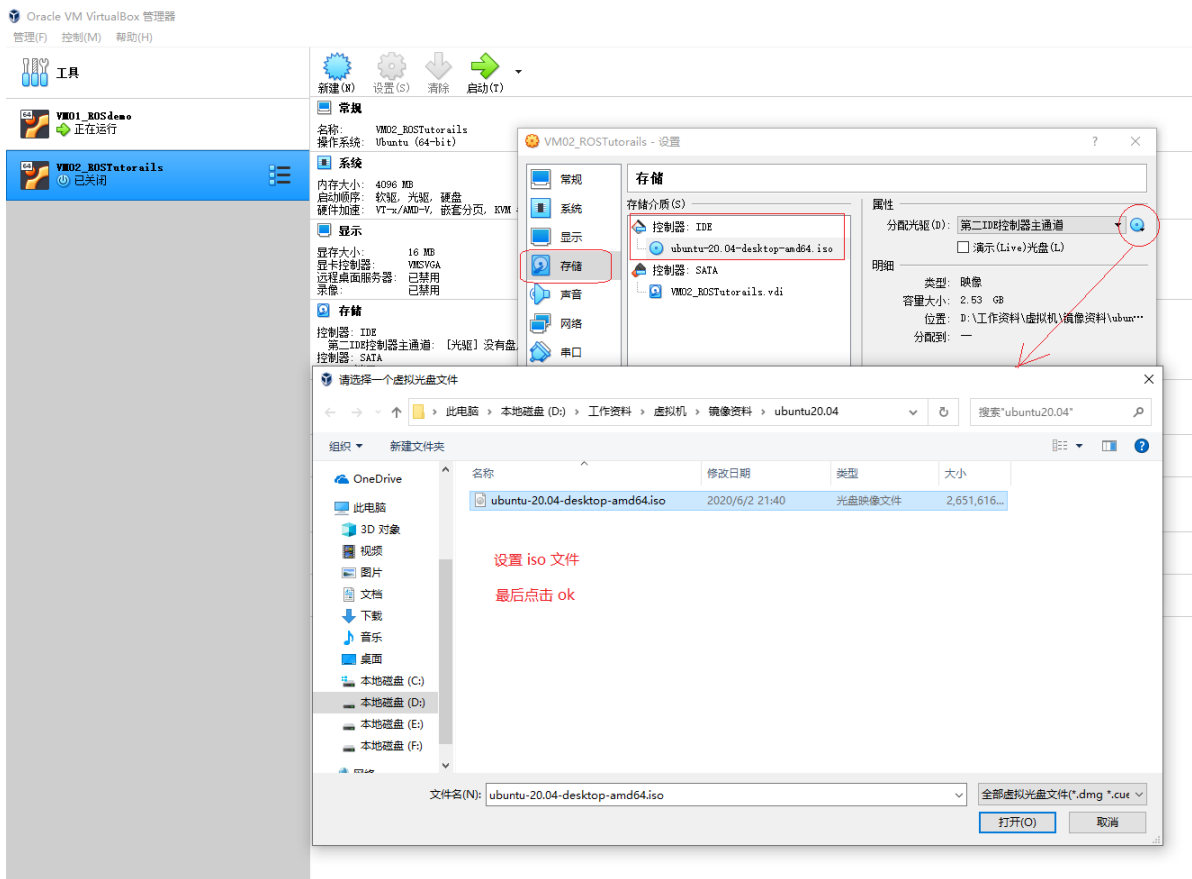
1.2.3 安装ubuntu

1.ubuntu安装

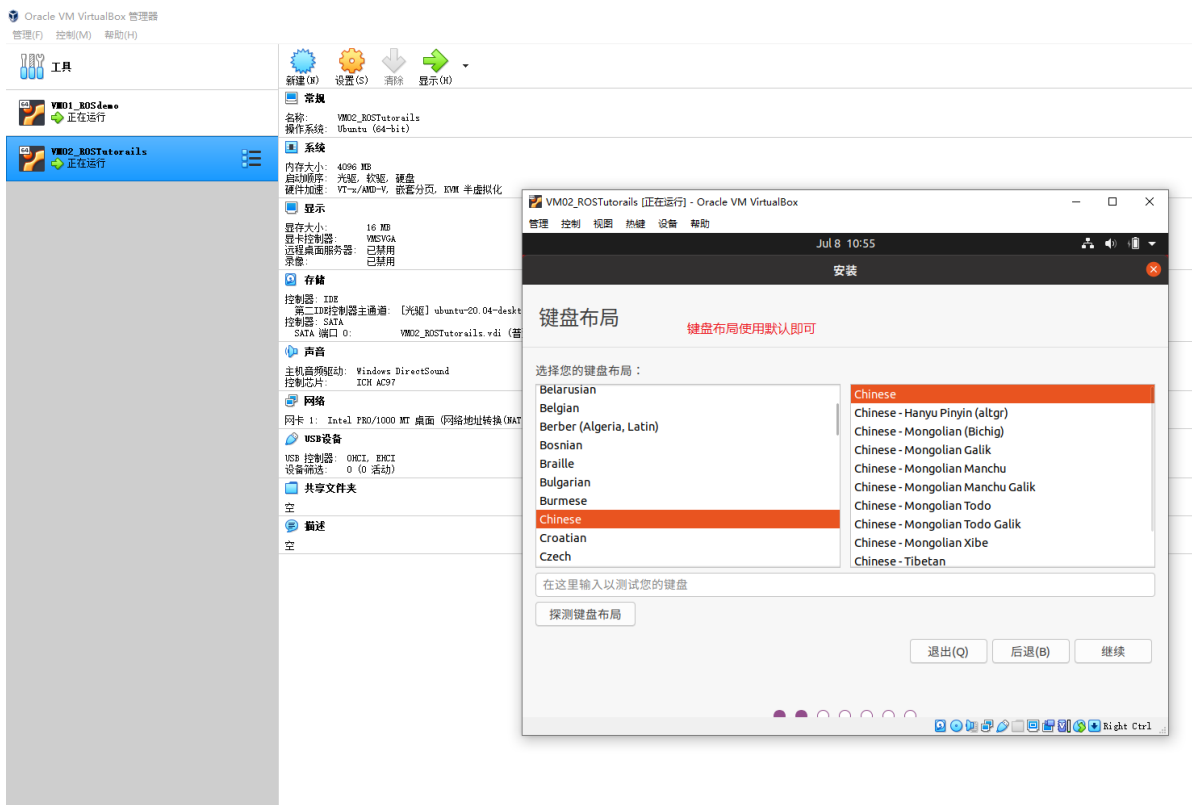
首先下载 Ubuntu 的镜像文件，链接如下：<http://mirrors.aliyun.com/ubuntu-releases/20.04/>;

然后，配置虚拟主机，关联 Ubuntu 镜像文件：





启动后，开始配置 ubuntu 操作系统：



安装过程中，断开网络连接，可以提升安装速度：

工具

VM01_ROSde...
正在运行

VM02_ROSTutorials
正在运行

新建(N) 设置(S) 清除 显示(O)

常规
名称: VM02_ROSTutorials
操作系统: Ubuntu (64-bit)

系统
内存大小: 4096 MB
启动顺序: 光驱, 软驱, 硬盘
硬件加速: VT-x/AMD-V, 嵌套分页, EXM 半虚拟化

显示
显存大小: 16 MB
显卡控制器: VMVGA
远程桌面服务器: 已禁用
录像: 已禁用

存储
控制器: IDE
第二IDE控制器主通道: [光驱] ubuntu-20.04-desktop-amd64.iso
控制器: SATA
SATA 端口 0: VM02_ROSTutorials.vdi (虚拟硬盘)

声音
主机音频驱动: Windows DirectSound
控制芯片: ICH AC97

网络
网卡 1: Intel PRO/1000 MT 桌面 (网络地址转换(NAT))

USB设备
USB 控制器: OHCI, EHCI
设备筛选: 0 (0 活动)

共享文件夹
空

描述
空



工具

VM01_ROSde...
正在运行

VM02_ROSTutorials
正在运行

新建(N) 设置(S) 清除 显示(O)

常规
名称: VM02_ROSTutorials
操作系统: Ubuntu (64-bit)

系统
内存大小: 4096 MB
启动顺序: 光驱, 软驱, 硬盘
硬件加速: VT-x/AMD-V, 嵌套分页, EXM 半虚拟化

显示
显存大小: 16 MB
显卡控制器: VMVGA
远程桌面服务器: 已禁用
录像: 已禁用

存储
控制器: IDE
第二IDE控制器主通道: [光驱] ubuntu-20.04-desktop-amd64.iso
控制器: SATA
SATA 端口 0: VM02_ROSTutorials.vdi (虚拟硬盘)

声音
主机音频驱动: Windows DirectSound
控制芯片: ICH AC97

网络
网卡 1: Intel PRO/1000 MT 桌面 (网络地址转换(NAT))

USB设备
USB 控制器: OHCI, EHCI
设备筛选: 0 (0 活动)

共享文件夹
空

描述
空



工具

VM01_ROSde... 正在运行

VM02_ROSTutorials 正在运行

新建(N) 设置(S) 清除 显示(O)

常规
名称: VM02_ROSTutorials
操作系统: Ubuntu (64-bit)

系统
内存大小: 4096 MB
启动顺序: 光驱, 软驱, 硬盘
硬件加速: VT-x/AMD-V, 嵌套分页, EXM 半虚拟化

显示
显存大小: 16 MB
显卡控制器: VMCI VGA
远程桌面服务器: 已禁用
录像: 已禁用

存储
控制器: IDE
第二IDE控制器主通道: [光驱] ubuntu-20.04-desktop.iso
控制器: SATA
SATA 端口 0: VM02_ROSTutorials.vdi (普通)

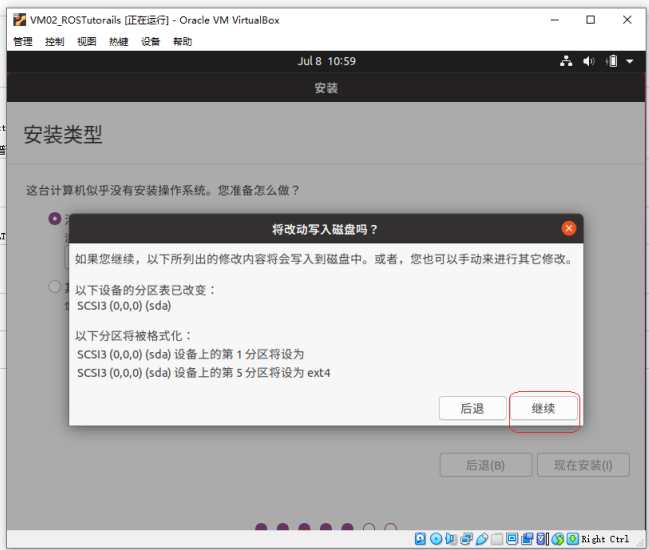
声音
主机音频驱动: Windows DirectSound
控制芯片: ICH AC97

网络
网卡 1: Intel PRO/1000 MT 桌面 (网络地址转换(NAT))

USB设备
USB 控制器: OHCI, EHCI
设备筛选: 0 (0 活动)

共享文件夹
空

描述
空



工具

VM01_ROSde... 正在运行

VM02_ROSTutorials 正在运行

新建(N) 设置(S) 清除 显示(O)

常规
名称: VM02_ROSTutorials
操作系统: Ubuntu (64-bit)

系统
内存大小: 4096 MB
启动顺序: 光驱, 软驱, 硬盘
硬件加速: VT-x/AMD-V, 嵌套分页, EXM 半虚拟化

显示
显存大小: 16 MB
显卡控制器: VMCI VGA
远程桌面服务器: 已禁用
录像: 已禁用

存储
控制器: IDE
第二IDE控制器主通道: [光驱] ubuntu-20.04-desktop.iso
控制器: SATA
SATA 端口 0: VM02_ROSTutorials.vdi (普通)

声音
主机音频驱动: Windows DirectSound
控制芯片: ICH AC97

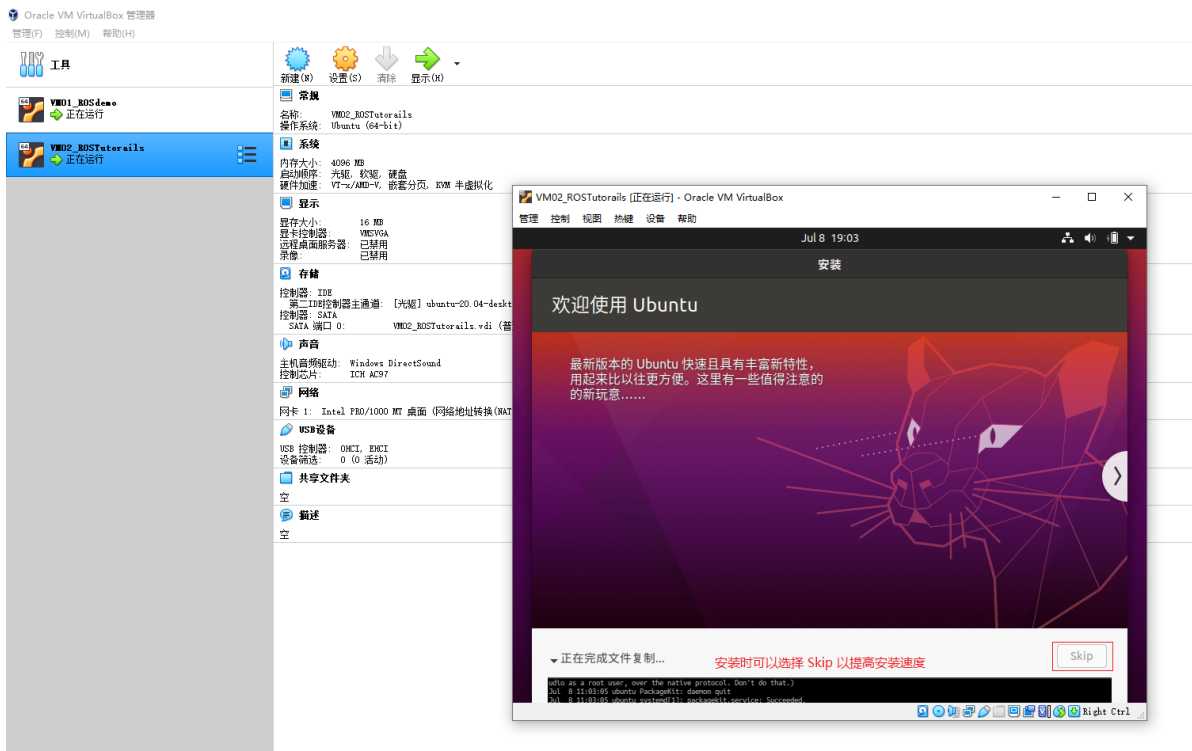
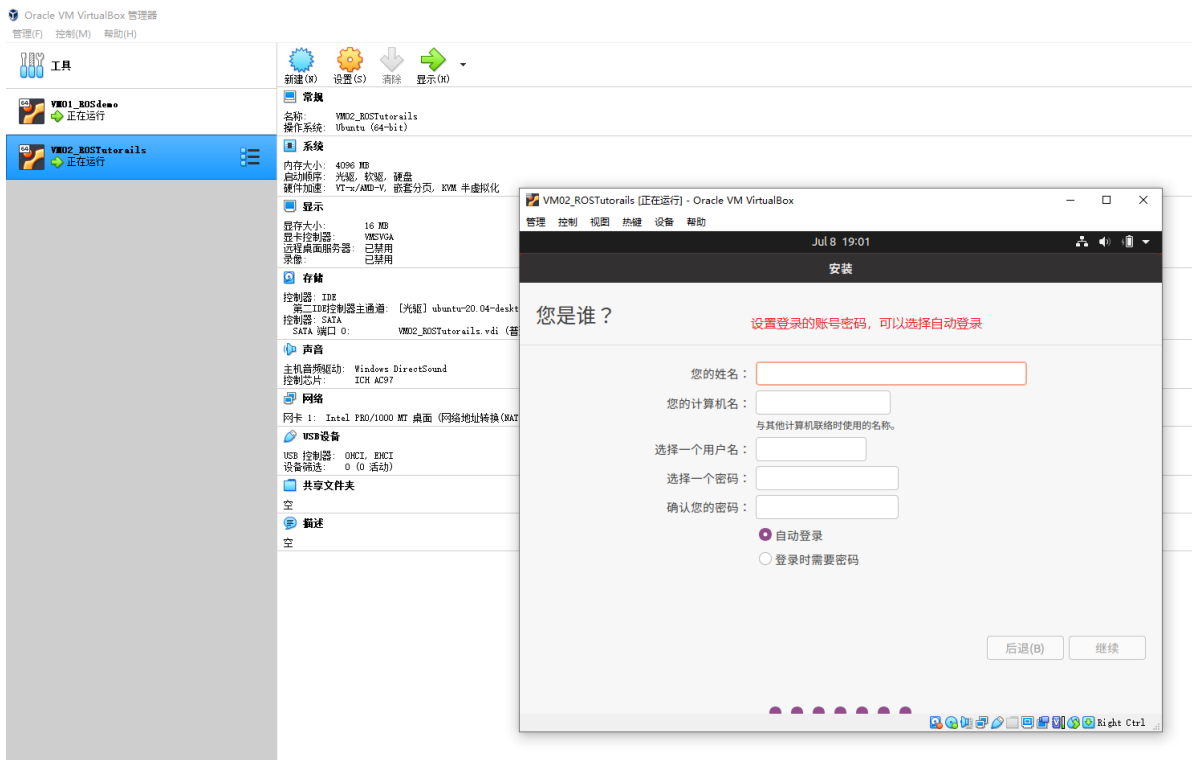
网络
网卡 1: Intel PRO/1000 MT 桌面 (网络地址转换(NAT))

USB设备
USB 控制器: OHCI, EHCI
设备筛选: 0 (0 活动)

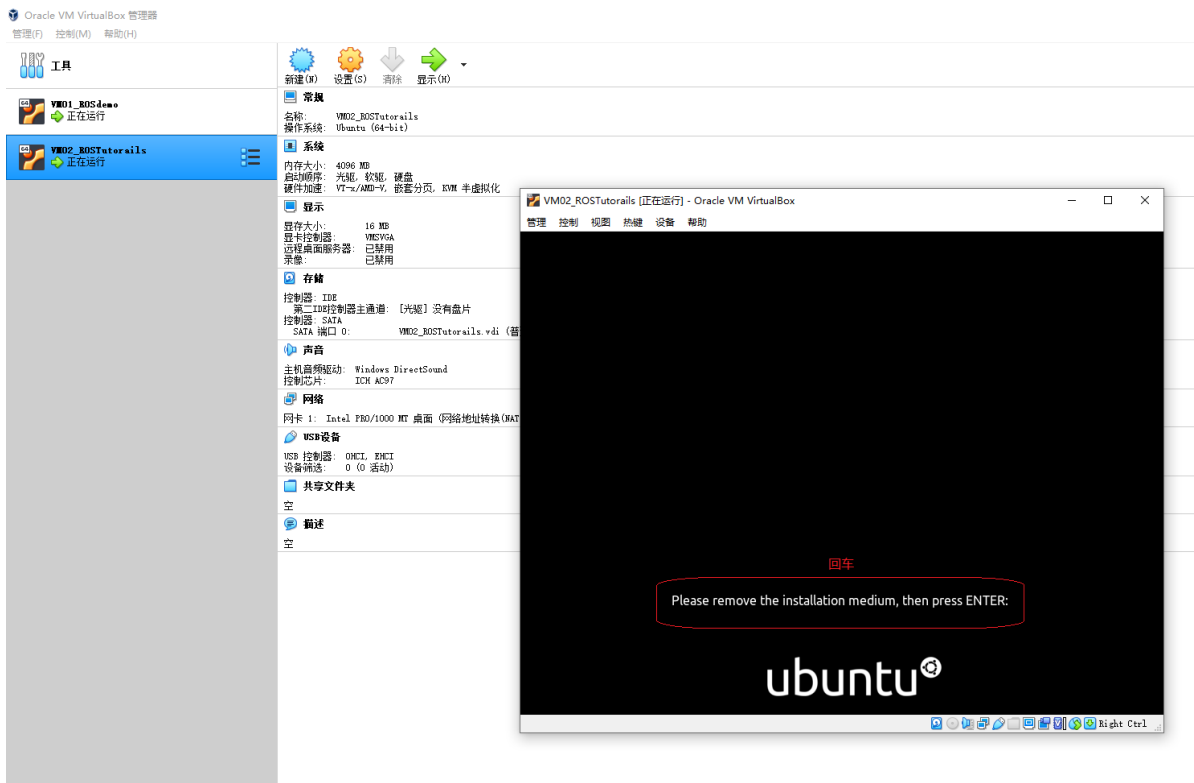
共享文件夹
空

描述
空





安装完毕后，会给出重启提示，点击重启确定按钮即可：



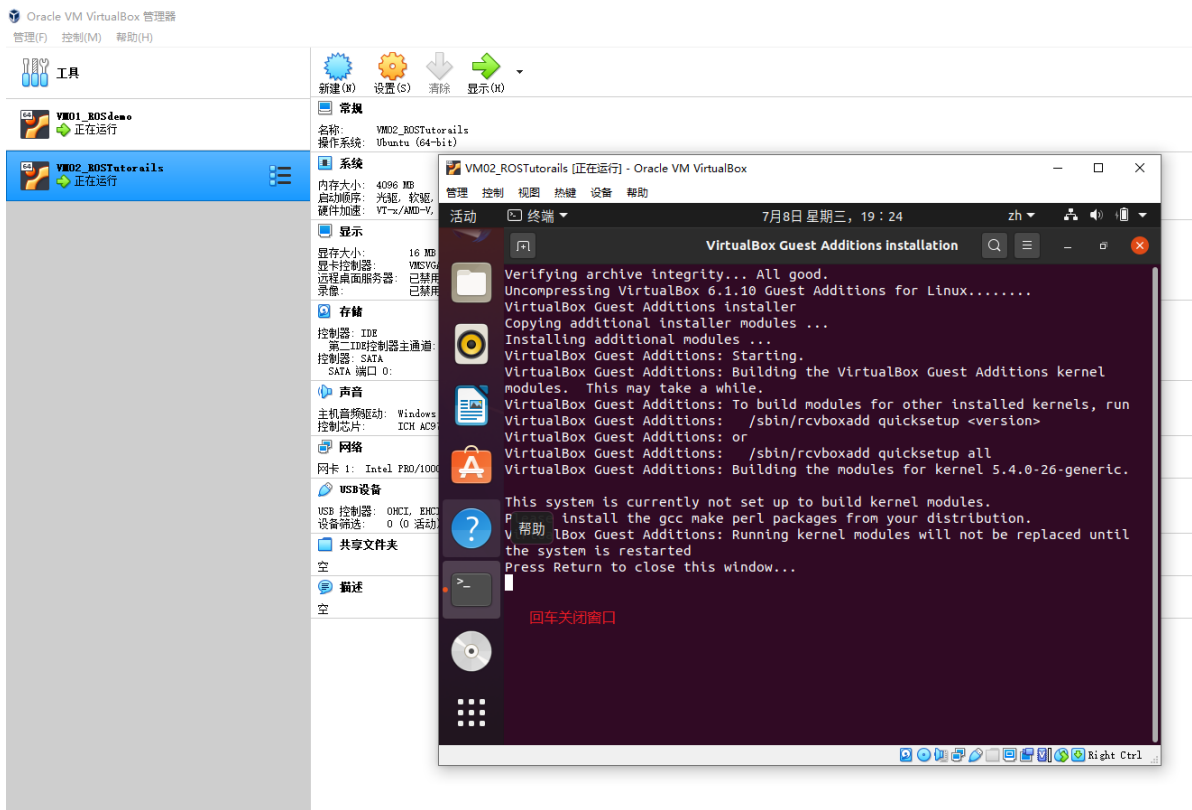
到目前为止 VirtualBox 已经正常安装了 ubuntu, 并启动成功。

2.使用优化

为了优化 ubuntu 操作的用户体验, 方便虚拟机与宿主机的文件交换以及 USB 设备的正常使用, 还需做如下操作:

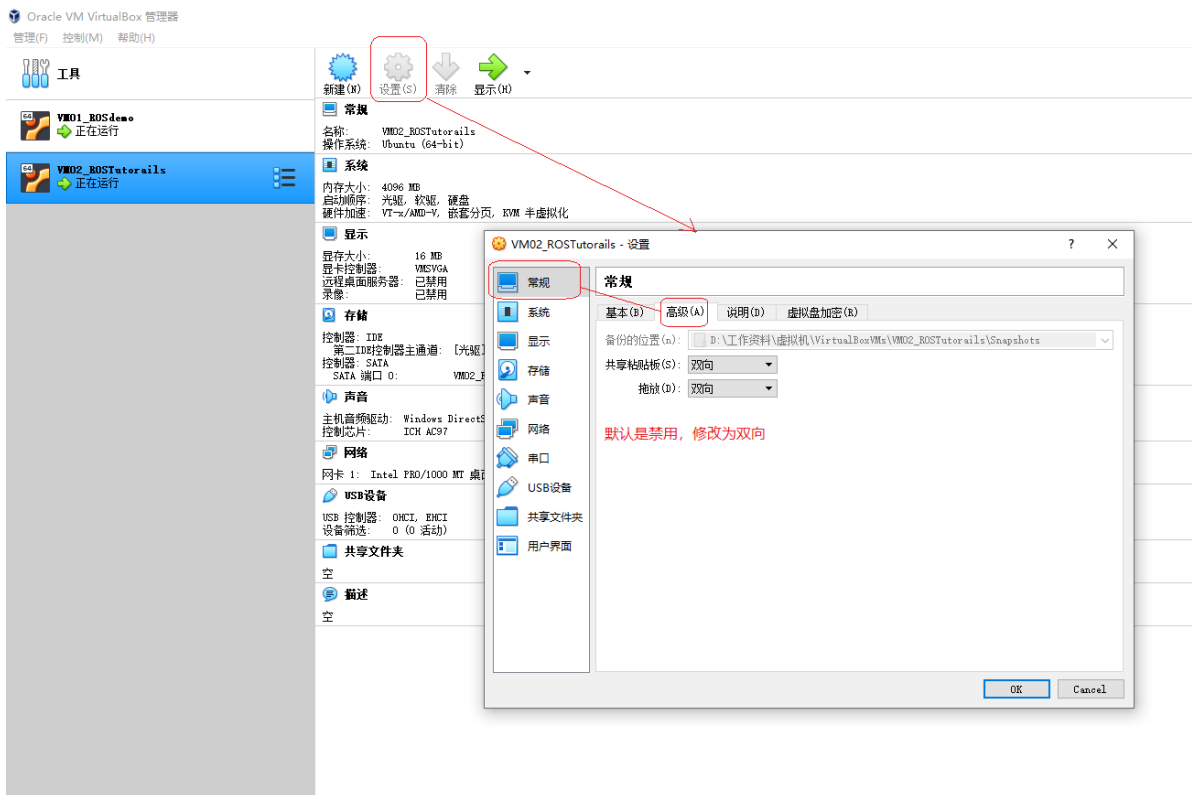
1.安装虚拟机工具





重启使之生效，选择菜单栏的自动调整窗口大小，然后ubuntu 桌面会自动使用窗口大小：右ctrl + F 全屏。

2.启动文件交换模式



3.安装扩展插件

先去 virtualbox 官网下载扩展包

virtualbox.org/wiki/Downloads

VirtualBox

Download VirtualBox

Here you will find links to VirtualBox binaries and its source code.

VirtualBox binaries

By downloading, you agree to the terms and conditions of the respective license.

If you're looking for the latest VirtualBox 6.0 packages, see [VirtualBox 6.0 builds](#). Please also use version 6.0 if you need to run VMs with software virtualiz:

If you're looking for the latest VirtualBox 5.2 packages, see [VirtualBox 5.2 builds](#). Please also use version 5.2 if you still need support for 32-bit hosts, as th

VirtualBox 6.1.10 platform packages

- Windows hosts
- OS X hosts
- Linux distributions
- Solaris hosts

按需选择

The binaries are released under the terms of the GPL version 2.

See the [changelog](#) for what has changed.

You might want to compare the checksums to verify the integrity of downloaded packages. *The SHA256 checksums should be favored as the MD5 algorithm*

- SHA256 checksums, MD5 checksums

Note: After upgrading VirtualBox it is recommended to upgrade the guest additions as well.

VirtualBox 6.1.10 Oracle VM VirtualBox Extension Pack

- All supported platforms

扩展插件也一并下载，后期会使用

Support for USB 2.0 and USB 3.0 devices, VirtualBox RDP, disk encryption, NVMe and PXE boot for Intel cards. See [this chapter from the User Manual](#) for a [VirtualBox Personal Use and Evaluation License \(PUEL\)](#). Please install the same version extension pack as your installed version of VirtualBox.

VirtualBox 6.1.10 Software Developer Kit (SDK)

- All platforms

在 virtual box 中添加扩展工具

Oracle VM VirtualBox 管理器

管理(F) 控制(M) 帮助(H)

工具 全局设定

新建(N) 设置(S) 清除 显示(H)

常规

名称: VM02_R0STutorials
操作系统: Ubuntu (64-bit)

系统

内存大小: 4096 MB
启动顺序: 光驱, 软驱, 硬盘
硬件加速: VT-x/AMD-V, 嵌套分页, SVM 半虚拟化

显示

显存大小: 16 MB
显卡控制器: VM SVGA
远程桌面服务器: 已禁用
录像: 已禁用

存储

控制器: IDE
第二 IDE 控制器主通道: [光驱] VBOX
控制器: SATA
SATA 端口 0: VM02_R0STu

声音

主机音频驱动: Windows DirectSound
控制芯片: ICH AC97

网络

网卡 1: Intel PRO/1000 MT 桌面 (网

USB 设备

USB 控制器: xHCI
设备筛选: 0 (0 活动)

共享文件夹

空

描述

空

VirtualBox - 全局设定

扩展

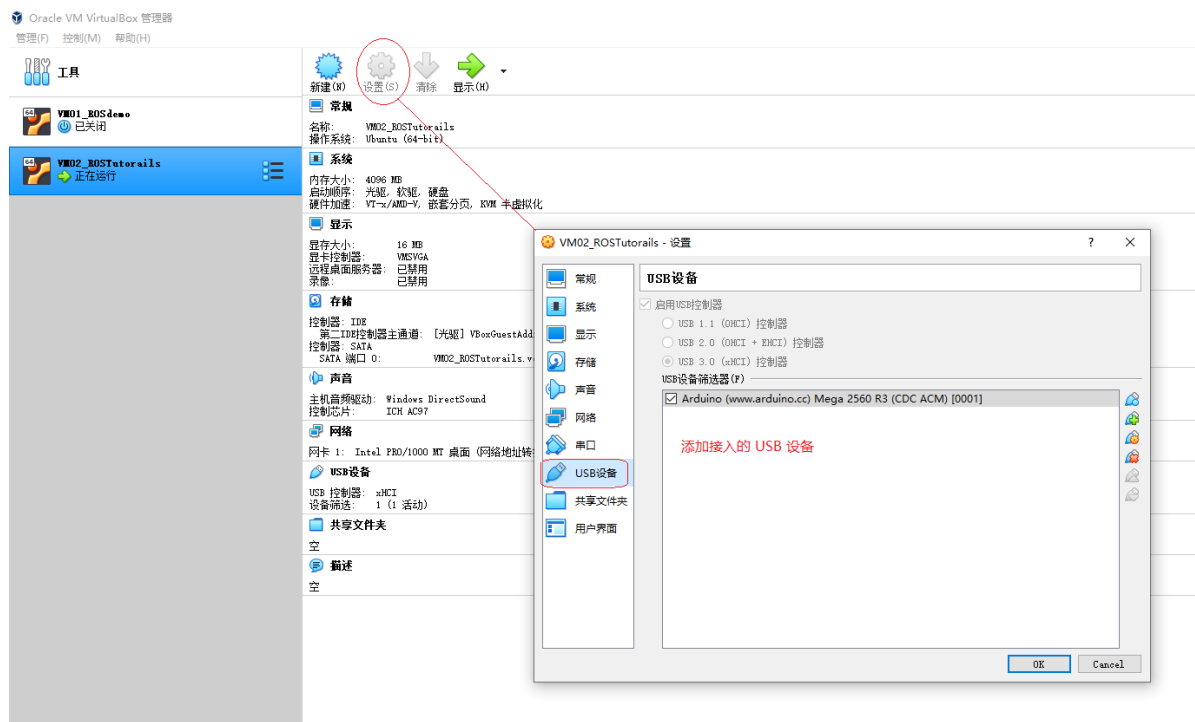
扩展包(E)

活动	名称	版本
✓	Oracle VM VirtualBox Extension Pack	6.1.10r138449

添加下载好的扩展工具

OK Cancel

在虚拟机中添加 USB 设备



重启后，使用 `ll /dev/ttyUSB*` 或 `ll /dev/ttyACM*` 即可查看新接入的设备。

4.其他

其他设置，比如输入法可以根据喜好自行下载安装。

ubuntu 20.04 鼠标右击没有创建文件选项，如果想要设置此选项，可以进入主目录下的模板目录，使用 gedit 创建一个空文本文件，以后，鼠标右击就可以添加新建文档选项，并且创建的文档与当前自定义的文档名称一致

1.2.4 安装 ROS

Ubuntu 安装完毕后，就可以安装 ROS 操作系统了，大致步骤如下：

1. 配置ubuntu的软件和更新；
2. 设置安装源；
3. 设置key；
4. 安装；
5. 配置环境变量。

1.配置ubuntu的软件和更新

配置ubuntu的软件和更新，允许安装不经认证的软件。

首先打开“软件和更新”对话框，具体可以在 Ubuntu 搜索按钮中搜索。

打开后按照下图进行配置（确保勾选了"restricted"， "universe, " 和 "multiverse."）



2.设置安装源

官方默认安装源:

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main"
> /etc/apt/sources.list.d/ros-latest.list'
Copy
```

或来自国内清华的安装源

```
sudo sh -c '. /etc/lsb-release && echo "deb
http://mirrors.tuna.tsinghua.edu.cn/ros/ubuntu/ `lsb_release -cs` main" >
/etc/apt/sources.list.d/ros-latest.list'
Copy
```

或来自国内中科大的安装源

```
sudo sh -c '. /etc/lsb-release && echo "deb
http://mirrors.ustc.edu.cn/ros/ubuntu/ `lsb_release -cs` main" >
/etc/apt/sources.list.d/ros-latest.list'
Copy
```

PS:

1. 回车后,可能需要输入管理员密码
2. 建议使用国内资源, 安装速度更快。

3.设置key

```
sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key
C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
Copy
```

4.安装

首先需要更新 apt(以前是 apt-get, 官方建议使用 apt 而非 apt-get),apt 是用于从互联网仓库搜索、安装、升级、卸载软件或操作系统的工具。

```
sudo apt update
Copy
```

等待...

然后, 再安装所需类型的 ROS:ROS 多个类型:**Desktop-Full**、**Desktop**、**ROS-Base**。这里介绍较为常用的Desktop-Full(官方推荐)安装: ROS, rqt, rviz, robot-generic libraries, 2D/3D simulators, navigation and 2D/3D perception

```
sudo apt install ros-noetic-desktop-full
Copy
```

等待.....(比较耗时)

友情提示: 由于网络原因,导致连接超时, 可能会安装失败, 如下所示:

```
错误:321 http://cn.archive.ubuntu.com/ubuntu focal/main amd64 libfreetype-dev amd64 2.10.1-2
504 Gateway Timeout [IP: 117.128.6.16 80]

错误:522 http://cn.archive.ubuntu.com/ubuntu focal/main amd64 libpcre32-3 amd64 2:8.39-12build1
无法发起与 cn.archive.ubuntu.com:80 (2001:67c:1562::18) 的连接 - connect (101: 网络不可达) 无法发起与

E: 无法下载 http://cn.archive.ubuntu.com/ubuntu/pool/universe/libv/libva/va-driver-all_2.7.0-2_amd64.deb
的连接 - connect (101: 网络不可达) 无法发起与 cn.archive.ubuntu.com:80 (2001:67c:1562::15) 的连接 - co
E: 无法下载 http://cn.archive.ubuntu.com/ubuntu/pool/main/libv/libvdpau/vdpau-driver-all_1.3-1ubuntu2_a
62::18) 的连接 - connect (101: 网络不可达) 无法发起与 cn.archive.ubuntu.com:80 (2001:67c:1562::15) 的连
E: 无法下载 http://cn.archive.ubuntu.com/ubuntu/pool/universe/p/proj/proj-bin_6.3.1-1_amd64.deb 无法发
connect (101: 网络不可达) 无法发起与 cn.archive.ubuntu.com:80 (2001:67c:1562::15) 的连接 - connect (10
E: 有几个软件包无法下载, 要不运行 apt-get update 或者加上 --fix-missing 的选项再试试?
```

可以多次重复调用 更新 和 安装命令, 直至成功。

5.配置环境变量

配置环境变量, 方便在任意 终端中使用 ROS。

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source ~/.bashrc
Copy
```

卸载

如果需要卸载ROS可以调用如下命令:

```
sudo apt remove ros-noetic-*
Copy
```

注意: 在 ROS 版本 noetic 中无需构建软件包的依赖关系, 没有 rosdep 的相关安装与配置。

另请参考: <http://wiki.ros.org/noetic/Installation/Ubuntu>。

后记

6.安装构建依赖

在 noetic 最初发布时，和其他历史版本稍有差异的是:没有安装构建依赖这一步骤。随着 noetic 不断完善，官方补齐了这一操作。

首先安装构建依赖的相关工具

```
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator  
python3-wstool build-essential  
Copy
```

ROS中使用许多工具前，要求需要初始化rosdep(可以安装系统依赖) -- 上一步实现已经安装过了。

```
sudo apt install python3-rosdep  
Copy
```

初始化rosdep

```
sudo rosdep init  
rosdep update  
Copy
```

如果一切顺利的话，rosdep 初始化与更新的打印结果如下:

```
rosnoetic@rosnoetic-VirtualBox:~$ sudo rosdep init  
Wrote /etc/ros/rosdep/sources.list.d/20-default.list  
Recommended: please run  
  
rosdep update
```

```
rosnoetic@rosnoetic-VirtualBox:~$ rosdep update  
Reading in sources list data from /etc/ros/rosdep/sources.list.d  
Hit https://gitee.com/zhao-xuzuo/rosdistro/raw/master/rosdep/osx-homebrew.yaml  
Hit https://gitee.com/zhao-xuzuo/rosdistro/raw/master/rosdep/base.yaml  
Hit https://gitee.com/zhao-xuzuo/rosdistro/raw/master/rosdep/python.yaml  
Hit https://gitee.com/zhao-xuzuo/rosdistro/raw/master/rosdep/ruby.yaml  
Hit https://gitee.com/zhao-xuzuo/rosdistro/raw/master/releases/fuerte.yaml  
Query rosdistro index https://gitee.com/zhao-xuzuo/rosdistro/raw/master/index-v4.yaml  
Skip end-of-life distro "ardent"  
Skip end-of-life distro "bouncy"  
Skip end-of-life distro "crystal"  
Add distro "dashing"  
Skip end-of-life distro "eloquent"  
Add distro "foxy"  
Skip end-of-life distro "groovy"  
Skip end-of-life distro "hydro"  
Skip end-of-life distro "indigo"  
Skip end-of-life distro "jade"  
Add distro "kinetic"  
Skip end-of-life distro "lunar"  
Add distro "melodic"  
Add distro "noetic"  
Add distro "rolling"  
Updated cache in /home/rosnoetic/.ros/rosdep/sources.cache
```

但是，在 rosdep 初始化时，多半会抛出异常。

问题:

```
rosnoetic@rosnoetic-VirtualBox:~$ sudo rosdep init
[sudo] rosnoetic 的密码:
ERROR: cannot download default sources list from:
https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/sources.list.d/20-default.list
Website may be down.
```

原因:

境外资源被屏蔽。

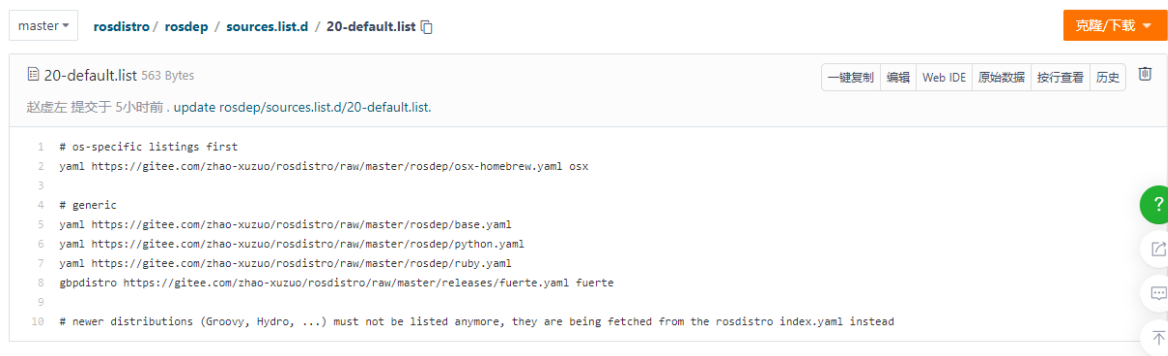
解决:

百度或google搜索, 解决方式有多种(<https://github.com/ros/rosdistro/issues/9721>), 可惜在ubuntu20.04下, 集体失效。

新思路: 将相关资源备份到gitee, 修改rosdep源码, 重新定位资源。

实现:

1. 先打开资源备份路径: <https://gitee.com/zhao-xuzuo/rosdistro>, 打开rosdistro/rosdep/sources.list.d/20-default.list文件留作备用(主要是复用URL的部分内容: gitee.com/zhao-xuzuo/rosdistro/raw/master)。



2. 进入"/usr/lib/python3/dist-packages/" 查找rosdep中和 raw.githubusercontent.com 相关的内容, 调用命令:

```
find . -type f | xargs grep "raw.githubusercontent.com"
Copy
```

```
rosnoetic@rosnoetic-VirtualBox:/usr/lib/python3/dist-packages$ find . -type f | xargs grep "raw.githubusercontent.com"
grep: ./setuptools/command/launcher: 没有那个文件或目录
grep: manifest.xml: 没有那个文件或目录
grep: ./setuptools/script: 没有那个文件或目录
grep: (dev).tmpl: 没有那个文件或目录
./requests-2.22.0.egg-info/PKG-INFO:      !![image](https://raw.githubusercontent.com/requests/requests/master/docs/_static/requests-logo-small.png)](http://docs.python-requests.org/)
匹配到二进制文件 ./rosdistro/manifest_provider/__pycache__/github.cpython-38.pyc
./rosdistro/manifest_provider/github.py:    url = 'https://raw.githubusercontent.com/%s/%s/package.xml' % (path, release_tag)
./rosdistro/manifest_provider/github.py:    url = 'https://raw.githubusercontent.com/%s/%s/%s' % \
./rosdistro/manifest_provider/github.py:    url = 'https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml'
匹配到二进制文件 ./rosdistro/__pycache__/__init__.cpython-38.pyc
匹配到二进制文件 ./rosdep2/__pycache__/rep3.cpython-38.pyc
匹配到二进制文件 ./rosdep2/__pycache__/sources_list.cpython-38.pyc
匹配到二进制文件 ./rosdep2/__pycache__/gbpdistro_support.cpython-38.pyc
./rosdep2/gbpdistro_support.py:FUERTE_GBPDISTRO_URL = 'https://raw.githubusercontent.com/ros/rosdistro/' \
./rosdep2/sources_list.py:DEFAULT_SOURCES_LIST_URL = 'https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/sources_list.d/20-default.list'
./rosdep2/rep3.py:REP3_TARGETS_URL = 'https://raw.githubusercontent.com/ros/rosdistro/master/releases/targets.yaml'
```

3. 修改相关文件, 主要有:

./rosdistro/init.py、./rosdep2/gbpdistro_support.py、./rosdep2/sources_list.py、./rosdep2/rep3.py。可以使用 `sudo gedit` 命令修改文件:

文件中涉及的 URL 内容, 如果是: raw.githubusercontent.com/ros/rosdistro/master 都替换成步骤1中准备的 gitee.com/zhao-xuzuo/rosdistro/raw/master 即可。

修改完毕, 再重新执行命令:

```
sudo rosdep init
rosdep update
Copy
```

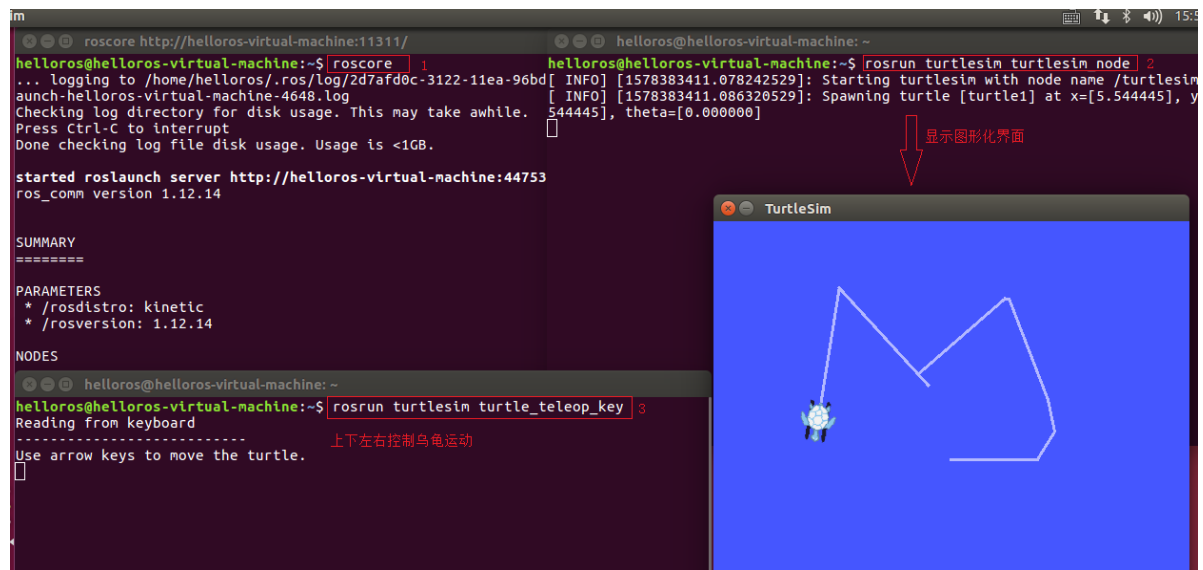
就可以正常实现 rosdep 的初始化与更新了。

1.2.5 测试 ROS

ROS 内置了一些小程序，可以通过运行这些小程序以检测 ROS 环境是否可以正常运行

1. 首先启动三个命令行(ctrl + alt + T)
2. 命令行1键入:**roscore**
3. 命令行2键入:**roslaunch turtlesim turtlesim_node**(此时会弹出图形化界面)
4. 命令行3键入:**roslaunch turtlesim turtlesim_teleop_key**(在3中可以通过上下左右控制2中乌龟的运动)

最终结果如下所示:

The screenshot shows a terminal window with three terminal sessions. The first session runs 'roscore' and shows logs for 'roscore' and 'roslaunch server'. The second session runs 'roslaunch turtlesim turtlesim_node' and shows logs for 'Starting turtlesim with node name /turtlesim' and 'Spawning turtle [turtle1]'. The third session runs 'roslaunch turtlesim turtlesim_teleop_key' and shows 'Reading from keyboard' and 'Use arrow keys to move the turtle.' To the right of the terminal is a 'TurtleSim' window showing a blue background with a white line representing a path and a small turtle icon at the start of the path. A red arrow points from the text '显示图形化界面' to the TurtleSim window.

注意：光标必须聚焦在键盘控制窗口，否则无法控制乌龟运动。

1.2.6 资料:其他ROS版本安装

我们的教程采用的是ROS的最新版本noetic，不过noetic较之于之前的ROS版本变动较大且部分功能包还未更新，因此如果有需要(比如到后期实践阶段，由于部分重要的功能包还未更新，需要ROS降级)，也会安装之前版本的ROS，在此，建议选用的版本是melodic或kinetic。

接下来，就以melodic为例演示ROS历史版本安装(当然先要准备与melodic对应的Ubuntu18.04):

1.配置ubuntu的软件和更新

首先打开“软件和更新”对话框，打开后按照下图进行配置（确保你的"restricted", "universe, " 和 "multiverse."前是打上勾的）

00ROS安装之ubuntu准备

2.安装源

官方默认安装源:

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main"
> /etc/apt/sources.list.d/ros-latest.list'
Copy
```

或来自国内中科大的安装源

```
sudo sh -c '. /etc/lsb-release && echo "deb
http://mirrors.ustc.edu.cn/ros/ubuntu/ `lsb_release -cs` main" >
/etc/apt/sources.list.d/ros-latest.list'
Copy
```

或来自国内清华的安装源

```
sudo sh -c '. /etc/lsb-release && echo "deb
http://mirrors.tuna.tsinghua.edu.cn/ros/ubuntu/ `lsb_release -cs` main" >
/etc/apt/sources.list.d/ros-latest.list'
Copy
```

PS:回车后,可能需要输入管理员密码

3.设置key

```
sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key
C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
Copy
```

4.安装

首先需要更新 apt(以前是 apt-get, 官方建议使用 apt 而非 apt-get),apt 是用于从互联网仓库搜索、安装、升级、卸载软件或操作系统的工具。

```
sudo apt update
Copy
```

等待...

然后,再安装所需类型的 ROS:ROS 多个类型:**Desktop-Full**、**Desktop**、**ROS-Base**。这里介绍较为常用的Desktop-Full(官方推荐)安装: ROS, rqt, rviz, robot-generic libraries, 2D/3D simulators, navigation and 2D/3D perception

```
sudo apt install ros-melodic-desktop-full
Copy
```

等待...

5.环境设置

配置环境变量,方便在任意 终端中使用 ROS。

```
echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc
source ~/.bashrc
Copy
```

6.安装构建依赖

首先安装构建依赖的相关工具

```
sudo apt install python-rosdep python-rosinstall python-rosinstall-generator  
python-wstool build-essential  
Copy
```

然后安装rosdep(可以安装系统依赖)

```
sudo apt install python-rosdep  
Copy
```

初始化rosdep

```
sudo rosdep init  
rosdep update  
Copy
```

注意:

当执行到最后 `sudo rosdep init` 是, 可能会抛出异常;

错误提示:

ERROR: cannot download default sources list from:

<https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/sources.list.d/20-default.list>

Website may be down.

原因:

境外资源被屏蔽

解决思路:

查询错误提示中域名的IP地址, 然后修改 `/etc/hosts` 文件, 添加域名与IP映射

实现:

1.访问域名查询网址:<https://site.ip138.com/>

2.查询域名ip, 搜索框中输入: `raw.githubusercontent.com`, 自由复制一个查询到的IP

iP或域名查询

raw.githubusercontent.com

X

查询

访问

域名注册com42元

top10元 cn18元 xyz12元

广告QQ:1073353388



买卖域名, 网站, 自媒体
上中介网!

广告

域名被劫持? DNS被污染?

全面检测 解决DNS劫持

广告

iP	子域名	备案	Whois	快照
raw.githubusercontent.com服务器iP:				
当前解析:				
151.101.108.133				日本 东京
151.101.0.133				美国
0.0.0.0				保留地址
151.101.228.133				日本 东京
151.101.196.133				美国 加利福尼亚 洛杉矶
151.101.8.133				新加坡
151.101.76.133				中国 香港
151.101.24.133				美国 加利福尼亚 洛杉矶

3.修改 /etc/hosts 文件, 命令:

```
sudo gedit /etc/hosts
Copy
```

添加内容:151.101.76.133 raw.githubusercontent.com (查询到的ip与域名), 保存并退出。

```
打开(O) ▾ [icon]
127.0.0.1      localhost
127.0.1.1      rosmelodic-VirtualBox
151.101.76.133 raw.githubusercontent.com

# The following lines are desirable for IPv6 capable hosts
::1          ip6-localhost ip6-loopback
fe00::0      ip6-localnet
ff00::0      ip6-mcastprefix
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters
```

或者, 也可以使用 vi 或 vim 修改。

4.重新执行rosdep初始化与更新命令，如果rosdep update 抛出异常，基本都是网络原因导致的(建议使用手机热点)，多尝试几次即可。

```
rosmelodic@rosmelodic-VirtualBox:~$ sudo rosdep init
Wrote /etc/ros/rosdep/sources.list.d/20-default.list
Recommended: please run

rosdep update
```

```
rosmelodic@rosmelodic-VirtualBox:~$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Add distro "dashing"
Add distro "eloquent"
Add distro "foxy"
Skip end-of-life distro "groovy"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "jade"
Add distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "melodic"
Add distro "noetic"
Add distro "rolling"
updated cache in /home/rosmelodic/.ros/rosdep/sources.cache
```

综上，历史版本的安装与noetic流程类似，只是多出了“构建功能包依赖关系”的步骤。

另请参考：<http://wiki.ros.org/melodic/Installation/Ubuntu>

1.3 ROS快速体验

编写 ROS 程序，在控制台输出文本: Hello World，分别使用 C++ 和 Python 实现。

1.3.1 HelloWorld实现简介

ROS中涉及的编程语言以C++和Python为主，ROS中的大多数程序两者都可以实现，在本系列教程中，每一个案例也都会分别使用C++和Python两种方案演示，大家可以根据自身情况选择合适的实现方案。

ROS中的程序即便使用不同的编程语言，实现流程也大致类似，以当前HelloWorld程序为例，实现流程大致如下：

1. 先创建一个工作空间；
2. 再创建一个功能包；
3. 编辑源文件；
4. 编辑配置文件；
5. 编译并执行。

上述流程中，C++和Python只是在步骤3和步骤4的实现细节上存在差异，其他流程基本一致。本节先实现C++和Python程序编写的通用部分步骤1与步骤2，1.3.2节和1.3.3节再分别使用C++和Python编写HelloWorld。

1.创建工作空间并初始化

```
mkdir -p 自定义空间名称/src
cd 自定义空间名称
catkin_make
Copy
```

上述命令，首先会创建一个工作空间以及一个 src 子目录，然后再进入工作空间调用 catkin_make 命令编译。

2.进入 src 创建 ros 包并添加依赖

```
cd src
catkin_create_pkg 自定义ROS包名 roscpp rospy std_msgs
Copy
```

上述命令，会在工作空间下生成一个功能包，该功能包依赖于 roscpp、rospy 与 std_msgs，其中 roscpp 是使用 C++ 实现的库，而 rospy 则是使用 python 实现的库，std_msgs 是标准消息库，创建 ROS 功能包时，一般都会依赖这三个库实现。

注意: 在 ROS 中，虽然实现同一功能时，C++ 和 Python 可以互换，但是具体选择哪种语言，需要视需求而定，因为两种语言相较而言：C++ 运行效率高但是编码效率低，而 Python 则反之，基于二者互补的特点，ROS 设计者分别设计了 roscpp 与 rospy 库，前者旨在成为 ROS 的高性能库，而后者则一般用于对性能无要求的场景，旨在提高开发效率。

1.3.2 HelloWorld(C++版)

本节内容基于 1.3.1，假设你已经创建了 ROS 的工作空间，并且创建了 ROS 的功能包，那么就可以进入核心步骤了，使用 C++ 编写程序实现：

1.进入 ros 包的 src 目录编辑源文件

```
cd 自定义的包
Copy
```

C++ 源码实现(文件名自定义)

```
#include "ros/ros.h"

int main(int argc, char *argv[])
{
    //执行 ros 节点初始化
    ros::init(argc, argv, "hello");
    //创建 ros 节点句柄(非必须)
    ros::NodeHandle n;
    //控制台输出 hello world
    ROS_INFO("hello world!");

    return 0;
}
Copy
```

2.编辑 ros 包下的 Cmakelist.txt文件

```
add_executable(步骤3的源文件名
    src/步骤3的源文件名.cpp
)
target_link_libraries(步骤3的源文件名
    ${catkin_LIBRARIES}
)
Copy
```

3.进入工作空间目录并编译

```
cd 自定义空间名称
catkin_make
Copy
```

生成 build devel

4.执行

先启动命令行1:

```
roscore
Copy
```

再启动命令行2:

```
cd 工作空间
source ./devel/setup.bash
roslaunch 包名 C++节点
Copy
```

命令行输出: HelloWorld!

PS: `source ~/工作空间/devel/setup.bash` 可以添加进 `.bashrc` 文件, 使用上更方便

添加方式1: 直接使用 `gedit` 或 `vi` 编辑 `.bashrc` 文件, 最后添加该内容

添加方式2: `echo "source ~/工作空间/devel/setup.bash" >> ~/.bashrc`

1.3.3 HelloWorld(Python版)

本节内容基于1.3.1, 假设你已经创建了ROS的工作空间, 并且创建了ROS的功能包, 那么就可以进入核心步骤了, 使用Python编写程序实现:

1.进入 ros 包添加 scripts 目录并编辑 python 文件

```
cd ros包
mkdir scripts
Copy
```

新建 python 文件: (文件名自定义)

```
#!/usr/bin/env python

"""
    Python 版 HelloWorld

"""

import rospy

if __name__ == "__main__":
    rospy.init_node("Hello")
    rospy.loginfo("Hello world!!!!")
Copy
```

2.为 python 文件添加可执行权限

```
chmod +x 自定义文件名.py
Copy
```

3.编辑 ros 包下的 CamkeList.txt 文件

```
catkin_install_python(PROGRAMS scripts/自定义文件名.py
    DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
Copy
```

4.进入工作空间目录并编译

```
cd 自定义空间名称
catkin_make
Copy
```

5.进入工作空间目录并执行

先启动命令行1:

```
roscore
Copy
```

再启动命令行2:

```
cd 工作空间
source ./devel/setup.bash
roslaunch 包名 自定义文件名.py
Copy
```

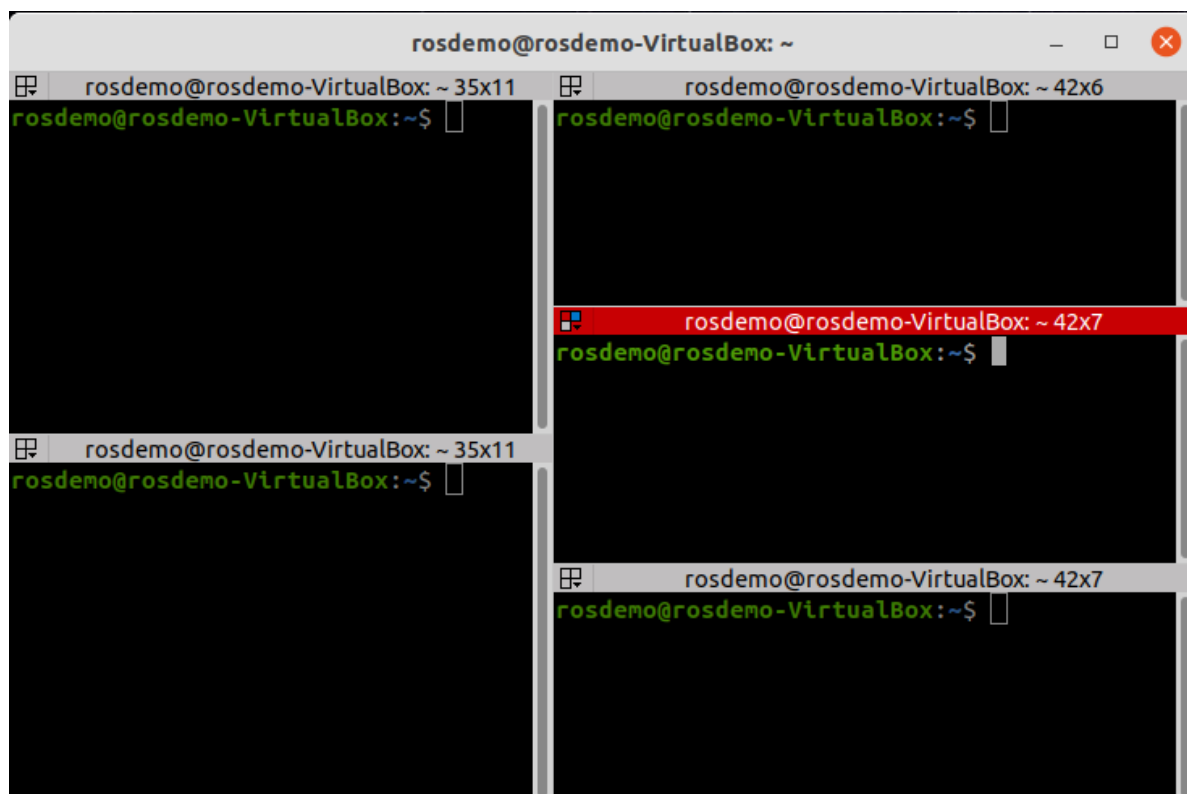
输出结果:Hello world!!!!

1.4 ROS集成开发环境搭建

和大多数开发环境一样，理论上，在 ROS 中，只需要记事本就可以编写基本的 ROS 程序，但是工欲善其事必先利其器，为了提高开发效率，可以先安装集成开发工具和使用方便的工具:终端、IDE....

1.4.1 安装终端

在 ROS 中，需要频繁的使用到终端，且可能需要同时开启多个窗口，推荐一款较为好用的终端:Terminator。效果如下:



1.安装

```
sudo apt install terminator
```

Copy

2.添加到收藏夹

显示应用程序 ---> 搜索 terminator ---> 右击 选择 添加到收藏夹

3.Terminator 常用快捷键

第一部份：关于在同一个标签内的操作

Alt+Up	//移动到上面的终端
Alt+Down	//移动到下面的终端
Alt+Left	//移动到左边的终端
Alt+Right	//移动到右边的终端
Ctrl+Shift+O	//水平分割终端
Ctrl+Shift+E	//垂直分割终端
Ctrl+Shift+Right	//在垂直分割的终端中将分割条向右移动
Ctrl+Shift+Left	//在垂直分割的终端中将分割条向左移动
Ctrl+Shift+Up	//在水平分割的终端中将分割条向上移动
Ctrl+Shift+Down	//在水平分割的终端中将分割条向下移动
Ctrl+Shift+S	//隐藏/显示滚动条
Ctrl+Shift+F	//搜索
Ctrl+Shift+C	//复制选中的内容到剪贴板
Ctrl+Shift+V	//粘贴剪贴板的内容到此处
Ctrl+Shift+W	//关闭当前终端
Ctrl+Shift+Q	//退出当前窗口，当前窗口的所有终端都将被关闭

```
Ctrl+Shift+X //最大化显示当前终端
Ctrl+Shift+Z //最大化显示当前终端并使字体放大
Ctrl+Shift+N or Ctrl+Tab //移动到下一个终端
Ctrl+Shift+P or Ctrl+Shift+Tab //Ctrl+Shift+Tab 移动到之前的一个终端
Copy
```

第二部份：有关各个标签之间的操作

```
F11 //全屏开关
Ctrl+Shift+T //打开一个新的标签
Ctrl+PageDown //移动到下一个标签
Ctrl+PageUp //移动到上一个标签
Ctrl+Shift+PageDown //将当前标签与其后一个标签交换位置
Ctrl+Shift+PageUp //将当前标签与其前一个标签交换位置
Ctrl+Plus (+) //增大字体
Ctrl+Minus (-) //减小字体
Ctrl+Zero (0) //恢复字体到原始大小
Ctrl+Shift+R //重置终端状态
Ctrl+Shift+G //重置终端状态并clear屏幕
Super+g //绑定所有的终端，以便向一个输入能够输入到所有的终端
Super+Shift+G //解除绑定
Super+t //绑定当前标签的所有终端，向一个终端输入的内容会自动输入到
其他终端
Super+Shift+T //解除绑定
Ctrl+Shift+I //打开一个窗口，新窗口与原来的窗口使用同一个进程
Super+i //打开一个新窗口，新窗口与原来的窗口使用不同的进程
```

1.4.2 安装VScode

VSCode 全称 Visual Studio Code，是微软出的一款轻量级代码编辑器，免费、开源而且功能强大。它支持几乎所有主流的程序语言的语法高亮、智能代码补全、自定义热键、括号匹配、代码片段、代码对比 Diff、GIT 等特性，支持插件扩展，并针对网页开发和云端应用开发做了优化。软件跨平台支持 Win、Mac 以及 Linux。

1. 下载

vscode 下载:<https://code.visualstudio.com/docs?start=true>

历史版本下载链接: <https://code.visualstudio.com/updates>

2. vscode 安装与卸载

2.1 安装

方式1:双击安装即可(或右击选择安装)

方式2: `sudo dpkg -i xxxx.deb`

2.2 卸载

```
sudo dpkg --purge code
Copy
```

3.vscode 集成 ROS 插件

使用 VScode 开发 ROS 程序，需要先安装一些插件，常用插件如下：



4.vscode 使用_基本配置

4.1 创建 ROS 工作空间

```
mkdir -p xxx_ws/src(必须得有 src)
cd xxx_ws
catkin_make
Copy
```

4.2 启动 vscode

进入 xxx_ws 启动 vscode

```
cd xxx_ws
code .
Copy
```

4.3 vscode 中编译 ros

快捷键 ctrl + shift + B 调用编译，选择: `catkin_make:build`

可以点击配置设置为默认，修改.vscode/tasks.json 文件

```
{
  // 有关 tasks.json 格式的文档，请参见
  // https://go.microsoft.com/fwlink/?LinkId=733558
  "version": "2.0.0",
  "tasks": [
    {
      "label": "catkin_make:debug", //代表提示的描述性信息
      "type": "shell", //可以选择shell或者process,如果是shell代码是在shell里面
      //运行一个命令，如果是process代表作为一个进程来运行
      "command": "catkin_make", //这个是我们需要运行的命令
      "args": [], //如果需要在命令后面加一些后缀，可以写在这里，比如-
      "DCATKIN_WHITELIST_PACKAGES="pac1;pac2"
      "group": {"kind": "build", "isDefault": true},
      "presentation": {
        "reveal": "always" //可选always或者silence，代表是否输出信息
      },
      "problemMatcher": "$msCompile"
    }
  ]
}
Copy
```

4.4 创建 ROS 功能包

选定 src 右击 ---> create catkin package

设置包名 添加依赖



4.5 C++ 实现

在功能包的 src 下新建 cpp 文件

```
/*
    控制台输出 HelloVSCode !!!
*/
#include "ros/ros.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    //执行节点初始化
    ros::init(argc, argv, "HelloVSCode");

    //输出日志
    ROS_INFO("Hello VSCode!!!哈哈哈哈哈");
    return 0;
}
Copy
```

PS1: 如果没有代码提示

修改 .vscode/c_cpp_properties.json

设置 "cppStandard": "c++17"

PS2: main 函数的参数不可以被 const 修饰

PS3: 当ROS_INFO 终端输出有中文时，会出现乱码

[INFO](#): ??????????????????????

解决办法：在函数开头加入下面代码的任意一句

```
setlocale(LC_CTYPE, "zh_CN.utf8");
setlocale(LC_ALL, "");
Copy
```

4.6 python 实现

在 功能包 下新建 scripts 文件夹，添加 python 文件，并添加可执行权限

```
#!/usr/bin/env python
"""
    Python 版本的 HelloVSCode，执行在控制台输出 HelloVSCode
    实现：
    1. 导包
    2. 初始化 ROS 节点
    3. 日志输出 HelloVSCode
"""

import rospy # 1. 导包

if __name__ == "__main__":
```

```
rospy.init_node("Hello_vscode_p") # 2.初始化 ROS 节点
rospy.loginfo("Hello vscode, 我是 Python ....") #3.日志输出 HelloWorld
Copy
```

4.7 配置 CMakeLists.txt

C++ 配置:

```
add_executable(节点名称
    src/C++源文件名.cpp
)
target_link_libraries(节点名称
    ${catkin_LIBRARIES}
)
Copy
```

Python 配置:

```
catkin_install_python(PROGRAMS scripts/自定义文件名.py
    DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
Copy
```

4.8 编译执行

编译: ctrl + shift + B

执行: 和之前一致, 只是可以在 VScode 中添加终端, 首先执行: `source ./devel/setup.bash`

 08vscode\执行PS:

如果不编译直接执行 python 文件, 会抛出异常。

- 1.第一行解释器声明, 可以使用绝对路径定位到 python3 的安装路径 `#!/usr/bin/python3`, 但是不建议
- 2.建议使用 `#!/usr/bin/env python` 但是会抛出异常: `/usr/bin/env: "python": 没有那个文件或目录`
- 3.解决1: `#!/usr/bin/env python3` 直接使用 python3 但存在问题: 不兼容之前的 ROS 相关 python 实现
- 4.解决2: 创建一个链接符号到 python 命令: `sudo ln -s /usr/bin/python3 /usr/bin/python`

5.其他 IDE

ROS 开发可以使用的 IDE 还是比较多的, 除了上述的 VScode, 还有 Eclipse、QT、PyCharm、Roboware, 详情可以参考官网介绍: <http://wiki.ros.org/IDEs>

QT Creator Plugin for ROS, 参考教程: <https://ros-qtc-plugin.readthedocs.io/en/latest/>

Roboware 参考: <http://www.roboware.me/#/> (PS: Roboware 已经停更了, 可惜....)

1.4.3 launch文件演示

1.需求

一个程序中可能需要启动多个节点，比如:ROS 内置的小乌龟案例，如果要控制乌龟运动，要启动多个窗口，分别启动 roscore、乌龟界面节点、键盘控制节点。如果每次都调用 rosrn 逐一启动，显然效率低下，如何优化？

官方给出的优化策略是使用 launch 文件，可以一次性启动多个 ROS 节点。

2.实现

1. 选定功能包右击 ---> 添加 launch 文件夹
2. 选定 launch 文件夹右击 ---> 添加 launch 文件
3. 编辑 launch 文件内容

```
<launch>
  <node pkg="helloworld" type="demo_hello" name="hello" output="screen" />
  <node pkg="turtlesim" type="turtlesim_node" name="t1"/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="key1" />
</launch>
Copy
```

- o node ---> 包含的某个节点
- o pkg -----> 功能包
- o type ----> 被运行的节点文件
- o name --> 为节点命名
- o output-> 设置日志的输出目标

4. 运行 launch 文件

`roslaunch` 包名 launch文件名

5. 运行结果: 一次性启动了多个节点

1.5 ROS架构

到目前为止，我们已经安装了ROS，运行了ROS中内置的小乌龟案例，并且也编写了ROS小程序，对ROS也有了一个大概的认知，当然这个认知可能还是比较模糊并不清晰的，接下来，我们要从宏观上来介绍一下ROS的架构设计。

立足不同的角度，对ROS架构的描述也是不同的，一般我们可以从设计者、维护者、系统结构与自身结构4个角度来描述ROS结构:

1.设计者

ROS**设计者**将ROS表述为“ROS = Plumbing + Tools + Capabilities + Ecosystem”

- Plumbing: **通讯机制(实现ROS不同节点之间的交互)**
- Tools :**工具软件包(ROS中的开发和调试工具)**
- Capabilities :机器人高层技能(ROS中某些功能的集合，比如:导航)
- Ecosystem:机器人生态系统(跨地域、跨软件与硬件的ROS联盟)

2.维护者

立足**维护者**的角度: ROS 架构可划分为两大部分

- main: 核心部分，主要由Willow Garage 和一些开发者设计、提供以及维护。它提供了一些分布式计算的基本工具，以及整个ROS的核心部分的程序编写。

- universe: 全球范围的代码, 有不同国家的ROS社区组织开发和维护。一种是库的代码, 如 OpenCV、PCL等; 库的上一层是从功能角度提供的代码, 如人脸识别, 他们调用下层的库; 最上层的代码是应用级的代码, 让机器人完成某一确定的功能。

3.系统架构

立足系统架构: ROS 可以划分为三层

- OS 层, 也即经典意义的操作系统
ROS 只是元操作系统, 需要依托真正意义的操作系统, 目前兼容性最好的是 Linux 的 Ubuntu, Mac、Windows 也支持 ROS 的较新版本
- 中间层
是 ROS 封装的关于机器人开发的中间件, 比如:
 - 基于 TCP/UDP 继续封装的 TCPROS/UDPROS 通信系统
 - 用于进程间通信 Nodelet, 为数据的实时性传输提供支持
 - 另外, 还提供了大量的机器人开发实现库, 如: 数据类型定义、坐标变换、运动控制....
- 应用层
功能包, 以及功能包内的节点, 比如: master、turtlesim的控制与运动节点...

4.自身结构

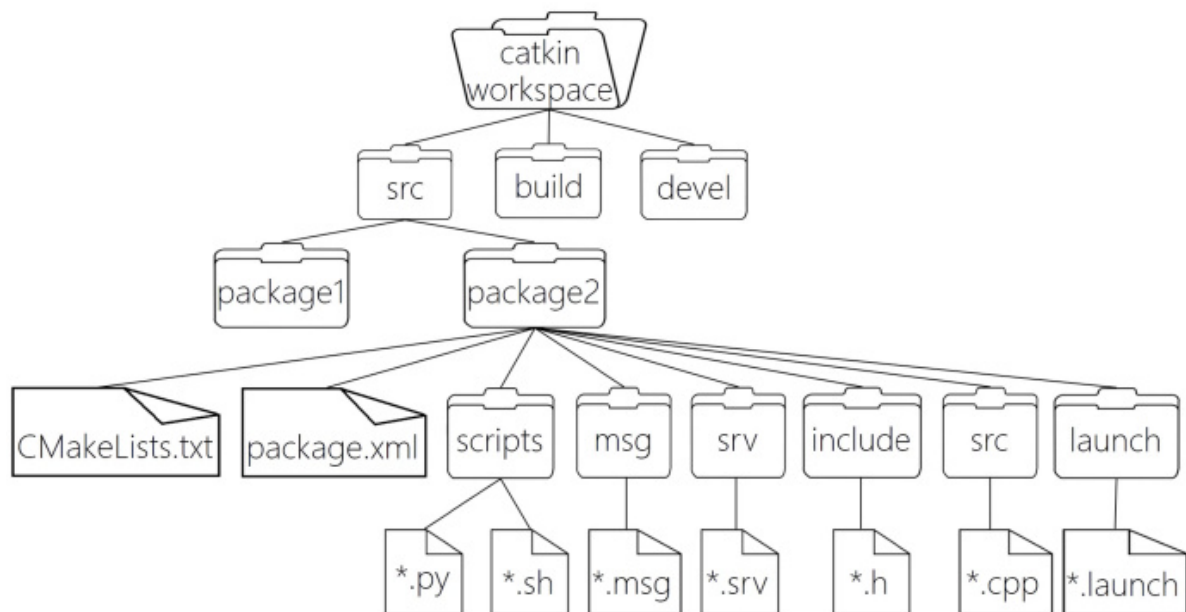
就 ROS 自身实现而言: 也可以划分为三层

- 文件系统
ROS文件系统级指的是在硬盘上面查看的ROS源代码的组织形式
- 计算图
ROS 分布式系统中不同进程需要进行数据交互, 计算图可以以点对点的网络形式表现数据交互过程, 计算图中的重要概念: 节点(Node)、消息(message)、通信机制主题(topic)、通信机制服务(service)
- 开源社区
ROS的社区级概念是ROS网络上进行代码发布的一种表现形式
 - 发行版 (Distribution) ROS发行版是可以独立安装、带有版本号的一系列综合功能包。ROS发行版像Linux发行版一样发挥类似的作用。这使得ROS软件安装更加容易, 而且能够通过一个软件集合维持一致的版本。
 - 软件库 (Repository) ROS依赖于共享开源代码与软件库的网站或主机服务, 在这里不同的机构能够发布和分享各自的机器人软件与程序。
 - ROS维基 (ROS Wiki) ROS Wiki是用于记录有关ROS系统信息的主要论坛。任何人都可以注册账户、贡献自己的文件、提供更正或更新、编写教程以及其他行为。网址是<http://wiki.ros.org/>。
 - Bug提交系统 (Bug Ticket System) 如果你发现问题或者想提出一个新功能, ROS提供这个资源去做这些。
 - 邮件列表 (Mailing list) ROS用户邮件列表是关于ROS的主要交流渠道, 能够像论坛一样交流从ROS软件更新到ROS软件使用中的各种疑问或信息。网址是<http://lists.ros.org/>。
 - ROS问答 (ROS Answer) 用户可以使用这个资源去提问题。网址是<https://answers.ros.org/questions/>。
 - 博客 (Blog) 你可以看到定期更新、照片和新闻。网址是<https://www.ros.org/news/>, 不过博客系统已经退休, ROS社区取而代之, 网址是<https://discourse.ros.org/>。

现在处于学习的初级阶段, 只是运行了ROS的内置案例, 编写了简单的ROS实现, 因此, 受限于当前进度, 不会详细介绍所有设计架构中的所有模块, 当前只介绍文件系统与计算图, 下一章会介绍 ROS 的通信机制, 这也是ROS的核心实现之一。

1.5.1 ROS文件系统

ROS文件系统级指的是在硬盘上ROS源代码的组织形式，其结构大致可以如下图所示：



workspace --- 自定义的工作空间

|--- **build**:编译空间，用于存放CMake和catkin的缓存信息、配置信息和其他中间文件。

|--- **devel**:开发空间，用于存放编译后生成的目标文件，包括头文件、动态&静态链接库、可执行文件等。

|--- **src**: 源码

|-- **package**: 功能包(ROS基本单元)包含多个节点、库与配置文件，包名所有字母小写，只能由字母、数字与下划线组成

|-- **CMakeLists.txt** 配置编译规则，比如源文件、依赖项、目标文件

|-- **package.xml** 包信息，比如:包名、版本、作者、依赖项...(以前版本是manifest.xml)

|-- **scripts** 存储python文件

|-- **src** 存储C++源文件

|-- **include** 头文件

|-- **msg** 消息通信格式文件

|-- **srv** 服务通信格式文件

|-- **action** 动作格式文件

|-- **launch** 可一次性运行多个节点

|-- **config** 配置信息

|-- **CMakeLists.txt**: 编译的基本配置

Copy

ROS 文件系统中部分目录和文件前面编程中已经有所涉及，比如功能包的创建、src目录下cpp文件的编写、scripts目录下python文件的编写、launch目录下launch文件的编写，并且也配置了 package.xml 与 CMakeLists.txt 文件。其他目录下的内容后面教程将会再行介绍，当前我们主要介绍: package.xml 与 CMakeLists.txt 这两个配置文件。

1.package.xml

该文件定义有关软件包的属性，例如软件包名称，版本号，作者，维护者以及对其他catkin软件包的依赖性。请注意，该概念类似于旧版 rosbuild 构建系统中使用的*manifest.xml*文件。

```
<?xml version="1.0"?>
<!-- 格式：以前是 1，推荐使用格式 2 -->
<package format="2">
  <!-- 包名 -->
  <name>demo01_hello_vscode</name>
  <!-- 版本 -->
  <version>0.0.0</version>
  <!-- 描述信息 -->
  <description>The demo01_hello_vscode package</description>

  <!-- One maintainer tag required, multiple allowed, one person per tag -->
  <!-- Example: -->
  <!-- <maintainer email="jane.doe@example.com">Jane Doe</maintainer> -->
  <!-- 维护人员 -->
  <maintainer email="xuzuo@todo.todo">xuzuo</maintainer>

  <!-- One license tag required, multiple allowed, one license per tag -->
  <!-- Commonly used license strings: -->
  <!--   BSD, MIT, Boost Software License, GPLv2, GPLv3, LGPLv2.1, LGPLv3 -->
  <!-- 许可证信息，ROS核心组件默认 BSD -->
  <license>TODO</license>

  <!-- Url tags are optional, but multiple are allowed, one per tag -->
  <!-- Optional attribute type can be: website, bugtracker, or repository -->
  <!-- Example: -->
  <!-- <url type="website">http://wiki.ros.org/demo01_hello_vscode</url> -->

  <!-- Author tags are optional, multiple are allowed, one per tag -->
  <!-- Authors do not have to be maintainers, but could be -->
  <!-- Example: -->
  <!-- <author email="jane.doe@example.com">Jane Doe</author> -->

  <!-- The *depend tags are used to specify dependencies -->
  <!-- Dependencies can be catkin packages or system dependencies -->
  <!-- Examples: -->
  <!-- Use depend as a shortcut for packages that are both build and exec
dependencies -->
  <!--   <depend>roscpp</depend> -->
  <!--   Note that this is equivalent to the following: -->
  <!--   <build_depend>roscpp</build_depend> -->
  <!--   <exec_depend>roscpp</exec_depend> -->
  <!-- Use build_depend for packages you need at compile time: -->
```

```

<!-- <build_depend>message_generation</build_depend> -->
<!-- Use build_export_depend for packages you need in order to build against
this package: -->
<!-- <build_export_depend>message_generation</build_export_depend> -->
<!-- Use buildtool_depend for build tool packages: -->
<!-- <buildtool_depend>catkin</buildtool_depend> -->
<!-- Use exec_depend for packages you need at runtime: -->
<!-- <exec_depend>message_runtime</exec_depend> -->
<!-- Use test_depend for packages you need only for testing: -->
<!-- <test_depend>gtest</test_depend> -->
<!-- Use doc_depend for packages you need only for building documentation: -->
<!-- <doc_depend>doxygen</doc_depend> -->
<!-- 依赖的构建工具，这是必须的 -->
<buildtool_depend>catkin</buildtool_depend>

<!-- 指定构建此软件包所需的软件包 -->
<build_depend>roscpp</build_depend>
<build_depend>rospy</build_depend>
<build_depend>std_msgs</build_depend>

<!-- 指定根据这个包构建库所需要的包 -->
<build_export_depend>roscpp</build_export_depend>
<build_export_depend>rospy</build_export_depend>
<build_export_depend>std_msgs</build_export_depend>

<!-- 运行该程序包中的代码所需的程序包 -->
<exec_depend>roscpp</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>std_msgs</exec_depend>

<!-- The export tag contains other, unspecified, tags -->
<export>
  <!-- Other tools can request additional information be placed here -->

</export>
</package>
Copy

```

2.CMakeLists.txt

文件**CMakeLists.txt**是CMake构建系统的输入，用于构建软件包。任何兼容CMake的软件包都包含一个或多个CMakeLists.txt文件，这些文件描述了如何构建代码以及将代码安装到何处。

```

cmake_minimum_required(VERSION 3.0.2) #所需 cmake 版本
project(demo01_hello_vscode) #包名称，会被 ${PROJECT_NAME} 的方式调用

## Compile as C++11, supported in ROS kinetic and newer
# add_compile_options(-std=c++11)

## Find catkin macros and libraries
## if COMPONENTS list like find_package(catkin REQUIRED COMPONENTS xyz)
## is used, also find other catkin packages
#设置构建所需要的软件包
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy

```

```

std_msgs
)

## System dependencies are found with CMake's conventions
## 默认添加系统依赖
# find_package(Boost REQUIRED COMPONENTS system)

## Uncomment this if the package has a setup.py. This macro ensures
## modules and global scripts declared therein get installed
## See http://ros.org/doc/api/catkin/html/user\_guide/setup\_dot\_py.html
# 启动 python 模块支持
# catkin_python_setup()

#####
## Declare ROS messages, services and actions ##
## 声明 ROS 消息、服务、动作... ##
#####

## To declare and build messages, services or actions from within this
## package, follow these steps:
## * Let MSG_DEP_SET be the set of packages whose message types you use in
##   your messages/services/actions (e.g. std_msgs, actionlib_msgs, ...).
## * In the file package.xml:
##   * add a build_depend tag for "message_generation"
##   * add a build_depend and a exec_depend tag for each package in MSG_DEP_SET
##   * If MSG_DEP_SET isn't empty the following dependency has been pulled in
##     but can be declared for certainty nonetheless:
##   * add a exec_depend tag for "message_runtime"
## * In this file (CMakeLists.txt):
##   * add "message_generation" and every package in MSG_DEP_SET to
##     find_package(catkin REQUIRED COMPONENTS ...)
##   * add "message_runtime" and every package in MSG_DEP_SET to
##     catkin_package(CATKIN_DEPENDS ...)
##   * uncomment the add_*_files sections below as needed
##     and list every .msg/.srv/.action file to be processed
##   * uncomment the generate_messages entry below
##   * add every package in MSG_DEP_SET to generate_messages(DEPENDENCIES ...)

## Generate messages in the 'msg' folder
# add_message_files(
#   FILES
#   Message1.msg
#   Message2.msg
# )

## Generate services in the 'srv' folder
# add_service_files(
#   FILES
#   Service1.srv
#   Service2.srv
# )

## Generate actions in the 'action' folder
# add_action_files(
#   FILES
#   Action1.action
#   Action2.action

```

```

# )

## Generate added messages and services with any dependencies listed here
# 生成消息、服务时的依赖包
# generate_messages(
#     DEPENDENCIES
#     std_msgs
# )

#####
## Declare ROS dynamic reconfigure parameters ##
## 声明 ROS 动态参数配置 ##
#####

## To declare and build dynamic reconfigure parameters within this
## package, follow these steps:
## * In the file package.xml:
##   * add a build_depend and a exec_depend tag for "dynamic_reconfigure"
## * In this file (CMakeLists.txt):
##   * add "dynamic_reconfigure" to
##     find_package(catkin REQUIRED COMPONENTS ...)
##   * uncomment the "generate_dynamic_reconfigure_options" section below
##     and list every .cfg file to be processed

## Generate dynamic reconfigure parameters in the 'cfg' folder
# generate_dynamic_reconfigure_options(
#     cfg/DynReconf1.cfg
#     cfg/DynReconf2.cfg
# )

#####
## catkin specific configuration ##
## catkin 特定配置##
#####
## The catkin_package macro generates cmake config files for your package
## Declare things to be passed to dependent projects
## INCLUDE_DIRS: uncomment this if your package contains header files
## LIBRARIES: libraries you create in this project that dependent projects also
need
## CATKIN_DEPENDS: catkin_packages dependent projects also need
## DEPENDS: system dependencies of this project that dependent projects also
need
# 运行时依赖
catkin_package(
#   INCLUDE_DIRS include
#   LIBRARIES demo01_hello_vscode
#   CATKIN_DEPENDS roscpp rospy std_msgs
#   DEPENDS system_lib
)

#####
## Build ##
#####

## Specify additional locations of header files
## Your package locations should be listed before other locations
# 添加头文件路径，当前程序包的头文件路径位于其他文件路径之前
include_directories(

```

```

# include
  ${catkin_INCLUDE_DIRS}
)

## Declare a C++ library
# 声明 C++ 库
# add_library(${PROJECT_NAME}
#   src/${PROJECT_NAME}/demo01_hello_vscode.cpp
# )

## Add cmake target dependencies of the library
## as an example, code may need to be generated before libraries
## either from message generation or dynamic reconfigure
# 添加库的 cmake 目标依赖
# add_dependencies(${PROJECT_NAME} ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

## Declare a C++ executable
## with catkin_make all packages are built within a single CMake context
## The recommended prefix ensures that target names across packages don't
collide
# 声明 C++ 可执行文件
add_executable(Hello_VSCode src/Hello_VSCode.cpp)

## Rename C++ executable without prefix
## The above recommended prefix causes long target names, the following renames
the
## target back to the shorter version for ease of user use
## e.g. "roslaunch someones_pkg node" instead of "roslaunch someones_pkg
someones_pkg_node"
#重命名c++可执行文件
# set_target_properties(${PROJECT_NAME}_node PROPERTIES OUTPUT_NAME node PREFIX
"")

## Add cmake target dependencies of the executable
## same as for the library above
#添加可执行文件的 cmake 目标依赖
add_dependencies(Hello_VSCode ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

## Specify libraries to link a library or executable target against
#指定库、可执行文件的链接库
target_link_libraries(Hello_VSCode
  ${catkin_LIBRARIES}
)

#####
## Install ##
## 安装 ##
#####

# all install targets should use catkin DESTINATION variables
# See http://ros.org/doc/api/catkin/html/adv\_user\_guide/variables.html

## Mark executable scripts (Python etc.) for installation
## in contrast to setup.py, you can choose the destination
#设置用于安装的可执行脚本
catkin_install_python(PROGRAMS

```

```

scripts/Hi.py
DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)

## Mark executables for installation
## See
http://docs.ros.org/melodic/api/catkin/html/howto/format1/building_executables.html
# install(TARGETS ${PROJECT_NAME}_node
#   RUNTIME DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
# )

## Mark libraries for installation
## See
http://docs.ros.org/melodic/api/catkin/html/howto/format1/building_libraries.html
# install(TARGETS ${PROJECT_NAME}
#   ARCHIVE DESTINATION ${CATKIN_PACKAGE_LIB_DESTINATION}
#   LIBRARY DESTINATION ${CATKIN_PACKAGE_LIB_DESTINATION}
#   RUNTIME DESTINATION ${CATKIN_GLOBAL_BIN_DESTINATION}
# )

## Mark cpp header files for installation
# install(DIRECTORY include/${PROJECT_NAME}/
#   DESTINATION ${CATKIN_PACKAGE_INCLUDE_DESTINATION}
#   FILES_MATCHING PATTERN "*.h"
#   PATTERN ".svn" EXCLUDE
# )

## Mark other files for installation (e.g. launch and bag files, etc.)
# install(FILES
#   # myfile1
#   # myfile2
#   DESTINATION ${CATKIN_PACKAGE_SHARE_DESTINATION}
# )

#####
## Testing ##
#####

## Add gtest based cpp test target and link libraries
# catkin_add_gtest(${PROJECT_NAME}-test test/test_demo01_hello_vscode.cpp)
# if(TARGET ${PROJECT_NAME}-test)
#   target_link_libraries(${PROJECT_NAME}-test ${PROJECT_NAME})
# endif()

## Add folders to be run by python nosetests
# catkin_add_nosetests(test)

```

1.5.2 ROS文件系统相关命令

ROS 的文件系统本质上都还是操作系统文件，我们可以使用Linux命令来操作这些文件，不过，在ROS 中为了更好的用户体验，ROS专门提供了一些类似于Linux的命令，这些命令较之于Linux原生命令，更为简介、高效。文件操作，无外乎就是增删改查与执行等操作，接下来，我们就从这五个维度，来介绍ROS文件系统的一些常用命令。

1.增

catkin_create_pkg 自定义包名 依赖包 === 创建新的ROS功能包

sudo apt install xxx === 安装 ROS功能包

2.删

sudo apt purge xxx ===== 删除某个功能包

3.查

rospack list === 列出所有功能包

rospack find 包名 === 查找某个功能包是否存在，如果存在返回安装路径

roscd 包名 === 进入某个功能包

rosls 包名 === 列出某个包下的文件

apt search xxx === 搜索某个功能包

4.改

roscd 包名 文件名 === 修改功能包文件

需要安装 vim

比如:roscd turtlesim Color.msg

5.执行

5.1roscore

roscore === 是 ROS 的系统先决条件节点和程序的集合，必须运行 roscore 才能使 ROS 节点进行通信。

roscore 将启动:

- ros master
- ros 参数服务器
- roscout 日志节点

用法:

```
roscore  
Copy
```

或(指定端口号)

```
roscore -p xxxx  
Copy
```

5.2roslaunch

roslaunch 包名 可执行文件名 === 运行指定的ROS节点

比如:roslaunch turtlesim turtlesim_node

5.3roslaunch

roslaunch 包名 launch文件名 === 执行某个包下的 launch 文件

1.5.3 ROS计算图

1.计算图简介

前面介绍的是ROS文件结构，是磁盘上 ROS 程序的存储结构，是静态的，而 ros 程序运行之后，不同的节点之间是错综复杂的，ROS 中提供了一个实用的工具:rqt_graph。

rqt_graph能够创建一个显示当前系统运行情况的动态图形。ROS 分布式系统中不同进程需要进行数据交互，计算图可以以点对点的网络形式表现数据交互过程。rqt_graph是rqt程序包中的一部分。

2.计算图安装

如果前期把所有的功能包（package）都已经安装完成，则直接在终端窗口中输入

```
roslaunch rqt_graph rqt_graph
```

如果未安装则在终端（terminal）中输入

```
$ sudo apt install ros-<distro>-rqt
$ sudo apt install ros-<distro>-rqt-common-plugins
Copy
```

请使用你的ROS版本名称（比如:kinetic、melodic、Noetic等）来替换掉。

例如当前版本是 Noetic,就在终端窗口中输入

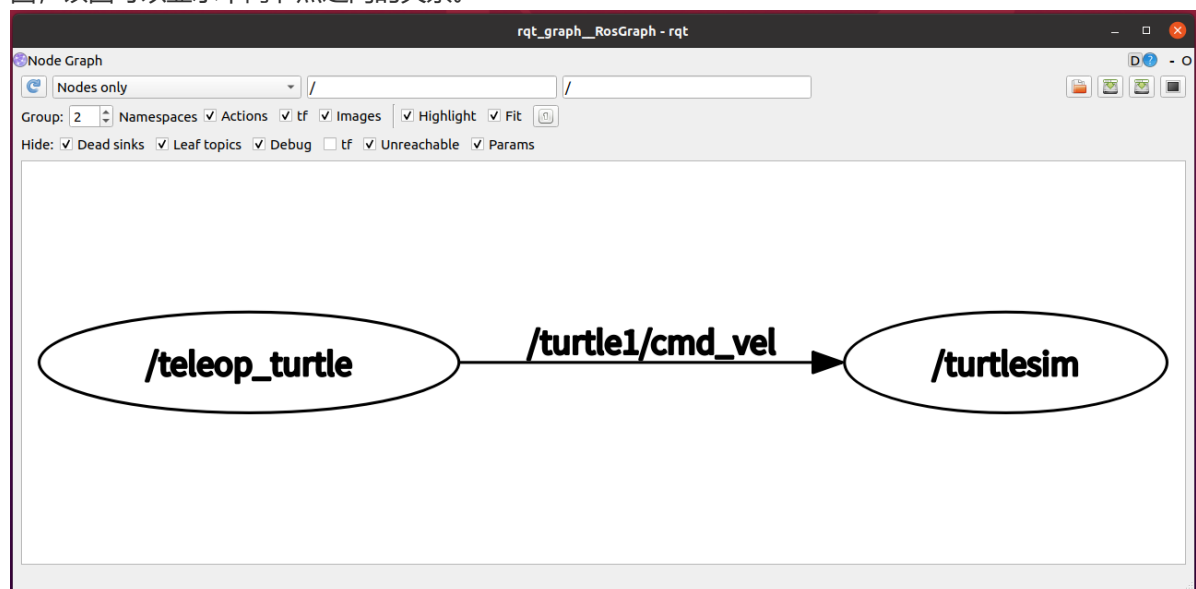
```
$ sudo apt install ros-noetic-rqt
$ sudo apt install ros-noetic-rqt-common-plugins
Copy
```

3.计算图演示

接下来以 ROS 内置的小乌龟案例来演示计算图

首先，按照前面所示，运行案例

然后，启动新终端，键入: `rqt_graph` 或 `roslaunch rqt_graph rqt_graph`，可以看到类似下图的网络拓扑图，该图可以显示不同节点之间的关系。



1.6 本章小结

本章内容主要介绍了ROS的相关概念、设计目标、发展历程等理论知识，安装了 ROS 并搭建了 ROS 的集成开发环境，编写了第一个 ROS 小程序，对ROS实现架构也有了宏观的认识。ROS的大门已经敞开，接下来就要步入新的征程了。

第 2 章 ROS通信机制

机器人是一种高度复杂的系统性实现，在机器人上可能集成各种传感器(雷达、摄像头、GPS...)以及运动控制实现，为了解耦合，在ROS中每一个功能点都是一个单独的进程，每一个进程都是独立运行的。更确切的讲，**ROS是进程（也称为*Nodes*）的分布式框架**。因为这些进程甚至还可分布于不同主机，不同主机协同工作，从而分散计算压力。不过随之也有一个问题: 不同的进程是如何通信的？也即不同进程间如何实现数据交换的？在此我们就需要介绍一下ROS中的通信机制了。

ROS 中的基本通信机制主要有如下三种实现策略:

- 话题通信(发布订阅模式)
- 服务通信(请求响应模式)
- 参数服务器(参数共享模式)

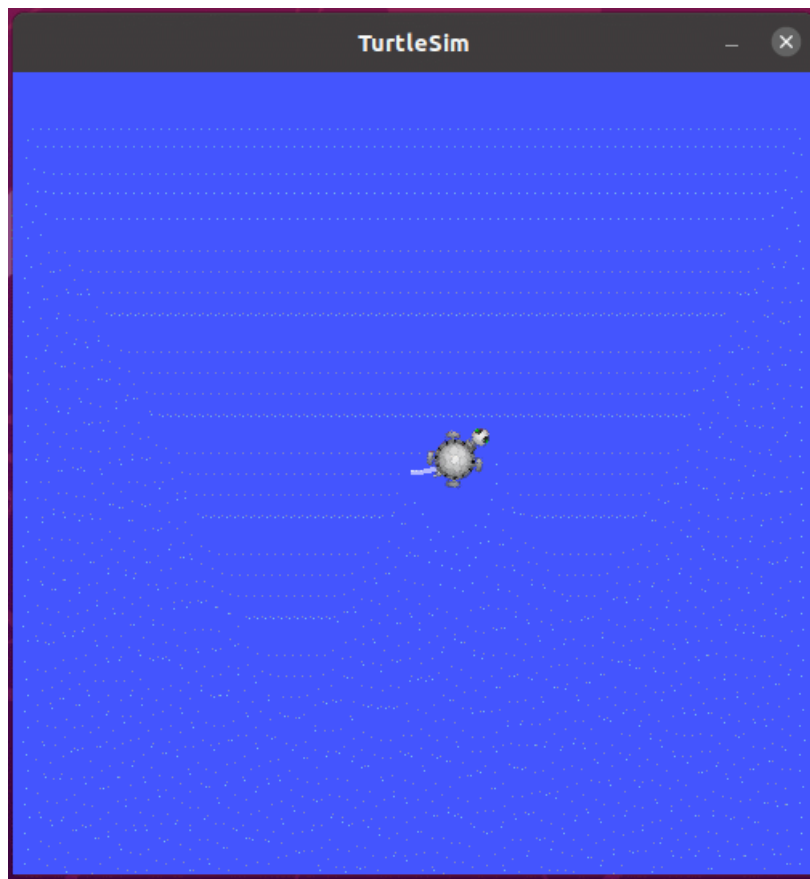
本章的主要内容就是介绍各个通信机制的应用场景、理论模型、代码实现以及相关操作命令。本章预期达成学习目标如下:

- 能够熟练介绍ROS中常用的通信机制
- 能够理解ROS中每种通信机制的理论模型
- 能够以代码的方式实现各种通信机制对应的案例
- 能够熟练使用ROS中的一些操作命令
- 能够独立完成相关实操案例

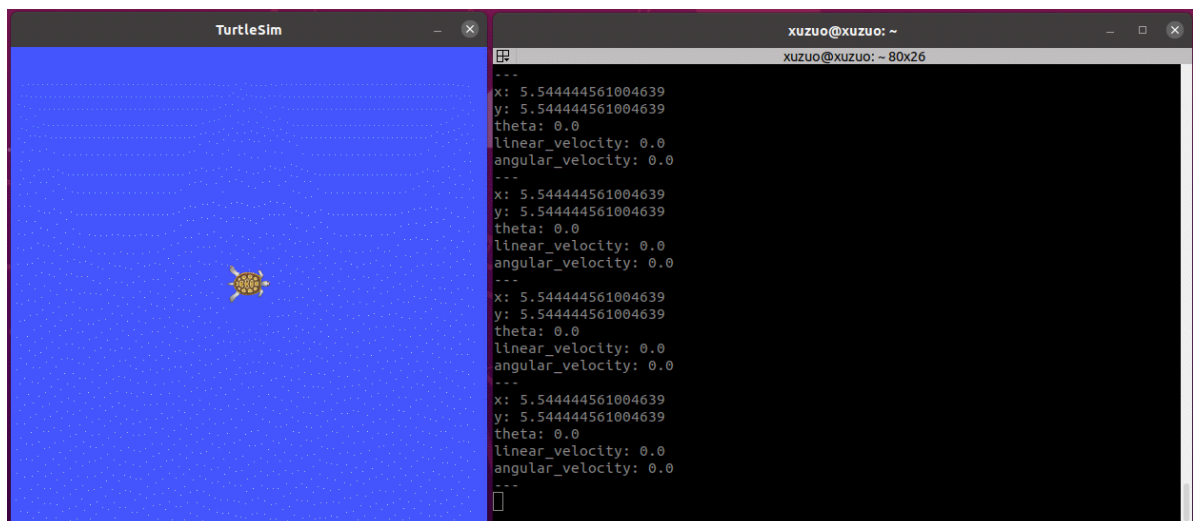
案例演示:

1.话题演示案例:

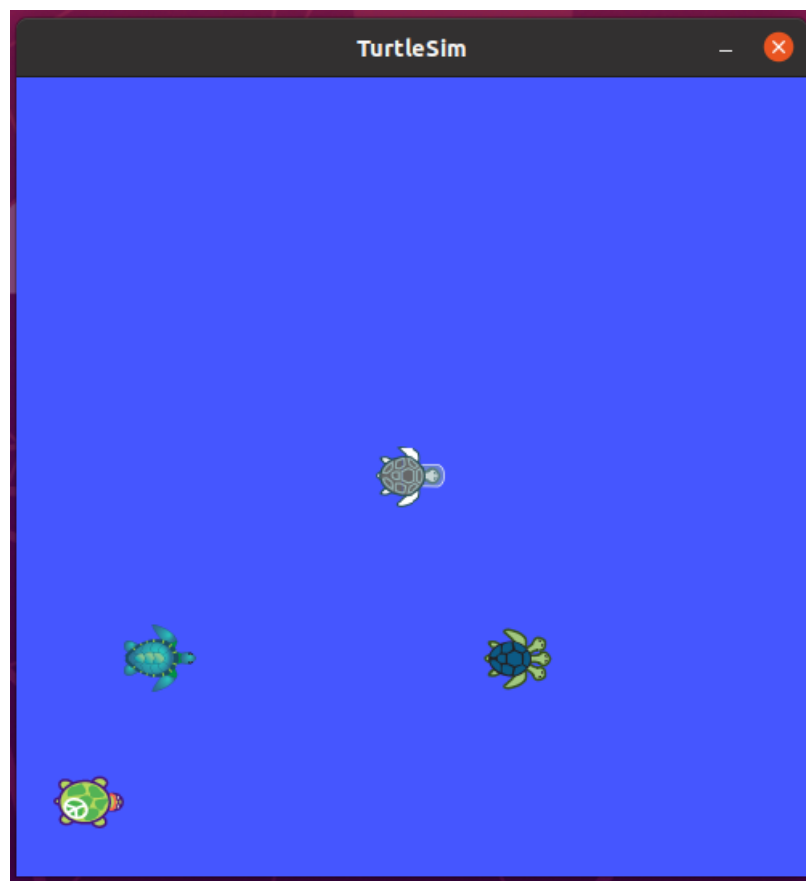
控制小乌龟做圆周运动



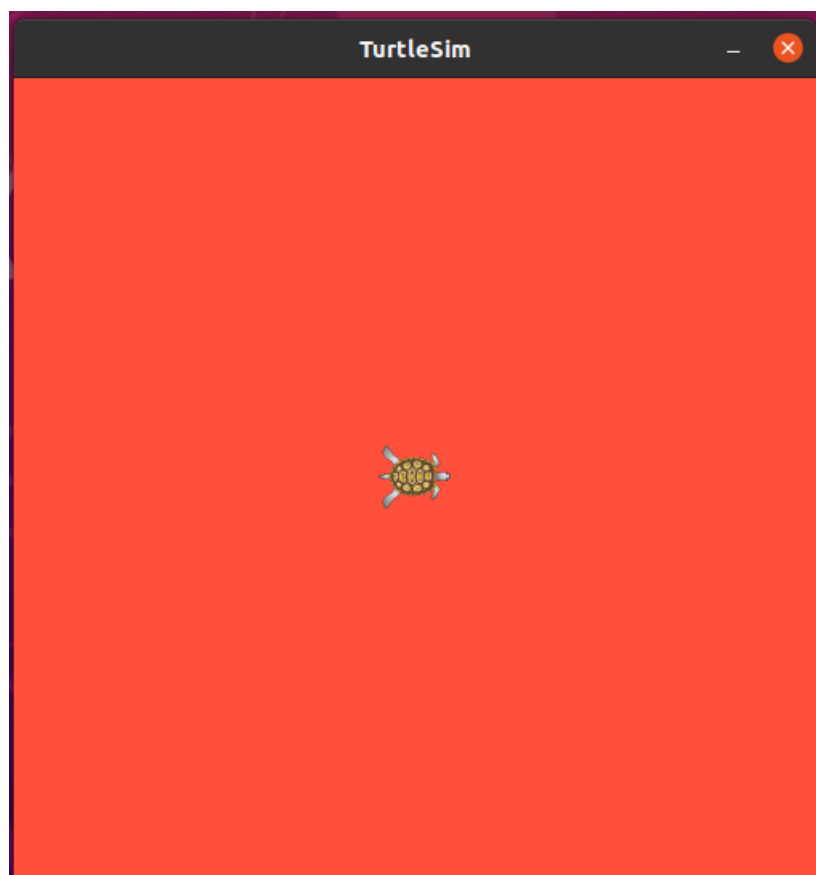
获取乌龟位姿



2.服务演示案例:在指定位置生成乌龟



3.参数演示案例:改变乌龟窗口的背景颜色



2.1 话题通信

```
rosdemo@rosdemo-VirtualBox: ~/demo02_ws
roscore http://rosdemo-VirtualBox:11311/40x24
ros_comm version 1.15.8

SUMMARY
=====
PARAMETERS
* /roscdistro: noetic
* /rosversion: 1.15.8

NODES

auto-starting new master
process[roscdistro]: started with pid [71893]
ROS_MASTER_URI=http://rosdemo-VirtualBox:11311/
setting /run_id to 649db858-2eef-11eb-a904-d570e3a6f380
process[roscdistro-1]: started with pid [71903]
started core service [/roscdistro]
```

```
rosdemo@rosdemo-VirtualBox: ~/demo02_ws 65x11
[ INFO] [1606289607.073599994]: 姓名:张三, 年龄:27, 身高:1.73
[ INFO] [1606289608.074001096]: 姓名:张三, 年龄:28, 身高:1.73
[ INFO] [1606289609.073615199]: 姓名:张三, 年龄:29, 身高:1.73
[ INFO] [1606289610.073580332]: 姓名:张三, 年龄:30, 身高:1.73
[ INFO] [1606289611.074265210]: 姓名:张三, 年龄:31, 身高:1.73
[ INFO] [1606289612.073834962]: 姓名:张三, 年龄:32, 身高:1.73
[ INFO] [1606289613.073997812]: 姓名:张三, 年龄:33, 身高:1.73
[ INFO] [1606289614.073765017]: 姓名:张三, 年龄:34, 身高:1.73
[ INFO] [1606289615.074094124]: 姓名:张三, 年龄:35, 身高:1.73
[ INFO] [1606289616.074266588]: 姓名:张三, 年龄:36, 身高:1.73

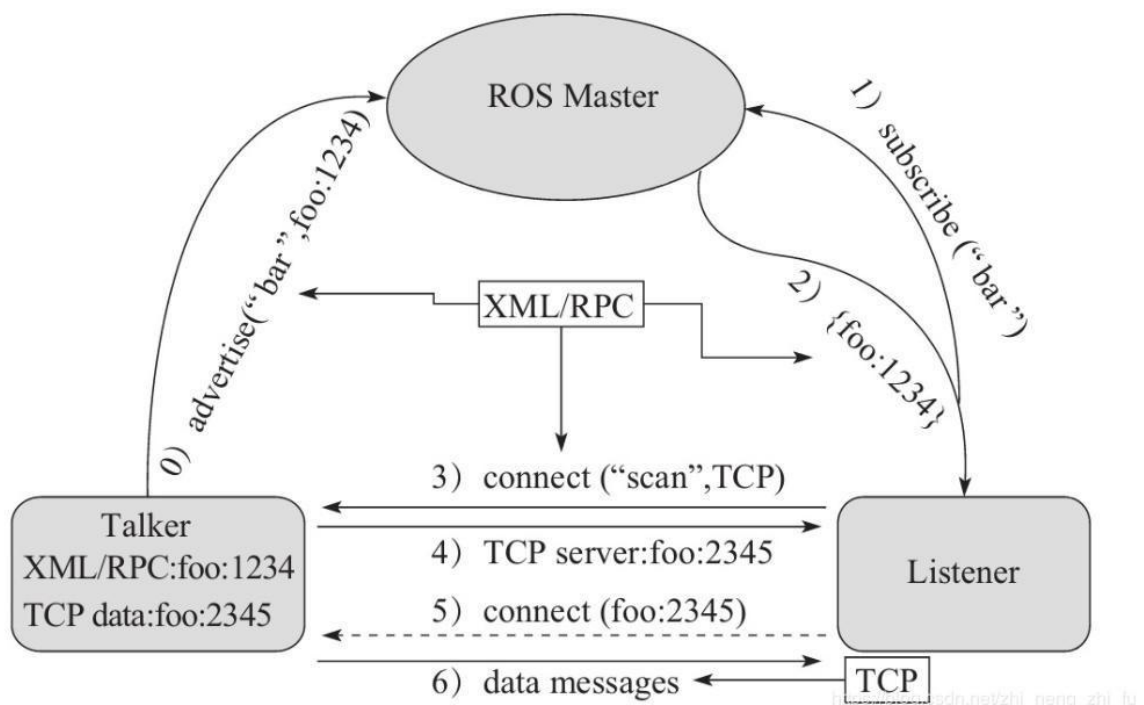
rosdemo@rosdemo-VirtualBox: ~/demo02_ws 65x11
[ INFO] [1606289607.073963613]: 接收到的人的信息是:张三,27,1.73
[ INFO] [1606289608.074292201]: 接收到的人的信息是:张三,28,1.73
[ INFO] [1606289609.073932831]: 接收到的人的信息是:张三,29,1.73
[ INFO] [1606289610.073793106]: 接收到的人的信息是:张三,30,1.73
[ INFO] [1606289611.074535290]: 接收到的人的信息是:张三,31,1.73
[ INFO] [1606289612.074150360]: 接收到的人的信息是:张三,32,1.73
[ INFO] [1606289613.074392754]: 接收到的人的信息是:张三,33,1.73
[ INFO] [1606289614.074179345]: 接收到的人的信息是:张三,34,1.73
[ INFO] [1606289615.074383618]: 接收到的人的信息是:张三,35,1.73
[ INFO] [1606289616.074666650]: 接收到的人的信息是:张三,36,1.73
```

2.1.1 理论模型

话题通信实现模型是比较复杂的，该模型如下图所示，该模型中涉及到三个角色：

- ROS Master (管理者)
- Talker (发布者)
- Listener (订阅者)

ROS Master 负责保管 Talker 和 Listener 注册的信息，并匹配话题相同的 Talker 与 Listener，帮助 Talker 与 Listener 建立连接，连接建立后，Talker 可以发布消息，且发布的消息会被 Listener 订阅。



整个流程由以下步骤实现：

0.Talker注册

Talker启动后，会通过RPC在 ROS Master 中注册自身信息，其中包含所发布消息的话题名称。ROS Master 会将节点的注册信息加入到注册表中。

1.Listener注册

Listener启动后，也会通过RPC在 ROS Master 中注册自身信息，包含需要订阅消息的话题名。ROS Master 会将节点的注册信息加入到注册表中。

2.ROS Master实现信息匹配

ROS Master 会根据注册表中的信息匹配Talker 和 Listener，并通过 RPC 向 Listener 发送 Talker 的 RPC 地址信息。

3.Listener向Talker发送请求

Listener 根据接收到的 RPC 地址，通过 RPC 向 Talker 发送连接请求，传输订阅的话题名称、消息类型以及通信协议(TCP/UDP)。

4.Talker确认请求

Talker 接收到 Listener 的请求后，也是通过 RPC 向 Listener 确认连接信息，并发送自身的 TCP 地址信息。

5.Listener与Talker件里连接

Listener 根据步骤4 返回的消息使用 TCP 与 Talker 建立网络连接。

6.Talker向Listener发送消息

连接建立后，Talker 开始向 Listener 发布消息。

注意1:上述实现流程中，前五步使用的 RPC协议，最后两步使用的是 TCP 协议

注意2: Talker 与 Listener 的启动无先后顺序要求

注意3: Talker 与 Listener 都可以有多个

注意4: Talker 与 Listener 连接建立后，不再需要 ROS Master。也即，即便关闭ROS Master，Talker 与 Listern 照常通信。

2.1.2 话题通信基本操作A(C++)

需求:

编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。

分析:

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个:

1. 发布方
2. 接收方
3. 数据(此处为普通文本)

流程:

1. 编写发布方实现;
2. 编写订阅方实现;

3. 编辑配置文件;
4. 编译并执行。

1.发布方

```
/*
    需求：实现基本的话题通信，一方发布数据，一方接收数据，
    实现的关键点：
    1.发送方
    2.接收方
    3.数据(此处为普通文本)

    PS：二者需要设置相同的话题

    消息发布方：
        循环发布信息:HelloWorld 后缀数字编号

    实现流程：
    1.包含头文件
    2.初始化 ROS 节点:命名(唯一)
    3.实例化 ROS 句柄
    4.实例化 发布者 对象
    5.组织被发布的数据，并编写逻辑发布数据

*/
// 1.包含头文件
#include "ros/ros.h"
#include "std_msgs/String.h" //普通文本类型的消息
#include <sstream>

int main(int argc, char *argv[])
{
    //设置编码
    setlocale(LC_ALL, "");

    //2.初始化 ROS 节点:命名(唯一)
    // 参数1和参数2 后期为节点传值会使用
    // 参数3 是节点名称，是一个标识符，需要保证运行后，在 ROS 网络拓扑中唯一
    ros::init(argc,argv,"talker");
    //3.实例化 ROS 句柄
    ros::NodeHandle nh;//该类封装了 ROS 中的一些常用功能

    //4.实例化 发布者 对象
    //泛型：发布的消息类型
    //参数1：要发布到的话题
    //参数2：队列中最大保存的消息数，超出此阈值时，先进的先销毁(时间早的先销毁)
    ros::Publisher pub = nh.advertise<std_msgs::String>("chatter",10);

    //5.组织被发布的数据，并编写逻辑发布数据
    //数据(动态组织)
    std_msgs::String msg;
    // msg.data = "你好啊!!!";
    std::string msg_front = "Hello 你好! "; //消息前缀
    int count = 0; //消息计数器

    //逻辑(一秒10次)
```

```

ros::Rate r(1);

//节点不死
while (ros::ok())
{
    //使用 stringstream 拼接字符串与编号
    std::stringstream ss;
    ss << msg_front << count;
    msg.data = ss.str();
    //发布消息
    pub.publish(msg);
    //加入调试，打印发送的消息
    ROS_INFO("发送的消息:%s",msg.data.c_str());

    //根据前面制定的发送频率自动休眠 休眠时间 = 1/频率；
    r.sleep();
    count++;//循环结束前，让 count 自增
    //暂无应用
    ros::spinOnce();
}

return 0;
}
Copy

```

2.订阅方

```

/*
    需求：实现基本的话题通信，一方发布数据，一方接收数据，
    实现的关键点：
    1.发送方
    2.接收方
    3.数据(此处为普通文本)

    消息订阅方：
        订阅话题并打印接收到的消息

    实现流程：
    1.包含头文件
    2.初始化 ROS 节点:命名(唯一)
    3.实例化 ROS 句柄
    4.实例化 订阅者 对象
    5.处理订阅的消息(回调函数)
    6.设置循环调用回调函数

*/
// 1.包含头文件
#include "ros/ros.h"
#include "std_msgs/String.h"

void doMsg(const std_msgs::String::ConstPtr& msg_p){
    ROS_INFO("我听见:%s",msg_p->data.c_str());
    // ROS_INFO("我听见:%s",(*msg_p).data.c_str());
}

int main(int argc, char *argv[])

```

```

{
    setlocale(LC_ALL, "");
    //2.初始化 ROS 节点:命名(唯一)
    ros::init(argc,argv,"listener");
    //3.实例化 ROS 句柄
    ros::NodeHandle nh;

    //4.实例化 订阅者 对象
    ros::Subscriber sub = nh.subscribe<std_msgs::String>("chatter",10,dMsg);
    //5.处理订阅的消息(回调函数)

    //      6.设置循环调用回调函数
    ros::spin();//循环读取接收的数据，并调用回调函数处理

    return 0;
}
Copy

```

3.配置 CMakeLists.txt

```

add_executable(Hello_pub
    src/Hello_pub.cpp
)
add_executable(Hello_sub
    src/Hello_sub.cpp
)

target_link_libraries(Hello_pub
    ${catkin_LIBRARIES}
)
target_link_libraries(Hello_sub
    ${catkin_LIBRARIES}
)
Copy

```

4.执行

- 1.启动 roscore;
- 2.启动发布节点;
- 3.启动订阅节点。

运行结果与引言部分的演示案例1类似。

5.注意

补充0:

vscode 中的 main 函数声明 `int main(int argc, char const *argv[]){}`, 默认生成 `argv` 被 `const` 修饰, 需要去除该修饰符

补充1:

ros/ros.h No such file or directory

检查 CMakeList.txt find_package 出现重复,删除内容少的即可

参考资料:<https://answers.ros.org/question/237494/fatal-error-rosrosh-no-such-file-or-directory/>

补充2:

find_package 不添加一些包，也可以运行啊， ros.wiki 答案如下

```
You may notice that sometimes your project builds fine even if you did not call
find_package with all dependencies. This is because catkin combines all your
projects into one, so if an earlier project calls find_package, yours is
configured with the same values. But forgetting the call means your project can
easily break when built in isolation.
Copy
```

补充3:

订阅时，第一条数据丢失

原因: 发送第一条数据时， publisher 还未在 roscore 注册完毕

解决: 注册后，加入休眠 ros::Duration(3.0).sleep(); 延迟第一条数据的发送

PS: 可以使用 rqt_graph 查看节点关系。

2.1.3 话题通信基本操作B(Python)

需求:

编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布文本消息，订阅方订阅消息并将消息内容打印输出。

分析:

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个:

1. 发布方
2. 接收方
3. 数据(此处为普通文本)

流程:

1. 编写发布方实现;
2. 编写订阅方实现;
3. 为python文件添加可执行权限;
4. 编辑配置文件;
5. 编译并执行。

1.发布方

```
#!/usr/bin/env python
"""
```

需求：实现基本的话题通信，一方发布数据，一方接收数据，
实现的关键点：

1. 发送方
2. 接收方
3. 数据(此处为普通文本)

PS：二者需要设置相同的话题

消息发布方:

循环发布信息:HelloWorld 后缀数字编号

实现流程:

1. 导包
2. 初始化 ROS 节点:命名(唯一)
3. 实例化 发布者 对象
4. 组织被发布的数据, 并编写逻辑发布数据

```
"""
#1.导包
import rospy
from std_msgs.msg import String

if __name__ == "__main__":
    #2.初始化 ROS 节点:命名(唯一)
    rospy.init_node("talker_p")
    #3.实例化 发布者 对象
    pub = rospy.Publisher("chatter",String,queue_size=10)
    #4.组织被发布的数据, 并编写逻辑发布数据
    msg = String() #创建 msg 对象
    msg_front = "hello 你好"
    count = 0 #计数器
    # 设置循环频率
    rate = rospy.Rate(1)
    while not rospy.is_shutdown():

        #拼接字符串
        msg.data = msg_front + str(count)

        pub.publish(msg)
        rate.sleep()
        rospy.loginfo("写出的数据:%s",msg.data)
        count += 1
```

Copy

2.订阅方

```
#!/usr/bin/env python
"""
```

需求: 实现基本的话题通信, 一方发布数据, 一方接收数据,

实现的关键点:

1. 发送方
2. 接收方
3. 数据(此处为普通文本)

消息订阅方:

订阅话题并打印接收到的消息

实现流程:

1. 导包
2. 初始化 ROS 节点:命名(唯一)
3. 实例化 订阅者 对象
4. 处理订阅的消息(回调函数)

5. 设置循环调用回调函数

```
"""
#1. 导包
import rospy
from std_msgs.msg import String

def doMsg(msg):
    rospy.loginfo("I heard:%s",msg.data)

if __name__ == "__main__":
    #2. 初始化 ROS 节点:命名(唯一)
    rospy.init_node("listener_p")
    #3. 实例化 订阅者 对象
    sub = rospy.Subscriber("chatter",String,doMsg,queue_size=10)
    #4. 处理订阅的消息(回调函数)
    #5. 设置循环调用回调函数
    rospy.spin()
Copy
```

3. 添加可执行权限

终端下进入 scripts 执行: `chmod +x *.py`

4. 配置 CMakeLists.txt

```
catkin_install_python(PROGRAMS
    scripts/talker_p.py
    scripts/listener_p.py
    DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
Copy
```

5. 执行

1. 启动 roscore;
2. 启动发布节点;
3. 启动订阅节点。

运行结果与引言部分的演示案例1类似。

PS: 可以使用 `rqt_graph` 查看节点关系。

2.1.4 话题通信自定义msg

在 ROS 通信协议中, 数据载体是一个较为重要组成部分, ROS 中通过 `std_msgs` 封装了一些原生的数据类型, 比如: `String`、`Int32`、`Int64`、`Char`、`Bool`、`Empty`.... 但是, 这些数据一般只包含一个 `data` 字段, 结构的单一意味着功能上的局限性, 当传输一些复杂的数据, 比如: 激光雷达的信息... `std_msgs` 由于描述性较差而显得力不从心, 这种场景下可以使用自定义的消息类型

`msgs`只是简单的文本文件, 每行具有字段类型和字段名称, 可以使用的字段类型有:

- `int8`, `int16`, `int32`, `int64` (或者无符号类型: `uint*`)

- float32, float64
- string
- time, duration
- other msg files
- variable-length array[] and fixed-length array[C]

ROS中还有一种特殊类型：`Header`，标头包含时间戳和ROS中常用的坐标帧信息。会经常看到msg文件的第一行具有 `Header` 标头。

需求:创建自定义消息，该消息包含人的信息:姓名、身高、年龄等。

流程:

1. 按照固定格式创建 msg 文件
2. 编辑配置文件
3. 编译生成可以被 Python 或 C++ 调用的中间文件

1.定义msg文件

功能包下新建 msg 目录，添加文件 Person.msg

```
string name
uint16 age
float64 height
Copy
```

2.编辑配置文件

`package.xml`中添加编译依赖与执行依赖

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
<!--
  exce_depend 以前对应的是 run_depend 现在非法
-->
Copy
```

`CMakeLists.txt`编辑 msg 相关配置

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
# 需要加入 message_generation,必须有 std_msgs
Copy
## 配置 msg 源文件
add_message_files(
  FILES
  Person.msg
)
Copy
# 生成消息时依赖于 std_msgs
generate_messages(
```

```

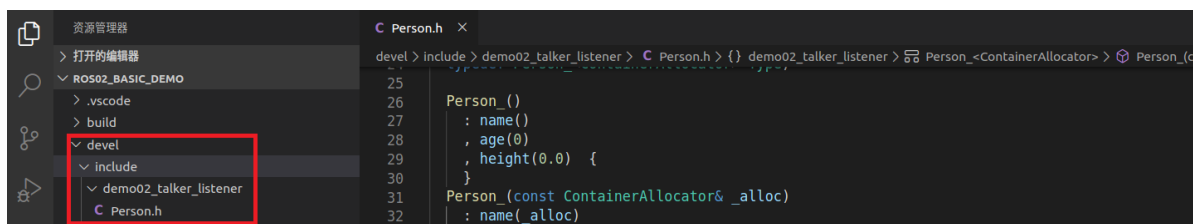
DEPENDENCIES
std_msgs
)
Copy
#执行时依赖
catkin_package(
# INCLUDE_DIRS include
# LIBRARIES demo02_talker_listener
CATKIN_DEPENDS roscpp rospy std_msgs message_runtime
# DEPENDS system_lib
)
Copy

```

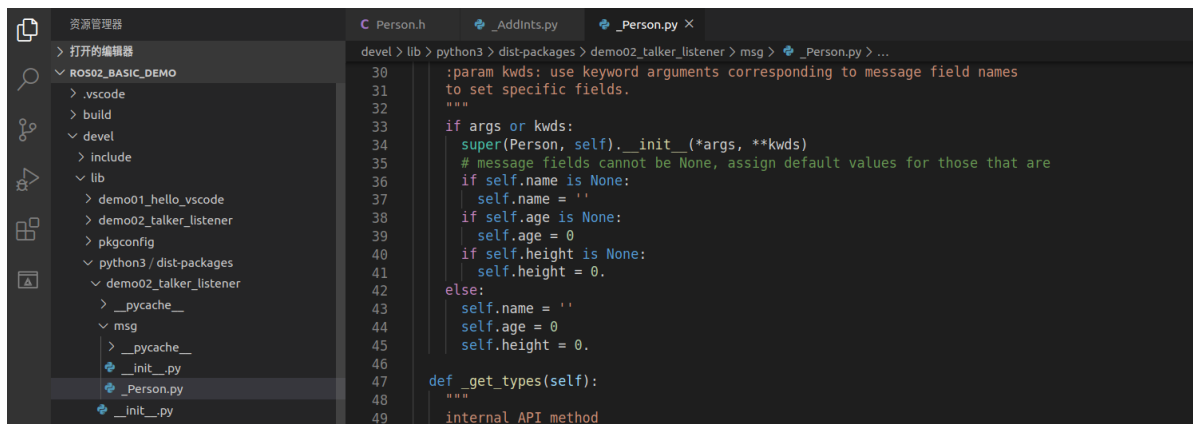
3.编译

编译后的中间文件查看:

C++ 需要调用的中间文件(.../工作空间/devel/include/包名/xxx.h)



Python 需要调用的中间文件(.../工作空间/devel/lib/python3/dist-packages/包名/msg)



后续调用相关 msg 时，是从这些中间文件调用的

2.1.5 话题通信自定义msg调用A(C++)

需求:

编写发布订阅实现，要求发布方以10HZ(每秒10次)的频率发布自定义消息，订阅方订阅自定义消息并将消息内容打印输出。

分析:

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个:

1. 发布方
2. 接收方
3. 数据(此处为自定义消息)

流程:

1. 编写发布方实现;

2. 编写订阅方实现;
3. 编辑配置文件;
4. 编译并执行。

0.vscode 配置

为了方便代码提示以及避免误抛异常，需要先配置 vscode，将前面生成的 head 文件路径配置进 c_cpp_properties.json 的 includepath属性:

```
{
  "configurations": [
    {
      "browse": {
        "databaseFilename": "",
        "limitsymbolsToIncludedHeaders": true
      },
      "includePath": [
        "/opt/ros/noetic/include/**",
        "/usr/include/**",
        "/xxx/yyy工作空间/devel/include/**" //配置 head 文件的路径
      ],
      "name": "ROS",
      "intellisenseMode": "gcc-x64",
      "compilerPath": "/usr/bin/gcc",
      "cStandard": "c11",
      "cppStandard": "c++17"
    }
  ],
  "version": 4
}
```

Copy

1.发布方

```
/*
  需求：循环发布人的信息
*/

#include "ros/ros.h"
#include "demo02_talker_listener/Person.h"

int main(int argc, char *argv[])
{
  setlocale(LC_ALL, "");

  //1.初始化 ROS 节点
  ros::init(argc, argv, "talker_person");

  //2.创建 ROS 句柄
  ros::NodeHandle nh;

  //3.创建发布者对象
  ros::Publisher pub = nh.advertise<demo02_talker_listener::Person>
("chatter_person", 1000);
```

```

//4.组织被发布的消息，编写发布逻辑并发布消息
demo02_talker_listener::Person p;
p.name = "sunwukong";
p.age = 2000;
p.height = 1.45;

ros::Rate r(1);
while (ros::ok())
{
    pub.publish(p);
    p.age += 1;
    ROS_INFO("我叫:%s,今年%d岁,高%.2f米", p.name.c_str(), p.age, p.height);

    r.sleep();
    ros::spinOnce();
}

return 0;
}
Copy

```

2.订阅方

```

/*
    需求：订阅人的信息
*/

#include "ros/ros.h"
#include "demo02_talker_listener/Person.h"

void doPerson(const demo02_talker_listener::Person::ConstPtr& person_p){
    ROS_INFO("订阅的人信息:%s, %d, %.2f", person_p->name.c_str(), person_p->age,
    person_p->height);
}

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");

    //1.初始化 ROS 节点
    ros::init(argc,argv,"listener_person");
    //2.创建 ROS 句柄
    ros::NodeHandle nh;
    //3.创建订阅对象
    ros::Subscriber sub = nh.subscribe<demo02_talker_listener::Person>
    ("chatter_person",10,doPerson);

    //4.回调函数中处理 person

    //5.ros::spin();
    ros::spin();
    return 0;
}
Copy

```

3.配置 CMakeLists.txt

需要添加 `add_dependencies` 用以设置所依赖的消息相关的中间文件。

```
add_executable(person_talker src/person_talker.cpp)
add_executable(person_listener src/person_listener.cpp)

add_dependencies(person_talker ${PROJECT_NAME}_generate_messages_cpp)
add_dependencies(person_listener ${PROJECT_NAME}_generate_messages_cpp)

target_link_libraries(person_talker
    ${catkin_LIBRARIES}
)
target_link_libraries(person_listener
    ${catkin_LIBRARIES}
)
Copy
```

4.执行

- 1.启动 roscore;
- 2.启动发布节点;
- 3.启动订阅节点。

运行结果与引言部分的演示案例2类似。

PS：可以使用 `rqt_graph` 查看节点关系。

2.1.6 话题通信自定义msg调用B(Python)

需求:

编写发布订阅实现，要求发布方以1HZ(每秒1次)的频率发布自定义消息，订阅方订阅自定义消息并将消息内容打印输出。

分析:

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个：

1. 发布方
2. 接收方
3. 数据(此处为自定义消息)

流程:

1. 编写发布方实现;
2. 编写订阅方实现;
3. 为python文件添加可执行权限;
4. 编辑配置文件;
5. 编译并执行。

0.vscode配置

为了方便代码提示以及误抛异常，需要先配置 vscode，将前面生成的 python 文件路径配置进 settings.json

```
{
  "python.autoComplete.extraPaths": [
    "/opt/ros/noetic/lib/python3/dist-packages",
    "/xxx/yyy工作空间/devel/lib/python3/dist-packages"
  ]
}
Copy
```

1.发布方

```
#!/usr/bin/env python
"""
    发布方：
        循环发送消息

"""
import rospy
from demo02_talker_listener.msg import Person

if __name__ == "__main__":
    #1.初始化 ROS 节点
    rospy.init_node("talker_person_p")
    #2.创建发布者对象
    pub = rospy.Publisher("chatter_person", Person, queue_size=10)
    #3.组织消息
    p = Person()
    p.name = "葫芦瓦"
    p.age = 18
    p.height = 0.75

    #4.编写消息发布逻辑
    rate = rospy.Rate(1)
    while not rospy.is_shutdown():
        pub.publish(p) #发布消息
        rate.sleep() #休眠
        rospy.loginfo("姓名:%s, 年龄:%d, 身高:%.2f", p.name, p.age, p.height)
Copy
```

2.订阅方

```
#!/usr/bin/env python
"""
    订阅方：
        订阅消息

"""
import rospy
from demo02_talker_listener.msg import Person

def doPerson(p):
```

```
rospy.loginfo("接收到的人的信息:%s, %d, %.2f",p.name, p.age, p.height)

if __name__ == "__main__":
    #1. 初始化节点
    rospy.init_node("listener_person_p")
    #2. 创建订阅者对象
    sub = rospy.Subscriber("chatter_person", Person, doPerson, queue_size=10)
    rospy.spin() #4. 循环
Copy
```

3. 权限设置

终端下进入 scripts 执行: `chmod +x *.py`

4. 配置 CMakeLists.txt

```
catkin_install_python(PROGRAMS
  scripts/talker_p.py
  scripts/listener_p.py
  scripts/person_talker.py
  scripts/person_listener.py
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
Copy
```

5. 执行

1. 启动 roscore;
2. 启动发布节点;
3. 启动订阅节点。

运行结果与引言部分的演示案例2类似。

PS: 可以使用 `rqt_graph` 查看节点关系。

2.2 服务通信

服务通信也是ROS中一种极其常用的通信模式，服务通信是基于**请求响应**模式的，是一种应答机制。也即: 一个节点A向另一个节点B发送请求，B接收处理请求并产生响应结果返回给A。比如如下场景:

机器人巡逻过程中，控制系统分析传感器数据发现可疑物体或人... 此时需要拍摄照片并留存。

在上述场景中，就使用到了服务通信。

- 一个节点需要向相机节点发送拍照请求，相机节点处理请求，并返回处理结果

与上述应用类似的，服务通信更适用于对时时性有要求、具有一定逻辑处理的应用场景。

概念

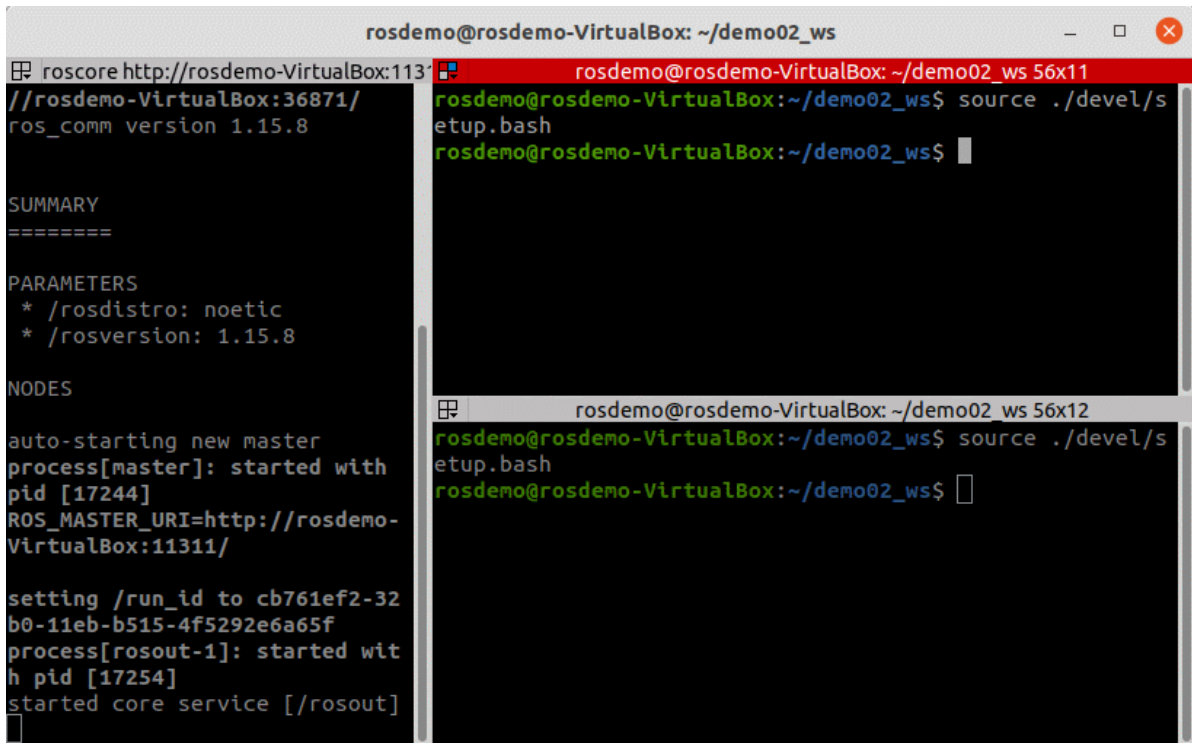
以请求响应的方式实现不同节点之间数据交互的通信模式。

作用

用于偶然的、对时时性有要求、有一定逻辑处理需求的数据传输场景。

案例

实现两个数字的求和，客户端节点，运行会向服务器发送两个数字，服务器端节点接收两个数字求和并将结果响应回客户端。



```
roscore http://rostdemo-VirtualBox:11311/
//rostdemo-VirtualBox:36871/
ros_comm version 1.15.8

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.8

NODES
auto-starting new master
process[master]: started with
pid [17244]
ROS_MASTER_URI=http://rostdemo-
VirtualBox:11311/

setting /run_id to cb761ef2-32
b0-11eb-b515-4f5292e6a65f
process[rosout-1]: started wit
h pid [17254]
started core service [/rosout]
[]

rostdemo@rostdemo-VirtualBox: ~/demo02_ws 56x11
rostdemo@rostdemo-VirtualBox:~/demo02_ws$ source ./devel/s
etup.bash
rostdemo@rostdemo-VirtualBox:~/demo02_ws$

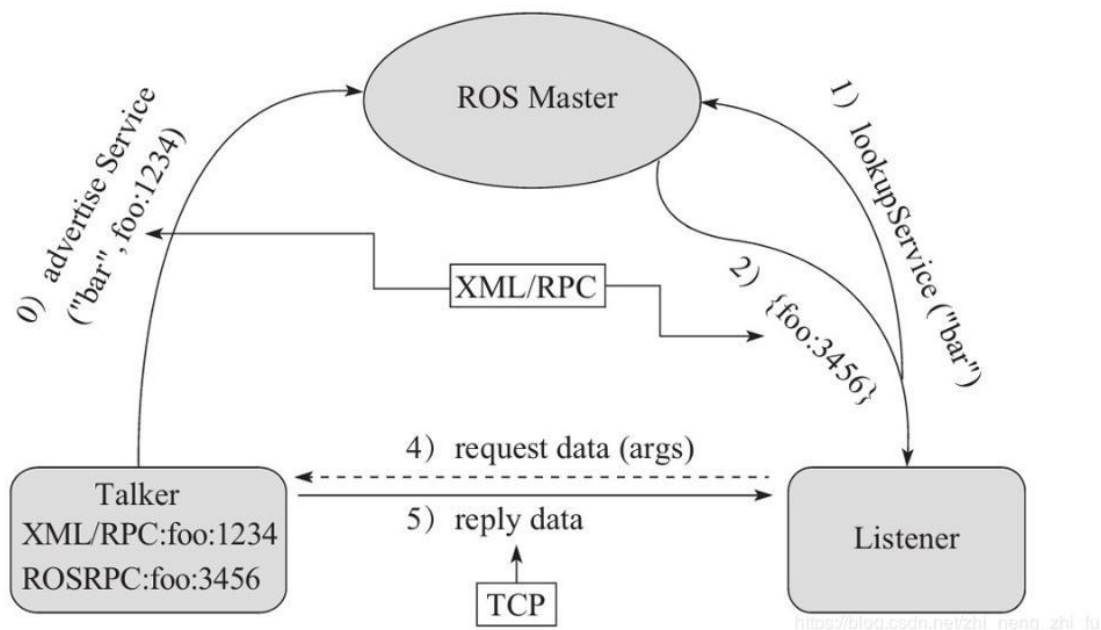
rostdemo@rostdemo-VirtualBox: ~/demo02_ws 56x12
rostdemo@rostdemo-VirtualBox:~/demo02_ws$ source ./devel/s
etup.bash
rostdemo@rostdemo-VirtualBox:~/demo02_ws$
```

2.2.1 服务通信理论模型

服务通信较之于话题通信更简单些，理论模型如下图所示，该模型中涉及到三个角色：

- ROS master(管理者)
- Server(服务端)
- Client(客户端)

ROS Master 负责保管 Server 和 Client 注册的信息，并匹配话题相同的 Server 与 Client，帮助 Server 与 Client 建立连接，连接建立后，Client 发送请求信息，Server 返回响应信息。



整个流程由以下步骤实现:

0.Server注册

Server 启动后, 会通过RPC在 ROS Master 中注册自身信息, 其中包含提供的服务的名称。ROS Master 会将节点的注册信息加入到注册表中。

1.Client注册

Client 启动后, 也会通过RPC在 ROS Master 中注册自身信息, 包含需要请求的服务的名称。ROS Master 会将节点的注册信息加入到注册表中。

2.ROS Master实现信息匹配

ROS Master 会根据注册表中的信息匹配Server和 Client, 并通过 RPC 向 Client 发送 Server 的 **TCP** 地址信息。

3.Client发送请求

Client 根据步骤2 响应的信息, 使用 TCP 与 Server 建立网络连接, 并发送请求数据。

4.Server发送响应

Server 接收、解析请求的数据, 并产生响应结果返回给 Client。

注意:

- 1.客户端请求被处理时, 需要保证服务器已经启动;
- 2.服务端和客户端都可以存在多个。

2.2.2 服务通信自定义srv

需求:

服务通信中, 客户端提交两个整数至服务端, 服务端求和并响应结果到客户端, 请创建服务器与客户端通信的数据载体。

流程:

srv 文件内的可用数据类型与 msg 文件一致，且定义 srv 实现流程与自定义 msg 实现流程类似：

1. 按照固定格式创建srv文件
2. 编辑配置文件
3. 编译生成中间文件

1.定义srv文件

服务通信中，数据分成两部分，请求与响应，在 srv 文件中请求和响应使用 `---` 分割，具体实现如下：

功能包下新建 srv 目录，添加 xxx.srv 文件，内容：

```
# 客户端请求时发送的两个数字
int32 num1
int32 num2
---
# 服务器响应发送的数据
int32 sum
Copy
```

2.编辑配置文件

package.xml中添加编译依赖与执行依赖

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
<!--
  exce_depend 以前对应的是 run_depend 现在非法
-->
Copy
```

CMakeLists.txt编辑 srv 相关配置

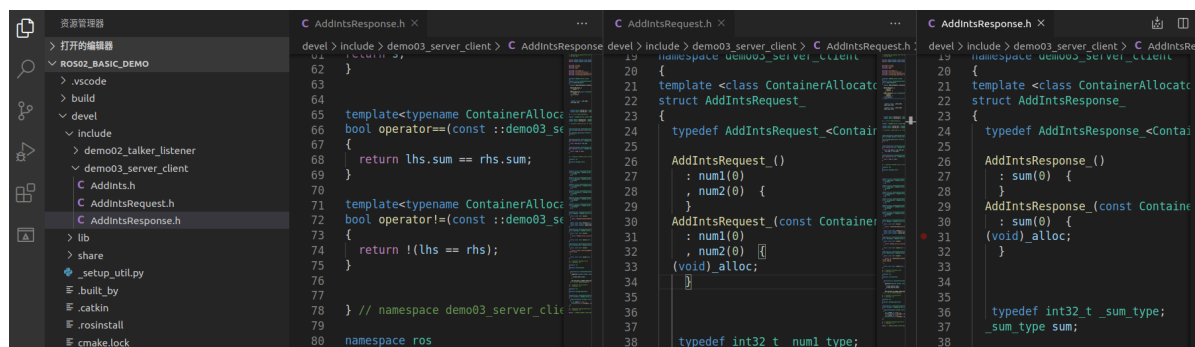
```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
# 需要加入 message_generation,必须有 std_msgs
Copy
add_service_files(
  FILES
  AddInts.srv
)
Copy
generate_messages(
  DEPENDENCIES
  std_msgs
)
Copy
```

注意：官网没有有在 catkin_package 中配置 message_runtime,经测试配置也可以

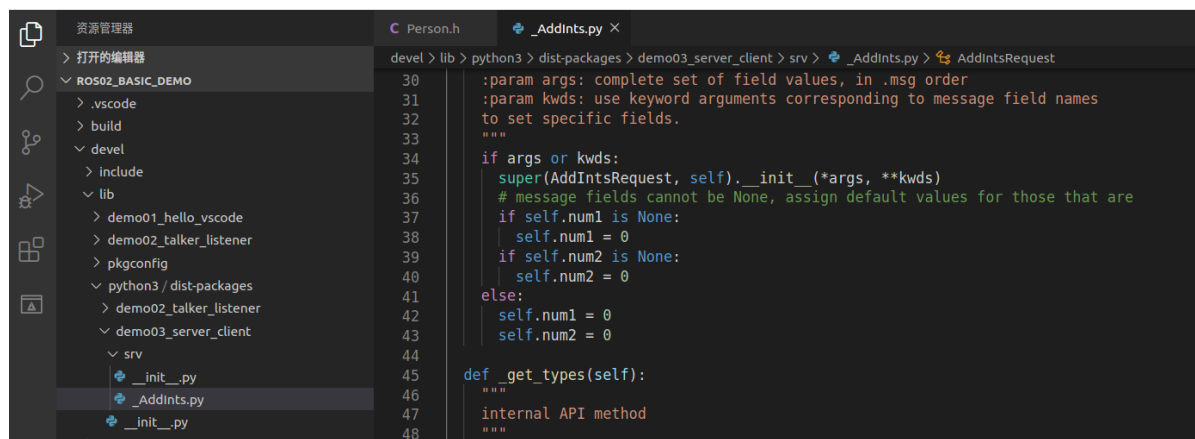
3.编译

编译后的中间文件查看:

C++ 需要调用的中间文件(.../工作空间/devel/include/包名/xxx.h)



Python 需要调用的中间文件(.../工作空间/devel/lib/python3/dist-packages/包名/srv)



后续调用相关 srv 时，是从这些中间文件调用的

2.2.3 服务通信自定义srv调用A(C++)

需求:

编写服务通信，客户端提交两个整数至服务端，服务端求和并响应结果到客户端。

分析:

在模型实现中，ROS master 不需要实现，而连接的建立也已经被封装了，需要关注的关键点有三个:

1. 服务端
2. 客户端
3. 数据

流程:

1. 编写服务端实现;
2. 编写客户端实现;
3. 编辑配置文件;
4. 编译并执行。

0.vscode配置

需要像之前自定义 msg 实现一样配置c_cpp_properties.json 文件，如果以前已经配置且没有变更工作空间，可以忽略，如果需要配置，配置方式与之前相同：

```
{
  "configurations": [
    {
      "browse": {
        "databaseFilename": "",
        "limitsSymbolsToIncludedHeaders": true
      },
      "includePath": [
        "/opt/ros/noetic/include/**",
        "/usr/include/**",
        "/xxx/yyy工作空间/devel/include/**" //配置 head 文件的路径
      ],
      "name": "ROS",
      "intelliSenseMode": "gcc-x64",
      "compilerPath": "/usr/bin/gcc",
      "cStandard": "c11",
      "cppStandard": "c++17"
    }
  ],
  "version": 4
}
```

Copy

1.服务端

```
/*
    需求：
        编写两个节点实现服务通信，客户端节点需要提交两个整数到服务器
        服务器需要解析客户端提交的数据，相加后，将结果响应回客户端，
        客户端再解析

    服务器实现：
        1.包含头文件
        2.初始化 ROS 节点
        3.创建 ROS 句柄
        4.创建 服务 对象
        5.回调函数处理请求并产生响应
        6.由于请求有多个，需要调用 ros::spin()

*/
#include "ros/ros.h"
#include "demo03_server_client/AddInts.h"

// bool 返回值由于标志是否处理成功
bool doReq(demo03_server_client::AddInts::Request& req,
           demo03_server_client::AddInts::Response& resp){
    int num1 = req.num1;
    int num2 = req.num2;

    ROS_INFO("服务器接收到的请求数据为:num1 = %d, num2 = %d", num1, num2);
```

```

//逻辑处理
if (num1 < 0 || num2 < 0)
{
    ROS_ERROR("提交的数据异常:数据不可以为负数");
    return false;
}

//如果没有异常，那么相加并将结果赋值给 resp
resp.sum = num1 + num2;
return true;

}

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ROS 节点
    ros::init(argc,argv,"AddInts_Server");
    // 3.创建 ROS 句柄
    ros::NodeHandle nh;
    // 4.创建 服务 对象
    ros::ServiceServer server = nh.advertiseService("AddInts",doReq);
    ROS_INFO("服务已经启动...");
    //      5.回调函数处理请求并产生响应
    //      6.由于请求有多个，需要调用 ros::spin()
    ros::spin();
    return 0;
}
Copy

```

2.客户端

```

/*
    需求：
        编写两个节点实现服务通信，客户端节点需要提交两个整数到服务器
        服务器需要解析客户端提交的数据，相加后，将结果响应回客户端，
        客户端再解析

    服务器实现：
        1.包含头文件
        2.初始化 ROS 节点
        3.创建 ROS 句柄
        4.创建 客户端 对象
        5.请求服务，接收响应

*/
// 1.包含头文件
#include "ros/ros.h"
#include "demo03_server_client/AddInts.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");

    // 调用时动态传值,如果通过 launch 的 args 传参，需要传递的参数个数 +3
    if (argc != 3)

```

```

// if (argc != 5)//launch 传参(0-文件路径 1传入的参数 2传入的参数 3节点名称 4日志路
径)
{
    ROS_ERROR("请提交两个整数");
    return 1;
}

// 2.初始化 ROS 节点
ros::init(argc,argv,"AddInts_Client");
// 3.创建 ROS 句柄
ros::NodeHandle nh;
// 4.创建 客户端 对象
ros::ServiceClient client = nh.serviceClient<demo03_server_client::AddInts>
("AddInts");
//等待服务启动成功
//方式1
ros::service::waitForService("AddInts");
//方式2
// client.waitForExistence();
// 5.组织请求数据
demo03_server_client::AddInts ai;
ai.request.num1 = atoi(argv[1]);
ai.request.num2 = atoi(argv[2]);
// 6.发送请求,返回 bool 值,标记是否成功
bool flag = client.call(ai);
// 7.处理响应
if (flag)
{
    ROS_INFO("请求正常处理,响应结果:%d",ai.response.sum);
}
else
{
    ROS_ERROR("请求处理失败...");
    return 1;
}

return 0;
}
Copy

```

3.配置 CMakeLists.txt

```

add_executable(AddInts_Server src/AddInts_Server.cpp)
add_executable(AddInts_Client src/AddInts_Client.cpp)

add_dependencies(AddInts_Server ${PROJECT_NAME}_gencpp)
add_dependencies(AddInts_Client ${PROJECT_NAME}_gencpp)

target_link_libraries(AddInts_Server
    ${catkin_LIBRARIES}
)
target_link_libraries(AddInts_Client
    ${catkin_LIBRARIES}
)

```

Copy

4.执行

流程:

- 需要先启动服务: `roslaunch 包名 服务`
- 然后再调用客户端: `roslaunch 包名 客户端 参数1 参数2`

结果:

会根据提交的数据响应相加后的结果。

注意:

如果先启动客户端, 那么会导致运行失败

优化:

在客户端发送请求前添加: `client.waitForExistence();`

或: `ros::service::waitForService("AddInts");`

这是一个阻塞式函数, 只有服务启动成功后才会继续执行

此处可以使用 launch 文件优化, 但是需要注意 args 传参特点

2.2.4 服务通信自定义srv调用B(Python)

需求:

编写服务通信, 客户端提交两个整数至服务端, 服务端求和并响应结果到客户端。

分析:

在模型实现中, ROS master 不需要实现, 而连接的建立也已经被封装了, 需要关注的关键点有三个:

1. 服务端
2. 客户端
3. 数据

流程:

1. 编写服务端实现;
2. 编写客户端实现;
3. 为python文件添加可执行权限;
4. 编辑配置文件;
5. 编译并执行。

0.vscode配置

需要像之前自定义 msg 实现一样配置settings.json 文件, 如果以前已经配置且没有变更工作空间, 可以忽略, 如果需要配置, 配置方式与之前相同:

```
{
  "python.autoComplete.extraPaths": [
    "/opt/ros/noetic/lib/python3/dist-packages",
  ]
}
```

Copy

1.服务端

```
#!/usr/bin/env python
"""
    需求：
        编写两个节点实现服务通信，客户端节点需要提交两个整数到服务器
        服务器需要解析客户端提交的数据，相加后，将结果响应回客户端，
        客户端再解析

    服务器端实现：
        1.导包
        2.初始化 ROS 节点
        3.创建服务对象
        4.回调函数处理请求并产生响应
        5.spin 函数

    """
# 1.导包
import rospy
from demo03_server_client.srv import AddInts,AddIntsRequest,AddIntsResponse
# 回调函数的参数是请求对象，返回值是响应对象
def doReq(req):
    # 解析提交的数据
    sum = req.num1 + req.num2
    rospy.loginfo("提交的数据:num1 = %d, num2 = %d, sum = %d",req.num1, req.num2,
sum)

    # 创建响应对象，赋值并返回
    # resp = AddIntsResponse()
    # resp.sum = sum
    resp = AddIntsResponse(sum)
    return resp

if __name__ == "__main__":
    # 2.初始化 ROS 节点
    rospy.init_node("addints_server_p")
    # 3.创建服务对象
    server = rospy.Service("AddInts",AddInts,doReq)
    # 4.回调函数处理请求并产生响应
    # 5.spin 函数
    rospy.spin()
Copy
```

2.客户端

```
#!/usr/bin/env python

"""
    需求：
        编写两个节点实现服务通信，客户端节点需要提交两个整数到服务器
        服务器需要解析客户端提交的数据，相加后，将结果响应回客户端，
        客户端再解析

    客户端实现：
        1.导包
```

- 2.初始化 ROS 节点
- 3.创建请求对象
- 4.发送请求
- 5.接收并处理响应

优化:

加入数据的动态获取

```
"""
#1.导包
import rospy
from demo03_server_client.srv import *
import sys

if __name__ == "__main__":

    #优化实现
    if len(sys.argv) != 3:
        rospy.logerr("请正确提交参数")
        sys.exit(1)

    # 2.初始化 ROS 节点
    rospy.init_node("AddInts_Client_p")
    # 3.创建请求对象
    client = rospy.ServiceProxy("AddInts",AddInts)
    # 请求前,等待服务已经就绪
    # 方式1:
    # rospy.wait_for_service("AddInts")
    # 方式2
    client.wait_for_service()
    # 4.发送请求,接收并处理响应
    # 方式1
    # resp = client(3,4)
    # 方式2
    # resp = client(AddIntsRequest(1,5))
    # 方式3
    req = AddIntsRequest()
    # req.num1 = 100
    # req.num2 = 200

    #优化
    req.num1 = int(sys.argv[1])
    req.num2 = int(sys.argv[2])

    resp = client.call(req)
    rospy.loginfo("响应结果:%d",resp.sum)
Copy
```

3.设置权限

终端下进入 scripts 执行: `chmod +x *.py`

4.配置 CMakeLists.txt

CMakeLists.txt

```
catkin_install_python(PROGRAMS
  scripts/AddInts_Server_p.py
  scripts/AddInts_Client_p.py
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
Copy
```

5.执行

流程:

- 需要先启动服务: `roslrun 包名 服务`
- 然后再调用客户端: `roslrun 包名 客户端 参数1 参数2`

结果:

会根据提交的数据响应相加后的结果。

2.3 参数服务器

参数服务器在ROS中主要用于实现不同节点之间的数据共享。参数服务器相当于是独立于所有节点的一个公共容器, 可以将数据存储在容器中, 被不同的节点调用, 当然不同的节点也可以往其中存储数据, 关于参数服务器的典型应用场景如下:

导航实现时, 会进行路径规划, 比如: 全局路径规划, 设计一个从出发点到目标点的大致路径。本地路径规划, 会根据当前路况生成时时的行进路径

上述场景中, 全局路径规划和本地路径规划时, 就会使用到参数服务器:

- 路径规划时, 需要参考小车的尺寸, 我们可以将这些尺寸信息存储到参数服务器, 全局路径规划节点与本地路径规划节点都可以从参数服务器中调用这些参数

参数服务器, 一般适用于存在数据共享的一些应用场景。

概念

以共享的方式实现不同节点之间数据交互的通信模式。

作用

存储一些多节点共享的数据, 类似于全局变量。

案例

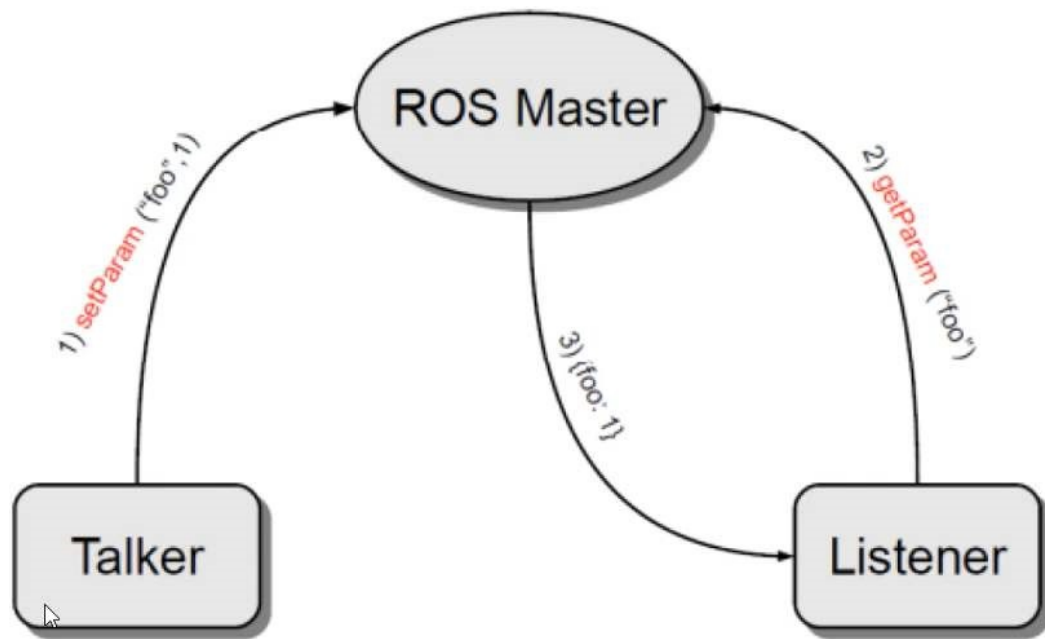
实现参数增删改查操作。

2.3.1 参数服务器理论模型

参数服务器实现是最为简单的, 该模型如下图所示, 该模型中涉及到三个角色:

- ROS Master (管理者)
- Talker (参数设置者)
- Listener (参数调用者)

ROS Master 作为一个公共容器保存参数，Talker 可以向容器中设置参数，Listener 可以获取参数。



整个流程由以下步骤实现:

1.Talker 设置参数

Talker 通过 RPC 向参数服务器发送参数(包括参数名与参数值)，ROS Master 将参数保存到参数列表中。

2.Listener 获取参数

Listener 通过 RPC 向参数服务器发送参数查找请求，请求中包含要查找的参数名。

3.ROS Master 向 Listener 发送参数值

ROS Master 根据步骤2请求提供的参数名查找参数值，并将查询结果通过 RPC 发送给 Listener。

参数可使用数据类型:

- 32-bit integers
- booleans
- strings
- doubles
- iso8601 dates
- lists
- base64-encoded binary data
- 字典

注意:参数服务器不是为高性能而设计的，因此最好用于存储静态的非二进制的简单数据

2.3.2 参数操作A(C++)

需求:实现参数服务器参数的增删改查操作。

在 C++ 中实现参数服务器数据的增删改查，可以通过两套 API 实现:

- `ros::NodeHandle`
- `ros::param`

下面为具体操作演示

1.参数服务器新增(修改)参数

```
/*
    参数服务器操作之新增与修改(二者API一样)_C++实现:
    在 roscpp 中提供了两套 API 实现参数操作
    ros::NodeHandle
        setParam("键",值)
    ros::param
        set("键","值")

    示例:分别设置整形、浮点、字符串、bool、列表、字典等类型参数
        修改(相同的键,不同的值)

*/
#include "ros/ros.h"

int main(int argc, char *argv[])
{
    ros::init(argc,argv,"set_update_param");

    std::vector<std::string> stus;
    stus.push_back("zhangsan");
    stus.push_back("李四");
    stus.push_back("王五");
    stus.push_back("孙大脑袋");

    std::map<std::string,std::string> friends;
    friends["guo"] = "huang";
    friends["yuang"] = "xiao";

    //NodeHandle-----
    ros::NodeHandle nh;
    nh.setParam("nh_int",10); //整型
    nh.setParam("nh_double",3.14); //浮点型
    nh.setParam("nh_bool",true); //bool
    nh.setParam("nh_string","hello NodeHandle"); //字符串
    nh.setParam("nh_vector",stus); // vector
    nh.setParam("nh_map",friends); // map

    //修改演示(相同的键,不同的值)
    nh.setParam("nh_int",10000);

    //param-----
    ros::param::set("param_int",20);
    ros::param::set("param_double",3.14);
    ros::param::set("param_string","Hello Param");
    ros::param::set("param_bool",false);
    ros::param::set("param_vector",stus);
    ros::param::set("param_map",friends);

    //修改演示(相同的键,不同的值)
    ros::param::set("param_int",20000);

    return 0;
}
```

2.参数服务器获取参数

```

/*
参数服务器操作之查询_C++实现：
在 roscpp 中提供了两套 API 实现参数操作
ros::NodeHandle

    param(键,默认值)
        存在，返回对应结果，否则返回默认值

    getParam(键,存储结果的变量)
        存在,返回 true,且将值赋值给参数2
        若果键不存在，那么返回值为 false，且不为参数2赋值

    getParamCached(键,存储结果的变量)--提高变量获取效率
        存在,返回 true,且将值赋值给参数2
        若果键不存在，那么返回值为 false，且不为参数2赋值

    getParamNames(std::vector<std::string>)
        获取所有的键,并存储在参数 vector 中

    hasParam(键)
        是否包含某个键，存在返回 true，否则返回 false

    searchParam(参数1, 参数2)
        搜索键，参数1是被搜索的键，参数2存储搜索结果的变量

ros::param ----- 与 NodeHandle 类似

*/

#include "ros/ros.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    ros::init(argc,argv,"get_param");

    //NodeHandle-----
    /*
    ros::NodeHandle nh;
    // param 函数
    int res1 = nh.param("nh_int",100); // 键存在
    int res2 = nh.param("nh_int2",100); // 键不存在
    ROS_INFO("param获取结果:%d,%d",res1,res2);

    // getParam 函数
    int nh_int_value;
    double nh_double_value;
    bool nh_bool_value;
    std::string nh_string_value;

```

```

std::vector<std::string> stus;
std::map<std::string, std::string> friends;

nh.getParam("nh_int",nh_int_value);
nh.getParam("nh_double",nh_double_value);
nh.getParam("nh_bool",nh_bool_value);
nh.getParam("nh_string",nh_string_value);
nh.getParam("nh_vector",stus);
nh.getParam("nh_map",friends);

ROS_INFO("getParam获取的结果:%d,%.2f,%s,%d",
        nh_int_value,
        nh_double_value,
        nh_string_value.c_str(),
        nh_bool_value
    );
for (auto &&stu : stus)
{
    ROS_INFO("stus 元素:%s",stu.c_str());
}

for (auto &&f : friends)
{
    ROS_INFO("map 元素:%s = %s",f.first.c_str(), f.second.c_str());
}

// getParamCached()
nh.getParamCached("nh_int",nh_int_value);
ROS_INFO("通过缓存获取数据:%d",nh_int_value);

//getParamNames()
std::vector<std::string> param_names1;
nh.getParamNames(param_names1);
for (auto &&name : param_names1)
{
    ROS_INFO("名称解析name = %s",name.c_str());
}
ROS_INFO("-----");

ROS_INFO("存在 nh_int 吗? %d",nh.hasParam("nh_int"));
ROS_INFO("存在 nh_intttt 吗? %d",nh.hasParam("nh_intttt"));

std::string key;
nh.searchParam("nh_int",key);
ROS_INFO("搜索键:%s",key.c_str());
*/
//param-----
ROS_INFO("+++++++");
int res3 = ros::param::param("param_int",20); //存在
int res4 = ros::param::param("param_int2",20); // 不存在返回默认
ROS_INFO("param获取结果:%d,%d",res3,res4);

// getParam 函数
int param_int_value;
double param_double_value;
bool param_bool_value;
std::string param_string_value;
std::vector<std::string> param_stus;

```

```

std::map<std::string, std::string> param_friends;

ros::param::get("param_int", param_int_value);
ros::param::get("param_double", param_double_value);
ros::param::get("param_bool", param_bool_value);
ros::param::get("param_string", param_string_value);
ros::param::get("param_vector", param_stus);
ros::param::get("param_map", param_friends);

ROS_INFO("getParam获取的结果:%d,%.2f,%s,%d",
        param_int_value,
        param_double_value,
        param_string_value.c_str(),
        param_bool_value
    );
for (auto &&stu : param_stus)
{
    ROS_INFO("stus 元素:%s", stu.c_str());
}

for (auto &&f : param_friends)
{
    ROS_INFO("map 元素:%s = %s", f.first.c_str(), f.second.c_str());
}

// getParamCached()
ros::param::getCached("param_int", param_int_value);
ROS_INFO("通过缓存获取数据:%d", param_int_value);

//getParamNames()
std::vector<std::string> param_names2;
ros::param::getParamNames(param_names2);
for (auto &&name : param_names2)
{
    ROS_INFO("名称解析name = %s", name.c_str());
}
ROS_INFO("-----");

ROS_INFO("存在 param_int 吗? %d", ros::param::has("param_int"));
ROS_INFO("存在 param_intttt 吗? %d", ros::param::has("param_intttt"));

std::string key;
ros::param::search("param_int", key);
ROS_INFO("搜索键:%s", key.c_str());

return 0;
}
Copy

```

3.参数服务器删除参数

```

/*
参数服务器操作之删除_C++实现:

ros::NodeHandle
    deleteParam("键")
    根据键删除参数, 删除成功, 返回 true, 否则(参数不存在), 返回 false

```

```

ros::param
del("键")
    根据键删除参数，删除成功，返回 true，否则(参数不存在)，返回 false

*/
#include "ros/ros.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    ros::init(argc, argv, "delete_param");

    ros::NodeHandle nh;
    bool r1 = nh.deleteParam("nh_int");
    ROS_INFO("nh 删除结果:%d", r1);

    bool r2 = ros::param::del("param_int");
    ROS_INFO("param 删除结果:%d", r2);

    return 0;
}

```

2.3.3 参数操作B(Python)

需求:实现参数服务器参数的增删改查操作。

1.参数服务器新增(修改)参数

```

#!/usr/bin/env python
"""
    参数服务器操作之新增与修改(二者API一样)_Python实现:
"""

import rospy

if __name__ == "__main__":
    rospy.init_node("set_update_paramter_p")

    # 设置各种类型参数
    rospy.set_param("p_int", 10)
    rospy.set_param("p_double", 3.14)
    rospy.set_param("p_bool", True)
    rospy.set_param("p_string", "hello python")
    rospy.set_param("p_list", ["hello", "haha", "xixi"])
    rospy.set_param("p_dict", {"name": "hulu", "age": 8})

    # 修改
    rospy.set_param("p_int", 100)
Copy

```

2.参数服务器获取参数

```
#!/usr/bin/env python

"""
    参数服务器操作之查询_Python实现：
        get_param(键,默认值)
            当键存在时，返回对应的值，如果不存在返回默认值
        get_param_cached
        get_param_names
        has_param
        search_param
"""

import rospy

if __name__ == "__main__":
    rospy.init_node("get_param_p")

    #获取参数
    int_value = rospy.get_param("p_int",10000)
    double_value = rospy.get_param("p_double")
    bool_value = rospy.get_param("p_bool")
    string_value = rospy.get_param("p_string")
    p_list = rospy.get_param("p_list")
    p_dict = rospy.get_param("p_dict")

    rospy.loginfo("获取的数据:%d,%.2f,%d,%s",
                  int_value,
                  double_value,
                  bool_value,
                  string_value)
    for ele in p_list:
        rospy.loginfo("ele = %s", ele)

    rospy.loginfo("name = %s, age = %d",p_dict["name"],p_dict["age"])

    # get_param_cached
    int_cached = rospy.get_param_cached("p_int")
    rospy.loginfo("缓存数据:%d",int_cached)

    # get_param_names
    names = rospy.get_param_names()
    for name in names:
        rospy.loginfo("name = %s",name)

    rospy.loginfo("-"*80)

    # has_param
    flag = rospy.has_param("p_int")
    rospy.loginfo("包含p_int吗? %d",flag)

    # search_param
    key = rospy.search_param("p_int")
    rospy.loginfo("搜索的键 = %s",key)
```

Copy

3.参数服务器删除参数

```
#!/usr/bin/env python
"""
    参数服务器操作之删除_Python实现：
    rospy.delete_param("键")
    键存在时，可以删除成功，键不存在时，会抛出异常
"""
import rospy

if __name__ == "__main__":
    rospy.init_node("delete_param_p")

    try:
        rospy.delete_param("p_int")
    except Exception as e:
        rospy.loginfo("删除失败")
```

2.4 常用命令

机器人系统中启动的节点少则几个，多则十几个、几十个，不同的节点名称各异，通信时使用话题、服务、消息、参数等等都各不相同，一个显而易见的问题是：当需要自定义节点和其他某个已经存在的节点通信时，如何获取对方的话题、以及消息载体的格式呢？

在 ROS 同提供了一些实用的命令行工具，可以用于获取不同节点的各类信息，常用的命令如下：

- rosnodetop : 操作节点
- rostopic : 操作话题
- rosservice : 操作服务
- rosmmsg : 操作msg消息
- rossrv : 操作srv消息
- rosparm : 操作参数

作用

和之前介绍的文件系统操作命令比较，文件操作命令是静态的，操作的是磁盘上的文件，而上述命令是动态的，在ROS程序启动后，可以动态的获取运行中的节点或参数的相关信息。

案例

本节将借助于2.1、2.2和2.3的通信实现介绍相关命令的基本使用，并通过练习ROS内置的小海龟例程来强化命令的应用。

另请参考：

- <http://wiki.ros.org/ROS/CommandLineTools>

ROS Command-line tools

目录

1. ROS Command-line tools
 1. Common user tools
 1. rosbag
 2. rosbash
 3. roscd
 4. rosclean
 5. roscore
 6. rosdep
 7. rosed
 8. roscrcat
 9. roscrcat
 10. roslaunch
 11. roslaunch
 12. roslocate
 13. rosmake
 14. rosmmsg
 15. rosnodet
 16. rospack
 17. rosparam
 18. rossrv
 19. rosservice
 20. rosstack
 21. rostopic
 22. rosver
 2. Graphical tools
 1. rqt_bag
 2. rqt_deps
 3. rqt_graph
 4. rqt_plot
 3. Less-used tools
 1. gendeps

维基

[Distributions](#)
[ROS/Installation](#)
[ROS/Tutorials](#)
[RecentChanges](#)
[ROS/CommandLineT](#)

网页

[只读网页](#)
[信息](#)
[附件](#)

更多操作:

[源码](#)

[执行](#)

用户

[登录](#)

2.4.1 rosnodet

rosnodet 是用于获取节点信息的命令

rosnodet ping	测试到节点的连接状态
rosnodet list	列出活动节点
rosnodet info	打印节点信息
rosnodet machine	列出指定设备上节点
rosnodet kill	杀死某个节点
rosnodet cleanup	清除不可连接的节点

Copy

- rosnodet ping
测试到节点的连接状态
- rosnodet list
列出活动节点
- rosnodet info
打印节点信息
- rosnodet machine
列出指定设备上的节点
- rosnodet kill
杀死某个节点
- rosnodet cleanup

清除无用节点，启动乌龟节点，然后 ctrl + c 关闭，该节点并没被彻底清除，可以使用 cleanup 清除节点

2.4.2 rostopic

rostopic包含rostopic命令行工具，用于显示有关ROS 主题的调试信息，包括发布者，订阅者，发布频率和ROS消息。它还包含一个实验性Python库，用于动态获取有关主题的信息并与之交互。

```
rostopic bw      显示主题使用的带宽
rostopic delay   显示带有 header 的主题延迟
rostopic echo    打印消息到屏幕
rostopic find    根据类型查找主题
rostopic hz      显示主题的发布频率
rostopic info    显示主题相关信息
rostopic list    显示所有活动状态下的主题
rostopic pub     将数据发布到主题
rostopic type    打印主题类型
Copy
```

- **rostopic list(-v)**

直接调用即可，控制台将打印当前运行状态下的主题名称

rostopic list -v : 获取话题详情(比如列出：发布者和订阅者个数...)

- **rostopic pub**

可以直接调用命令向订阅者发布消息

为roboware 自动生成的 发布/订阅 模型案例中的 订阅者 发布一条字符串

```
rostopic pub /主题名称 消息类型 消息内容
rostopic pub /chatter std_msgs/gagaxixi
Copy
```

为 小乌龟案例的 订阅者 发布一条运动信息

```
rostopic pub /turtle1/cmd_vel geometry_msgs/Twist
"linear:
  x: 1.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 2.0"
//只发布一次运动信息

rostopic pub -r 10 /turtle1/cmd_vel geometry_msgs/Twist
"linear:
  x: 1.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 2.0"
// 以 10HZ 的频率循环发送运动信息
```

Copy

- **rostopic echo**
获取指定话题当前发布的消息
- **rostopic info**
获取当前话题的小关信息
消息类型
发布者信息
订阅者信息
- **rostopic type**
列出话题的消息类型
- **rostopic find 消息类型**
根据消息类型查找话题
- **rostopic delay**
列出消息头信息
- **rostopic hz**
列出消息发布频率
- **rostopic bw**
列出消息发布带宽

2.4.3 rosmmsg

rosmmsg是用于显示有关 ROS消息类型的 信息的命令行工具。

rosmmsg 演示

```
rosmmsg show    显示消息描述
rosmmsg info    显示消息信息
rosmmsg list    列出所有消息
rosmmsg md5     显示 md5 加密后的消息
rosmmsg package 显示某个功能包下的所有消息
rosmmsg packages 列出包含消息的功能包
Copy
```

- rosmmsg list
会列出当前 ROS 中的所有 msg
- rosmmsg packages
列出包含消息的所有包
- rosmmsg package
列出某个包下的所有msg

```
//rosmmsg package 包名
rosmmsg package turtlesim
Copy
```

- rosmmsg show

显示消息描述

```
//rosmg show 消息名称
rosmg show turtlesim/Pose
结果:
float32 x
float32 y
float32 theta
float32 linear_velocity
float32 angular_velocity
Copy
```

- rosmg info

作用与 rosmg show 一样

- rosmg md5 (资

料:http://wiki.ros.org/ROS/Technical%20Overview#Message_serialization_and_msg_MD5_sums)

一种校验算法, 保证数据传输的一致性

2.4.4 rosservice

rosservice包含用于列出和查询ROS[Services](#)的rosservice命令行工具。

调用部分服务时, 如果对相关工作空间没有配置 path, 需要进入工作空间调用 source ./devel/setup.bash

```
rosservice args 打印服务参数
rosservice call 使用提供的参数调用服务
rosservice find 按照服务类型查找服务
rosservice info 打印有关服务的信息
rosservice list 列出所有活动的服务
rosservice type 打印服务类型
rosservice uri 打印服务的 ROSRPC uri
Copy
```

- rosservice list

列出所有活动的 service

```
~ rosservice list
/clear
/kill
/listener/get_loggers
/listener/set_logger_level
/reset
/rosout/get_loggers
/rosout/set_logger_level
/rostopic_4985_1578723066421/get_loggers
/rostopic_4985_1578723066421/set_logger_level
/rostopic_5582_1578724343069/get_loggers
/rostopic_5582_1578724343069/set_logger_level
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
```

```
/turtlesim/get_loggers
/turtlesim/set_logger_level
Copy
```

- rosservice args

打印服务参数

```
rosservice args /spawn
x y theta name
Copy
```

- rosservice call

调用服务

为小乌龟的案例生成一只新的乌龟

```
rosservice call /spawn "x: 1.0
y: 2.0
theta: 0.0
name: 'xxx'"
name: "xxx"

//生成一只叫 xxx 的乌龟
Copy
```

- rosservice find

根据消息类型获取话题

- rosservice info

获取服务话题详情

- rosservice type

获取消息类型

- rosservice uri

获取服务器 uri

2.4.5 rossrv

rossrv 是用于显示有关 ROS 服务类型的信息的命令行工具，与 rosmmsg 使用语法高度雷同。

```
rossrv show    显示服务消息详情
rossrv info    显示服务消息相关信息
rossrv list    列出所有服务信息
rossrv md5     显示 md5 加密后的服务消息
rossrv package 显示某个包下所有服务消息
rossrv packages 显示包含服务消息的所有包
Copy
```

- rossrv list

会列出当前 ROS 中的所有 srv 消息

- rossrv packages

列出包含服务消息的所有包

- rossrv package

列出某个包下的所有msg

```
//rossrv package 包名
rossrv package turtlesim
Copy
```

- rossrv show

显示消息描述

```
//rossrv show 消息名称
rossrv show turtlesim/Spawn
结果:
float32 x
float32 y
float32 theta
string name
---
string name
Copy
```

- rossrv info

作用与 rossrv show 一致

- rossrv md5

对 service 数据使用 md5 校验(加密)

2.4.6 rosparam

rosparam包含rosparam命令行工具，用于使用YAML编码文件在参数服务器上获取和设置ROS参数。

```
rosparam set    设置参数
rosparam get    获取参数
rosparam load   从外部文件加载参数
rosparam dump   将参数写出到外部文件
rosparam delete 删除参数
rosparam list   列出所有参数
Copy
```

- rosparam list

列出所有参数

```
rosparam list

//默认结果
/rosdistro
/roslaunch/uris/host_helloros_virtual_machine__42911
/rosversion
/run_id
Copy
```

- rosparam set

设置参数

```
rosparam set name huluwa

//再次调用 rosparam list 结果
/name
/rosdistro
/roslaunch/uris/host_helloros_virtual_machine__42911
/rosversion
/run_id
Copy
```

- rosparam get

获取参数

```
rosparam get name

//结果
huluwa
Copy
```

- rosparam delete

删除参数

```
rosparam delete name

//结果
//去除了name
Copy
```

- rosparam load(先准备 yaml 文件)

从外部文件加载参数

```
rosparam load xxx.yaml
Copy
```

- rosparam dump

将参数写出到外部文件

```
rosparam dump yyy.yaml
```

2.5 通信机制实操

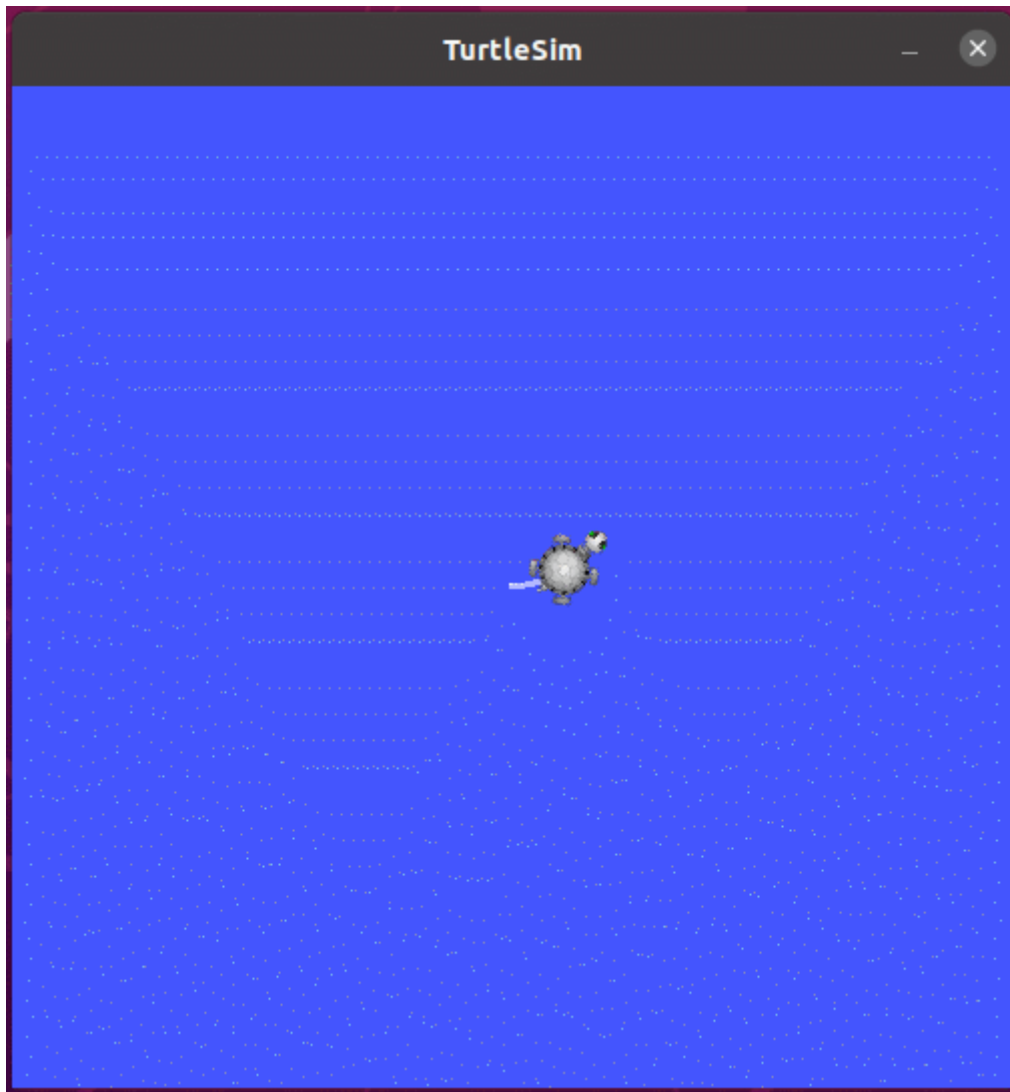
本节主要是通过ROS内置的turtlesim案例，结合已经介绍ROS命令获取节点、话题、话题消息、服务、服务消息与参数的信息，最终再以编码的方式实现乌龟运动的控制、乌龟位姿的订阅、乌龟生成与乌龟窗体背景颜色的修改。

目的:熟悉、强化通信模式应用

2.5.1 实操01_话题发布

需求描述:编码实现乌龟运动控制，让小乌龟做圆周运动。

结果演示:



实现分析:

1. 乌龟运动控制实现，关键节点有两个，一个是乌龟运动显示节点 `turtlesim_node`，另一个是控制节点，二者是订阅发布模式实现通信的，乌龟运动显示节点直接调用即可，运动控制节点之前是使用的 `turtle_teleop_key` 通过键盘控制，现在需要自定义控制节点。
2. 控制节点自实现时，首先需要了解控制节点与显示节点通信使用的话题与消息，可以使用 `ros` 命令结合计算图来获取。
3. 了解了话题与消息之后，通过 C++ 或 Python 编写运动控制节点，通过指定的话题，按照一定的逻辑发布消息即可。

实现流程:

1. 通过计算图结合 `ros` 命令获取话题与消息信息。
2. 编码实现运动控制节点。
3. 启动 `roscore`、`turtlesim_node` 以及自定义的控制节点，查看运行结果。

1.话题与消息获取

准备: 先启动键盘控制乌龟运动案例。

1.1话题获取

获取话题: /turtle1/cmd_vel

通过计算图查看话题，启动计算图：

```
rqt_graph  
Copy
```

或者通过 rostopic 列出话题：

```
rostopic list  
Copy
```

1.2消息获取

获取消息类型: geometry_msgs/Twist

```
rostopic type /turtle1/cmd_vel  
Copy
```

获取消息格式:

```
rosmmsg info geometry_msgs/Twist  
Copy
```

响应结果:

```
geometry_msgs/Vector3 linear  
  float64 x  
  float64 y  
  float64 z  
geometry_msgs/Vector3 angular  
  float64 x  
  float64 y  
  float64 z  
Copy
```

linear(线速度) 下的xyz分别对应在x、y和z方向上的速度(单位是 m/s)；

angular(角速度)下的xyz分别对应x轴上的翻滚、y轴上俯仰和z轴上偏航的速度(单位是rad/s)。

详情请查看补充资料。

2.实现发布节点

创建功能包需要依赖的功能包: roscpp rospy std_msgs geometry_msgs

实现方案A: C++

```
/*  
  编写 ROS 节点，控制小乌龟画圆
```

准备工作：

1. 获取topic(已知: /turtle1/cmd_vel)
2. 获取消息类型(已知: geometry_msgs/Twist)
3. 运行前, 注意先启动 turtlesim_node 节点

实现流程：

1. 包含头文件
2. 初始化 ROS 节点
3. 创建发布者对象
4. 循环发布运动控制消息

```
*/

#include "ros/ros.h"
#include "geometry_msgs/Twist.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ROS 节点
    ros::init(argc,argv,"control");
    ros::NodeHandle nh;
    // 3.创建发布者对象
    ros::Publisher pub = nh.advertise<geometry_msgs::Twist>
("/turtle1/cmd_vel",1000);
    // 4.循环发布运动控制消息
    //4-1.组织消息
    geometry_msgs::Twist msg;
    msg.linear.x = 1.0;
    msg.linear.y = 0.0;
    msg.linear.z = 0.0;

    msg.angular.x = 0.0;
    msg.angular.y = 0.0;
    msg.angular.z = 2.0;

    //4-2.设置发送频率
    ros::Rate r(10);
    //4-3.循环发送
    while (ros::ok())
    {
        pub.publish(msg);

        ros::spinOnce();
    }

    return 0;
}
Copy
```

配置文件此处略

实现方案B: Python

```
#!/usr/bin/env python
"""
```

编写 ROS 节点，控制小乌龟画圆

准备工作：

1. 获取topic(已知：/turtle1/cmd_vel)
2. 获取消息类型(已知：geometry_msgs/Twist)
3. 运行前，注意先启动 turtlesim_node 节点

实现流程：

1. 导包
2. 初始化 ROS 节点
3. 创建发布者对象
4. 循环发布运动控制消息

```
"""

import rospy
from geometry_msgs.msg import Twist

if __name__ == "__main__":
    # 2. 初始化 ROS 节点
    rospy.init_node("control_circle_p")
    # 3. 创建发布者对象
    pub = rospy.Publisher("/turtle1/cmd_vel", Twist, queue_size=1000)
    # 4. 循环发布运动控制消息
    rate = rospy.Rate(10)
    msg = Twist()
    msg.linear.x = 1.0
    msg.linear.y = 0.0
    msg.linear.z = 0.0
    msg.angular.x = 0.0
    msg.angular.y = 0.0
    msg.angular.z = 0.5

    while not rospy.is_shutdown():
        pub.publish(msg)
        rate.sleep()

Copy
```

权限设置以及配置文件此处略

3.运行

首先，启动 roscore；

然后启动乌龟显示节点；

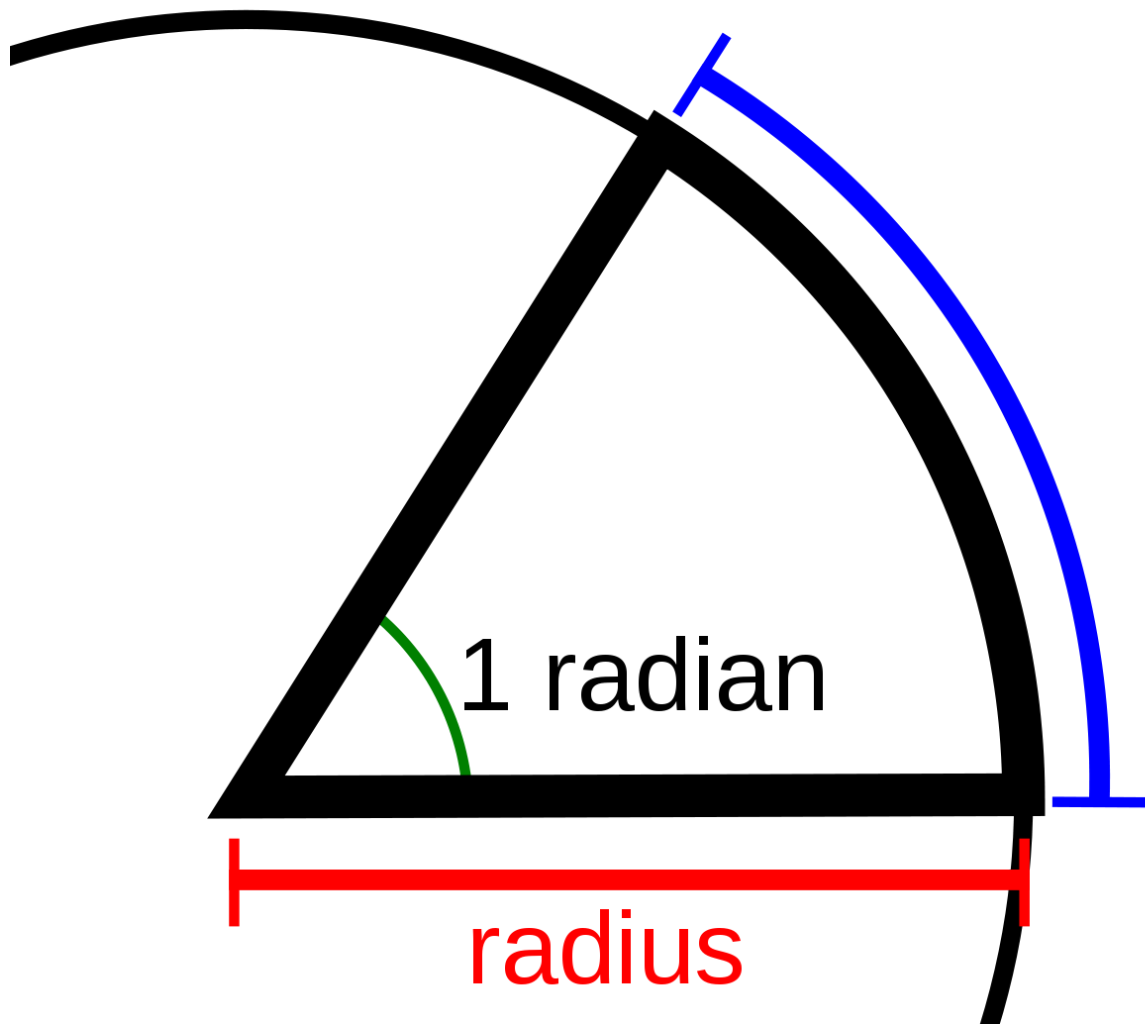
最后执行运动控制节点；

最终执行结果与演示结果类似。

补充资料1:

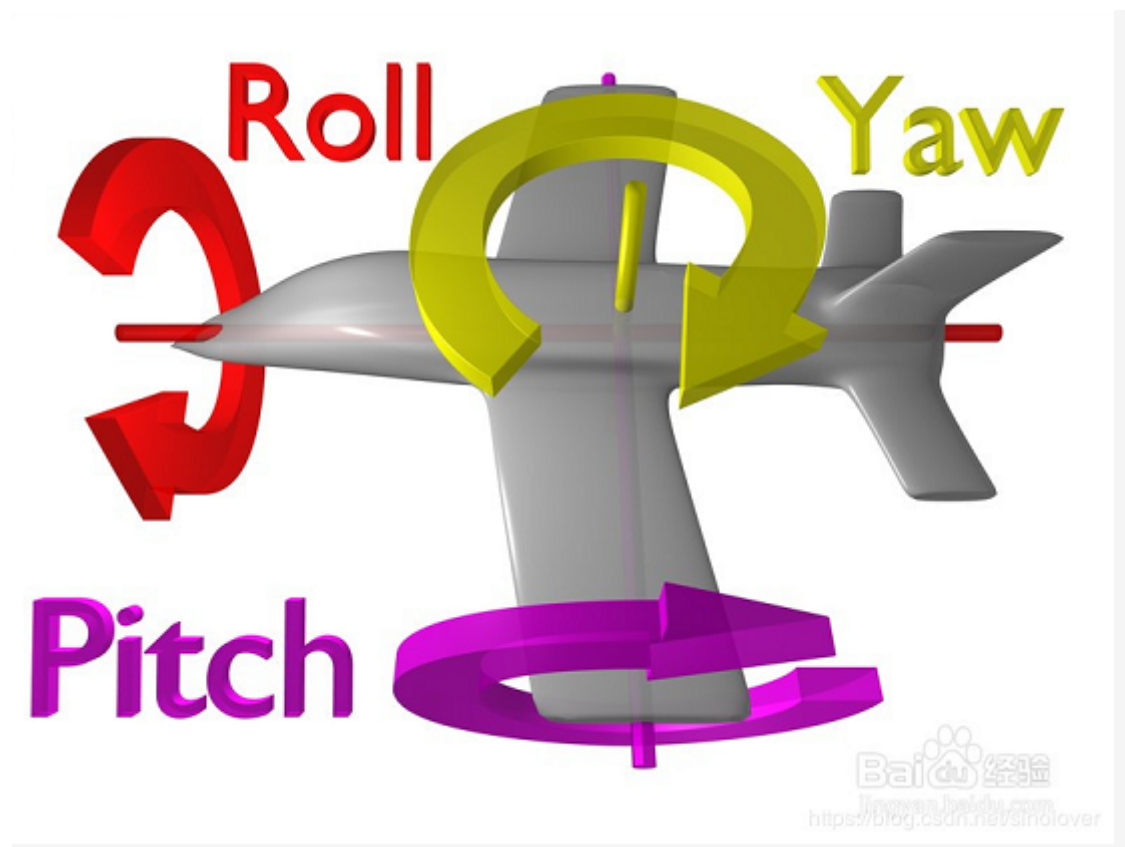
弧度: 单位弧度定义为圆弧长度等于半径时的圆心角。

arc length = radius

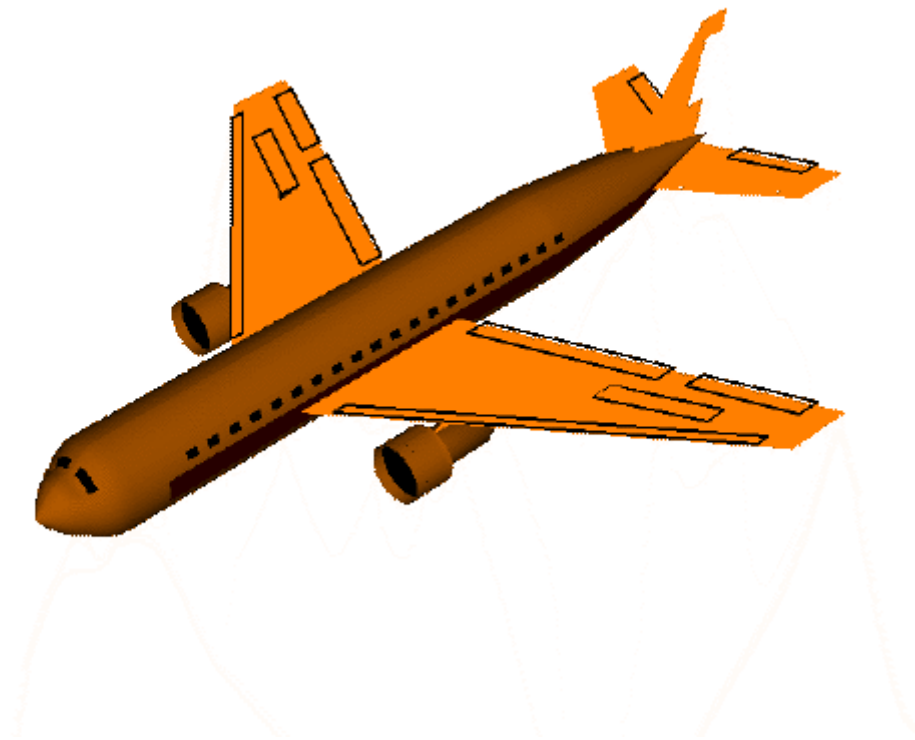


补充资料2:偏航、翻滚与俯仰

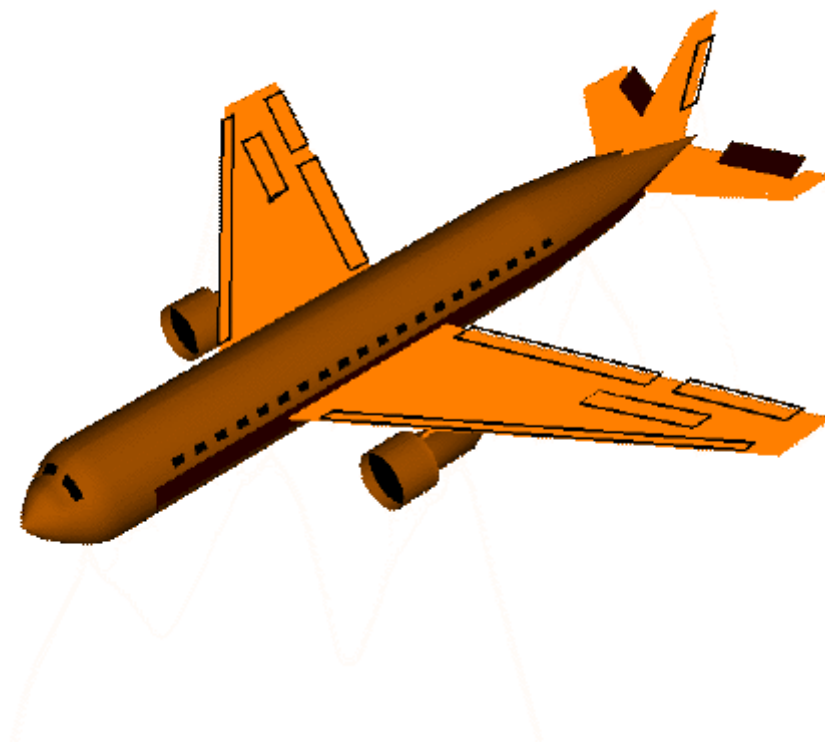
坐标系图解:



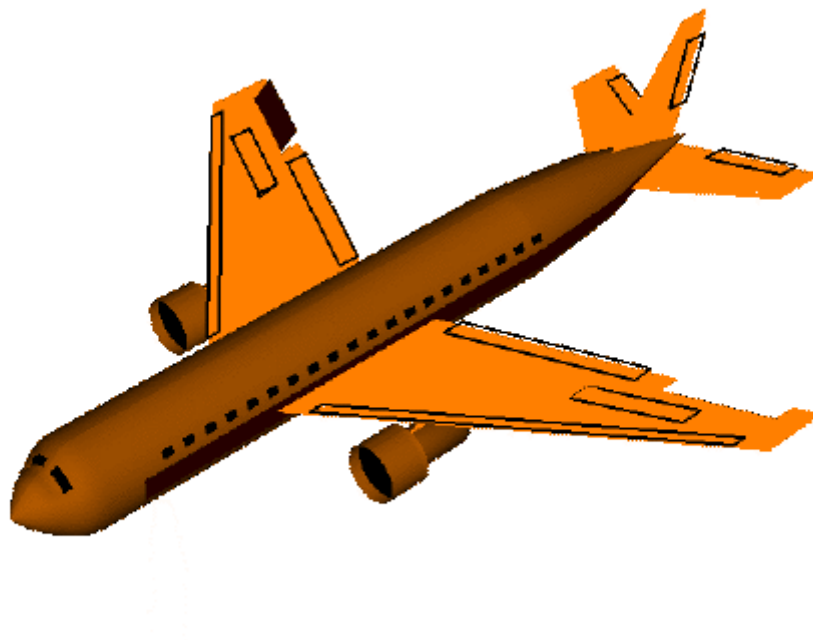
偏航:



俯仰:



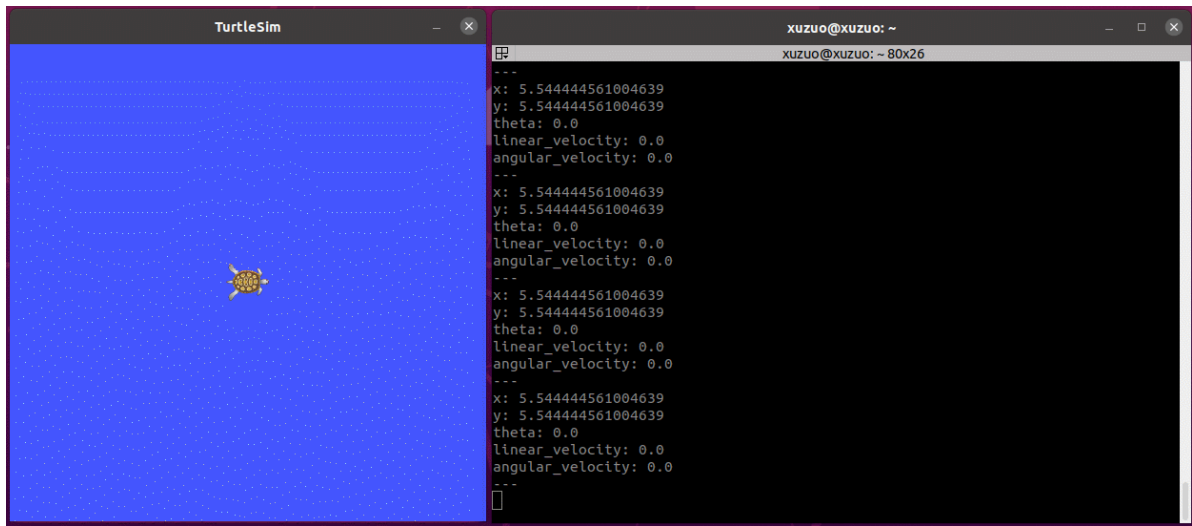
翻滚:



2.5.2 实操02_话题订阅

需求描述: 已知turtlesim中的乌龟显示节点，会发布当前乌龟的位姿(窗体中乌龟的坐标以及朝向)，要求控制乌龟运动，并时时打印当前乌龟的位姿。

结果演示:



实现分析:

1. 首先, 需要启动乌龟显示以及运动控制节点并控制乌龟运动。
2. 要通过ROS命令, 来获取乌龟位姿发布的话题以及消息。
3. 编写订阅节点, 订阅并打印乌龟的位姿。

实现流程:

1. 通过ros命令获取话题与消息信息。
2. 编码实现位姿获取节点。
3. 启动 roscore、turtlesim_node 、控制节点以及位姿订阅节点, 控制乌龟运动并输出乌龟的位姿。

1.话题与消息获取

获取话题:/turtle1/pose

```
rostopic list  
Copy
```

获取消息类型:turtlesim/Pose

```
rostopic type /turtle1/pose  
Copy
```

获取消息格式:

```
rosmg info turtlesim/Pose  
Copy
```

响应结果:

```
float32 x  
float32 y  
float32 theta  
float32 linear_velocity  
float32 angular_velocity  
Copy
```

2.实现订阅节点

创建功能包需要依赖的功能包: roscpp rospy std_msgs turtlesim

实现方案A: C++

```
/*
    订阅小乌龟的位姿：时时获取小乌龟在窗体中的坐标并打印
    准备工作：
        1.获取话题名称 /turtle1/pose
        2.获取消息类型 turtlesim/Pose
        3.运行前启动 turtlesim_node 与 turtle_teleop_key 节点

    实现流程：
        1.包含头文件
        2.初始化 ROS 节点
        3.创建 ROS 句柄
        4.创建订阅者对象
        5.回调函数处理订阅的数据
        6.spin
*/

#include "ros/ros.h"
#include "turtlesim/Pose.h"

void doPose(const turtlesim::Pose::ConstPtr& p){
    ROS_INFO("乌龟位姿信息:x=%.2f,y=%.2f,theta=%.2f,lv=%.2f,av=%.2f",
        p->x,p->y,p->theta,p->linear_velocity,p->angular_velocity
    );
}

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ROS 节点
    ros::init(argc,argv,"sub_pose");
    // 3.创建 ROS 句柄
    ros::NodeHandle nh;
    // 4.创建订阅者对象
    ros::Subscriber sub = nh.subscribe<turtlesim::Pose>
("/turtle1/pose",1000,doPose);
    // 5.回调函数处理订阅的数据
    // 6.spin
    ros::spin();
    return 0;
}
Copy
```

配置文件此处略

实现方案B: Python

```
#!/usr/bin/env python
"""
    订阅小乌龟的位姿：时时获取小乌龟在窗体中的坐标并打印
    准备工作：
        1.获取话题名称 /turtle1/pose
```

2. 获取消息类型 `turtlesim/Pose`
3. 运行前启动 `turtlesim_node` 与 `turtle_teleop_key` 节点

实现流程：

1. 导包
2. 初始化 ROS 节点
3. 创建订阅者对象
4. 回调函数处理订阅的数据
5. `spin`

```
"""

import rospy
from turtlesim.msg import Pose

def doPose(data):
    rospy.loginfo("乌龟坐标:x=%.2f, y=%.2f,theta=%.2f",data.x,data.y,data.theta)

if __name__ == "__main__":

    # 2.初始化 ROS 节点
    rospy.init_node("sub_pose_p")

    # 3.创建订阅者对象
    sub = rospy.Subscriber("/turtle1/pose",Pose,doPose,queue_size=1000)
    # 4.回调函数处理订阅的数据
    # 5.spin
    rospy.spin()

Copy
```

权限设置以及配置文件此处略

3.运行

首先，启动 `roscore`；

然后启动乌龟显示节点，执行运动控制节点；

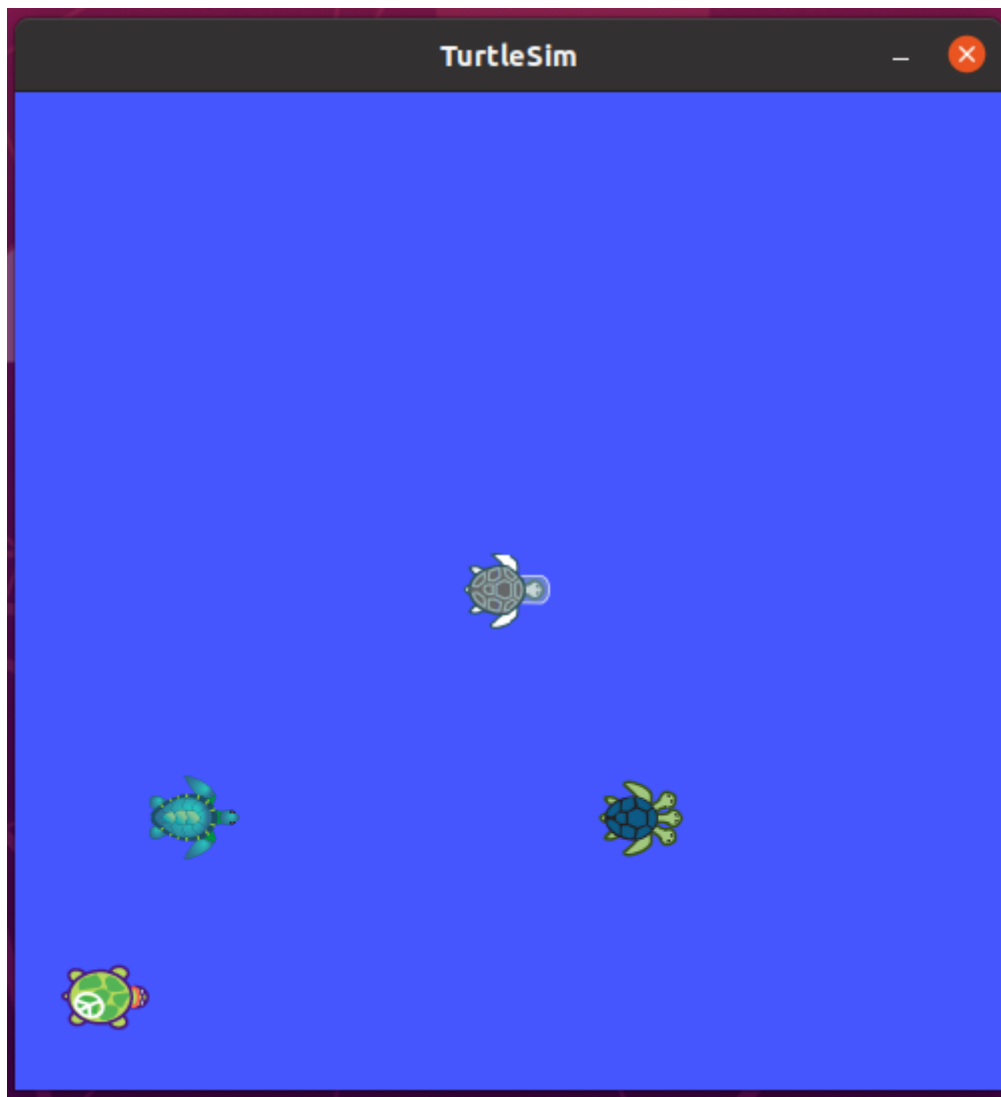
最后启动乌龟位姿订阅节点；

最终执行结果与演示结果类似。

2.5.3 实操03_服务调用

需求描述:编码实现向 `turtlesim` 发送请求，在乌龟显示节点的窗体指定位置生成一乌龟，这是一个服务请求操作。

结果演示:



实现分析:

1. 首先，需要启动乌龟显示节点。
2. 要通过ROS命令，来获取乌龟生成服务的服务名称以及服务消息类型。
3. 编写服务请求节点，生成新的乌龟。

实现流程:

1. 通过ros命令获取服务与服务消息信息。
2. 编码实现服务请求节点。
3. 启动 roscore、turtlesim_node 、乌龟生成节点，生成新的乌龟。

1.服务名称与服务消息获取

获取话题:/spawn

```
rosservice list  
Copy
```

获取消息类型:turtlesim/Spawn

```
rosservice type /spawn  
Copy
```

获取消息格式:

```
rossrv info turtlesim/Spawn
Copy
```

响应结果:

```
float32 x
float32 y
float32 theta
string name
---
string name
Copy
```

2.服务客户端实现

创建功能包需要依赖的功能包: roscpp rospy std_msgs turtlesim

实现方案A:C++

```
/*
    生成一只小乌龟
    准备工作：
        1.服务话题 /spawn
        2.服务消息类型 turtlesim/Spawn
        3.运行前先启动 turtlesim_node 节点

    实现流程：
        1.包含头文件
            需要包含 turtlesim 包下资源，注意在 package.xml 配置
        2.初始化 ros 节点
        3.创建 ros 句柄
        4.创建 service 客户端
        5.等待服务启动
        6.发送请求
        7.处理响应

*/

#include "ros/ros.h"
#include "turtlesim/Spawn.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ros 节点
    ros::init(argc, argv, "set_turtle");
    // 3.创建 ros 句柄
    ros::NodeHandle nh;
    // 4.创建 service 客户端
    ros::ServiceClient client = nh.serviceClient<turtlesim::Spawn>("/spawn");
    // 5.等待服务启动
    // client.waitForExistence();
    ros::service::waitForService("/spawn");
    // 6.发送请求
    turtlesim::Spawn spawn;
```

```

spawn.request.x = 1.0;
spawn.request.y = 1.0;
spawn.request.theta = 1.57;
spawn.request.name = "my_turtle";
bool flag = client.call(spawn);
// 7.处理响应结果
if (flag)
{
    ROS_INFO("新的乌龟生成,名字:%s", spawn.response.name.c_str());
} else {
    ROS_INFO("乌龟生成失败!!!");
}

return 0;
}
Copy

```

配置文件此处略

实现方案B:Python

```

#!/usr/bin/env python
"""
    生成一只小乌龟
    准备工作：
        1.服务话题 /spawn
        2.服务消息类型 turtlesim/Spawn
        3.运行前先启动 turtlesim_node 节点

    实现流程：
        1.导包
            需要包含 turtlesim 包下资源，注意在 package.xml 配置
        2.初始化 ros 节点
        3.创建 service 客户端
        4.等待服务启动
        5.发送请求
        6.处理响应

"""

import rospy
from turtlesim.srv import Spawn,SpawnRequest,SpawnResponse

if __name__ == "__main__":
    # 2.初始化 ros 节点
    rospy.init_node("set_turtle_p")
    # 3.创建 service 客户端
    client = rospy.ServiceProxy("/spawn",Spawn)
    # 4.等待服务启动
    client.wait_for_service()
    # 5.发送请求
    req = SpawnRequest()
    req.x = 2.0
    req.y = 2.0
    req.theta = -1.57
    req.name = "my_turtle_p"

```

```
try:
    response = client.call(req)
    # 6.处理响应
    rospy.loginfo("乌龟创建成功!, 叫:%s",response.name)
except expression as identifier:
    rospy.loginfo("服务调用失败")
```

Copy

权限设置以及配置文件此处略

3.运行

首先, 启动 roscore;

然后启动乌龟显示节点;

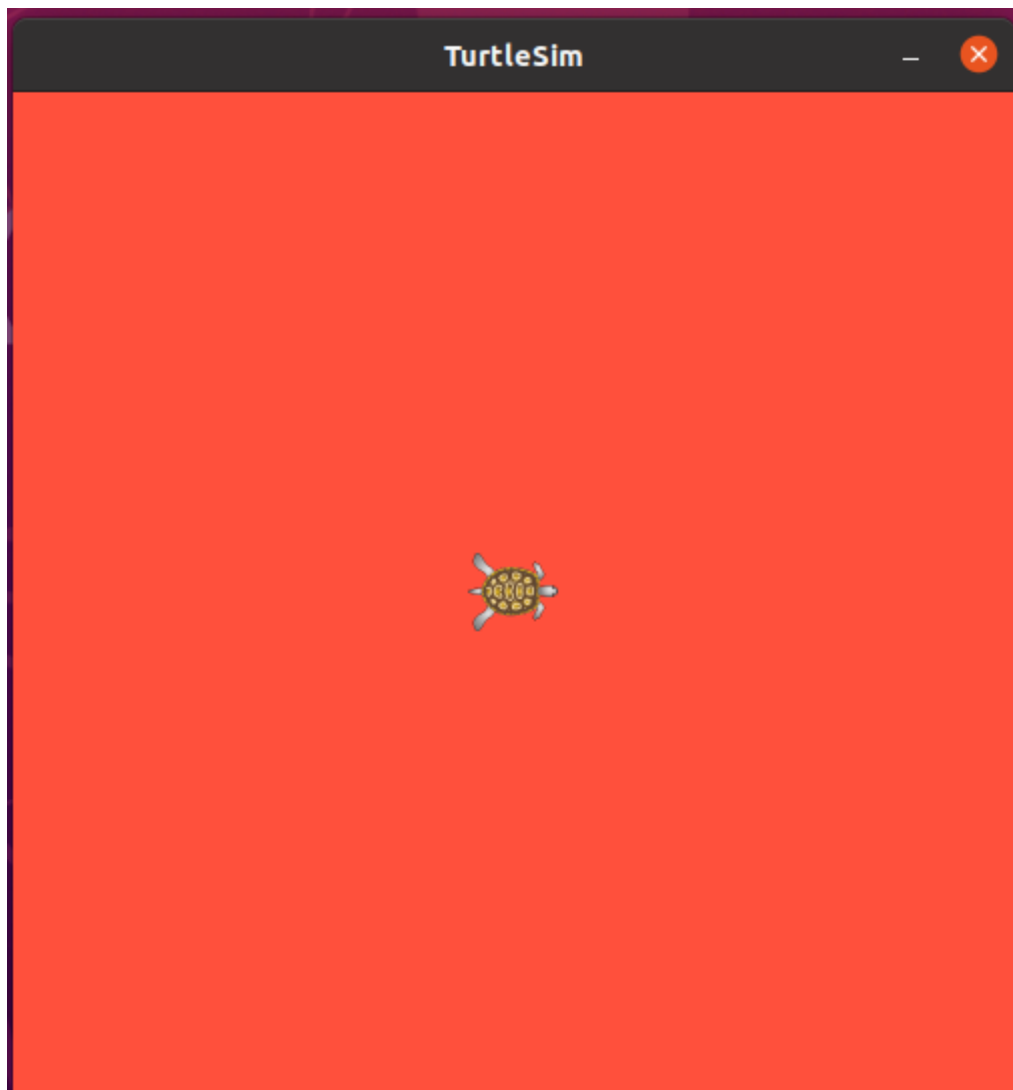
最后启动乌龟生成请求节点;

最终执行结果与演示结果类似。

2.5.4 实操04_参数设置

需求描述: 修改turtlesim乌龟显示节点窗体的背景色, 已知背景色是通过参数服务器的方式以 rgb 方式设置的。

结果演示:



实现分析:

1. 首先, 需要启动乌龟显示节点。
2. 要通过ROS命令, 来获取参数服务器中设置背景色的参数。
3. 编写参数设置节点, 修改参数服务器中的参数值。

实现流程:

1. 通过ros命令获取参数。
2. 编码实现参数设置节点。
3. 启动 roscore、turtlesim_node 与参数设置节点, 查看运行结果。

1.参数名获取

获取参数列表:

```
rosparam list  
Copy
```

响应结果:

```
/turtlesim/background_b  
/turtlesim/background_g  
/turtlesim/background_r  
Copy
```

2.参数修改

实现方案A:C++

```
/*  
    注意命名空间的使用。  
*/  
#include "ros/ros.h"  
  
int main(int argc, char *argv[])  
{  
    ros::init(argc,argv,"haha");  
  
    ros::NodeHandle nh("turtlesim");  
    //ros::NodeHandle nh;  
  
    // ros::param::set("/turtlesim/background_r",0);  
    // ros::param::set("/turtlesim/background_g",0);  
    // ros::param::set("/turtlesim/background_b",0);  
  
    nh.setParam("background_r",0);  
    nh.setParam("background_g",0);  
    nh.setParam("background_b",0);  
  
    return 0;  
}  
Copy
```

配置文件此处略

实现方案B:Python

```
#!/usr/bin/env python

import rospy

if __name__ == "__main__":
    rospy.init_node("hehe")
    # rospy.set_param("/turtlesim/background_r",255)
    # rospy.set_param("/turtlesim/background_g",255)
    # rospy.set_param("/turtlesim/background_b",255)
    rospy.set_param("background_r",255)
    rospy.set_param("background_g",255)
    rospy.set_param("background_b",255) # 调用时，需要传入 __ns:=xxx
Copy
```

权限设置以及配置文件此处略

3.运行

首先，启动 roscore；

然后启动背景色设置节点；

最后启动乌龟显示节点；

最终执行结果与演示结果类似。

PS: 注意节点启动顺序，如果先启动乌龟显示节点，后启动背景色设置节点，那么颜色设置不会生效。

4.其他设置方式

方式1:修改小乌龟节点的背景色(命令行实现)

```
rosparam set /turtlesim/background_b 自定义数值
Copy
rosparam set /turtlesim/background_g 自定义数值
Copy
rosparam set /turtlesim/background_r 自定义数值
Copy
```

修改相关参数后，重启 turtlesim_node 节点，背景色就会发生改变了

方式2:启动节点时，直接设置参数

```
roslaunch turtlesim turtlesim_node _background_r:=100 _background_g=0
_background_b=0
Copy
```

方式3:通过launch文件传参

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node" name="set_bg" output="screen">
    <!-- launch 传参策略1 -->
    <!-- <param name="background_b" value="0" type="int" />
    <param name="background_g" value="0" type="int" />
    <param name="background_r" value="0" type="int" /> -->

    <!-- launch 传参策略2 -->
    <rosparam command="load" file="$(find
demo03_test_parameter)/cfg/color.yaml" />
  </node>

</
```

2.6 通信机制比较

三种通信机制中，参数服务器是一种数据共享机制，可以在不同的节点之间共享数据，话题通信与服务通信是在不同的节点之间传递数据的，三者是ROS中最基础也是应用最为广泛的通信机制。

这其中，话题通信和服务通信有一定的相似性也有本质上的差异，在此将二者做一下简单比较：

二者的实现流程是比较相似的，都是涉及到四个要素：

- 要素1: 消息的发布方/客户端(Publisher/Client)
- 要素2: 消息的订阅方/服务端(Subscriber/Server)
- 要素3: 话题名称(Topic/Service)
- 要素4: 数据载体(msg/srv)

可以概括为: 两个节点通过话题关联到一起，并使用某种类型的数据载体实现数据传输。

二者的实现也是有本质差异的，具体比较如下：

	Topic(话题)	Service(服务)
通信模式	发布/订阅	请求/响应
同步性	异步	同步
底层协议	ROSTCP/ROSUDP	ROSTCP/ROSUDP
缓冲区	有	无
时时性	弱	强
节点关系	多对多	一对多(一个 Server)
通信数据	msg	srv
使用场景	连续高频的数据发布与接收:雷达、里程计	偶尔调用或执行某一项特定功能：拍照、语音识别

不同通信机制有一定的互补性，都有各自适应的应用场景。尤其是话题与服务通信，需要结合具体的应用场景与二者的差异，选择合适的通信机制。

2.7 本章小结

本章主要介绍了ROS中最基本的也是最核心的通信机制实现: 话题通信、服务通信、参数服务器。每种通信机制，都介绍了如下内容:

- 伊始介绍了当前通信机制的应用场景;
- 介绍了当前通信机制的理论模型;
- 分别介绍了当前通信机制的C++与Python实现。

除此之外，还介绍了:

- ROS中的常用命令方便操作、调节点以及通信信息;
- 通过实操又将上述知识点加以整合;
- 最后又着重比较了话题通信与服务通信的相同点以及差异。

掌握本章内容后，基本上就可以从容应对ROS中大部分应用场景了。

第 3 章 ROS通信机制进阶

上一章内容，主要介绍了ROS通信的实现，内容偏向于粗粒度的通信框架的讲解，没有详细介绍涉及的API，也没有封装代码，鉴于此，本章主要内容如下:

- ROS常用API介绍;
- ROS中自定义头文件与源文件的使用。

预期达成的学习目标:

- 熟练掌握ROS常用API;
- 掌握ROS中自定义头文件与源文件的配置。

[Autolabor主站Introduction课程视频第 1 章 ROS概述与环境搭建1.1 ROS简介1.1.1ROS概念1.1.2ROS设计目标1.1.3ROS发展历程1.2 ROS安装1.2.1 安装虚拟机软件1.2.2 虚拟一台主机1.2.3 安装ubuntu1.2.4 安装 ROS1.2.5 测试 ROS1.2.6 资料:其他ROS版本安装1.3 ROS快速体验1.3.1 HelloWorld实现简介1.3.2 HelloWorld实现A\(C++\)1.3.3 HelloWorld实现B\(Python\)1.4 ROS集成开发环境搭建1.4.1 安装终端1.4.2 安装VScode1.4.3 launch文件演示1.5 ROS架构1.5.1 ROS文件系统1.5.2 ROS文件系统相关命令1.5.3 ROS计算图1.6 本章小结第 2 章 ROS通信机制2.1 话题通信2.1.1 话题通信理论模型2.1.2 话题通信基本操作A\(C++\)2.1.3 话题通信基本操作B\(Python\)2.1.4 话题通信自定义msg2.1.5 话题通信自定义msg调用A\(C++\)2.1.6 话题通信自定义msg调用B\(Python\)2.2 服务通信2.2.1 服务通信理论模型2.2.2 服务通信自定义srv2.2.3 服务通信自定义srv调用A\(C++\)2.2.4 服务通信自定义srv调用B\(Python\)2.3 参数服务器2.3.1 参数服务器理论模型2.3.2 参数操作A\(C++\)2.3.3 参数操作B\(Python\)2.4 常用命令2.4.1 rosnodet2.4.2 rostopic2.4.3 rosmsg2.4.4 rosservice2.4.5 rossrv2.4.6 rosparm2.5 通信机制实操2.5.1 实操01 话题发布2.5.2 实操02 话题订阅2.5.3 实操03 服务调用2.5.4 实操04 参数设置2.6 通信机制比较2.7 本章小结第 3 章 ROS通信机制进阶3.1 常用API3.1.1 初始化3.1.2 话题与服务相关对象3.1.3 回旋函数3.1.4 时间3.1.5 其他函数3.2 ROS中的头文件与源文件3.3 Python模块导入3.4 本章小结第 4 章 ROS运行管理第 5 章 ROS常用组件第 6 章 机器人系统仿真第 7 章 机器人导航\(仿真\)7.1 概述7.2 导航实现7.2.1 导航实现01 SLAM建图7.2.2 导航实现02 地图服务7.2.3 导航实现03 定位7.2.4 导航实现04 路径规划7.2.5 导航与SLAM建图7.3 导航相关消息7.3.1 导航之地图7.3.2 导航之里程计7.3.3 导航之坐标变换7.3.4 导航之定位7.3.5 导航之目标点与路径规划7.3.6 导航之激光雷达7.3.7 导航之相机7.3.8 深度图像转激光数据7.4 本章小结待发布...本书使用 GitBook 发布](#)

3.1 常用API

首先，建议参考官方API文档或参考源码：

- ROS节点的初始化相关API;
- NodeHandle 的基本使用相关API;
- 话题的发布方，订阅方对象相关API;
- 服务的服务端，客户端对象相关API;
- 时间相关API;
- 日志输出相关API。

参数服务器相关API在第二章已经有详细介绍和应用，在此不再赘述。

3.1.1 初始化

C++

初始化

```
/** @brief ROS初始化函数。
 *
 * 该函数可以解析并使用节点启动时传入的参数(通过参数设置节点名称、命名空间...)
 *
 * 该函数有多个重载版本，如果使用NodeHandle建议调用该版本。
 *
 * \param argc 参数个数
 * \param argv 参数列表
 * \param name 节点名称，需要保证其唯一性，不允许包含命名空间
 * \param options 节点启动选项，被封装进了ros::init_options
 *
 */
void init(int &argc, char **argv, const std::string& name, uint32_t options =
0);
Copy
```

Python

初始化

```
def init_node(name, argv=None, anonymous=False, log_level=None,
disable_rostime=False, disable_rosout=False, disable_signals=False,
xmlrpc_port=0, tcpport=0):
    """
    在ROS master中注册节点

    @param name: 节点名称，必须保证节点名称唯一，节点名称中不能使用命名空间(不能包含 '/')
    @type name: str

    @param anonymous: 取值为 true 时，为节点名称后缀随机编号
    @type anonymous: bool
    """
```

3.1.2 话题与服务相关对象

C++

在 roscpp 中，话题和服务的相关对象一般由 NodeHandle 创建。

NodeHandle有一个重要作用是可以用于设置命名空间，这是后期的重点，但是本章暂不介绍。

1.发布对象

对象获取:

```
/**
 * \brief 根据话题生成发布对象
 *
 * 在 ROS master 注册并返回一个发布者对象，该对象可以发布消息
 *
 * 使用示例如下：
 *
 *   ros::Publisher pub = handle.advertise<std_msgs::Empty>("my_topic", 1);
 *
 * \param topic 发布消息使用的话题
 *
 * \param queue_size 等待发送给订阅者的最大消息数量
 *
 * \param latch (optional) 如果为 true,该话题发布的最后一条消息将被保存，并且后期当有订阅者
连接时会将该消息发送给订阅者
 *
 * \return 调用成功时，会返回一个发布对象
 *
 */
template <class M>
Publisher advertise(const std::string& topic, uint32_t queue_size, bool latch =
false)
Copy
```

消息发布函数:

```
/**
 * 发布消息
 */
template <typename M>
void publish(const M& message) const
Copy
```

2.订阅对象

对象获取:

```
/**
 * \brief 生成某个话题的订阅对象
 *
 * 该函数将根据给定的话题在ROS master 注册，并自动连接相同主题的发布方，每接收到一条消息，都
会调用回调
 * 函数，并且传入该消息的共享指针，该消息不能被修改，因为可能其他订阅对象也会使用该消息。

```

```

*
* 使用示例如下：

void callback(const std_msgs::Empty::ConstPtr& message)
{
}

ros::Subscriber sub = handle.subscribe("my_topic", 1, callback);

*
* \param M [template] M 是指消息类型
* \param topic 订阅的话题
* \param queue_size 消息队列长度，超出长度时，头部的消息将被弃用
* \param fp 当订阅到一条消息时，需要执行的回调函数
* \return 调用成功时，返回一个订阅者对象，失败时，返回空对象
*

void callback(const std_msgs::Empty::ConstPtr& message){...}
ros::NodeHandle nodeHandle;
ros::Subscriber sub = nodeHandle.subscribe("my_topic", 1, callback);
if (sub) // Enter if subscriber is valid
{
...
}

*/
template<class M>
Subscriber subscribe(const std::string& topic, uint32_t queue_size, void(*fp)
(const boost::shared_ptr<M const>&), const TransportHints& transport_hints =
TransportHints())
Copy

```

3.服务对象

对象获取:

```

/**
* \brief 生成服务端对象
*
* 该函数可以连接到 ROS master，并提供一个具有给定名称的服务对象。
*
* 使用示例如下：
\verbatim
bool callback(std_srvs::Empty& request, std_srvs::Empty& response)
{
return true;
}

ros::ServiceServer service = handle.advertiseService("my_service", callback);
\endverbatim
*
* \param service 服务的主题名称
* \param srv_func 接收到请求时，需要处理请求的回调函数
* \return 请求成功时返回服务对象，否则返回空对象：
\verbatim
bool Foo::callback(std_srvs::Empty& request, std_srvs::Empty& response)
{

```

```

return true;
}
ros::NodeHandle nodeHandle;
Foo foo_object;
ros::ServiceServer service = nodeHandle.advertiseService("my_service",
callback);
if (service) // Enter if advertised service is valid
{
...
}
\endverbatim

*/
template<class MReq, class MRes>
ServiceServer advertiseService(const std::string& service, bool(*srv_func)
(MReq&, MRes&))
Copy

```

4.客户端对象

对象获取:

```

/**
 * @brief 创建一个服务客户端对象
 *
 * 当清除最后一个连接的引用句柄时，连接将被关闭。
 *
 * @param service_name 服务主题名称
 */
template<class Service>
ServiceClient serviceClient(const std::string& service_name, bool persistent =
false,
                                const M_string& header_values = M_string())
Copy

```

请求发送函数:

```

/**
 * @brief 发送请求
 * 返回值为 bool 类型，true，请求处理成功，false，处理失败。
 */
template<class Service>
bool call(Service& service)
Copy

```

等待服务函数1:

```

/**
 * ros::service::waitForService("addInts");
 * \brief 等待服务可用，否则一直处于阻塞状态
 * \param service_name 被"等待"的服务的话题名称
 * \param timeout 等待最大时常，默认为 -1，可以永久等待直至节点关闭
 * \return 成功返回 true，否则返回 false。
 */
ROSCPP_DECL bool waitForService(const std::string& service_name, ros::Duration
timeout = ros::Duration(-1));
Copy

```

等待服务函数2:

```

/**
 * client.waitForExistence();
 * \brief 等待服务可用，否则一直处于阻塞状态
 * \param timeout 等待最大时常，默认为 -1，可以永久等待直至节点关闭
 * \return 成功返回 true，否则返回 false。
 */
bool waitForExistence(ros::Duration timeout = ros::Duration(-1));
Copy

```

Python

1.发布对象

对象获取:

```

class Publisher(Topic):
    """
    在ROS master注册为相关话题的发布方
    """

    def __init__(self, name, data_class, subscriber_listener=None,
tcp_nodeLAY=False, latch=False, headers=None, queue_size=None):
    """
    Constructor
    @param name: 话题名称
    @type name: str
    @param data_class: 消息类型

    @param latch: 如果为 true,该话题发布的最后一条消息将被保存，并且后期当有订阅者连接
时会该消息发送给订阅者
    @type latch: bool

    @param queue_size: 等待发送给订阅者的最大消息数量
    @type queue_size: int

    """
Copy

```

消息发布函数:

```
def publish(self, *args, **kwargs):
    """
    发布消息
    """
```

Copy

2.订阅对象

对象获取:

```
class Subscriber(Topic):
    """
    类注册为指定主题的订阅者，其中消息是给定类型的。
    """

    def __init__(self, name, data_class, callback=None, callback_args=None,
                  queue_size=None, buff_size=DEFAULT_BUFF_SIZE,
                  tcp_nodelay=False):
        """
        Constructor.

        @param name: 话题名称
        @type name: str
        @param data_class: 消息类型
        @type data_class: L{Message} class
        @param callback: 处理订阅到的消息的回调函数
        @type callback: fn(msg, cb_args)

        @param queue_size: 消息队列长度，超出长度时，头部的消息将被弃用

        """
```

Copy

3.服务对象

对象获取:

```
class Service(ServiceImpl):
    """
    声明一个ROS服务

    使用示例::
        s = Service('getmapservice', GetMap, get_map_handler)
    """

    def __init__(self, name, service_class, handler,
                  buff_size=DEFAULT_BUFF_SIZE, error_handler=None):
        """

        @param name: 服务主题名称 ``str``
        @param service_class: 服务消息类型

        @param handler: 回调函数，处理请求数据，并返回响应数据

        @type handler: fn(req)->resp
```

```
Copy
```

4.客户端对象

对象获取:

```
class ServiceProxy(_Service):
    """
    创建一个ROS服务的句柄

    示例用法::
        add_two_ints = ServiceProxy('add_two_ints', AddTwoInts)
        resp = add_two_ints(1, 2)
    """

    def __init__(self, name, service_class, persistent=False, headers=None):
        """
        ctor.
        @param name: 服务主题名称
        @type name: str
        @param service_class: 服务消息类型
        @type service_class: Service class
        """
```

```
Copy
```

请求发送函数:

```
def call(self, *args, **kwargs):
    """
    发送请求，返回值为响应数据

    """
```

```
Copy
```

等待服务函数:

```
def wait_for_service(service, timeout=None):
    """
    调用该函数时，程序会处于阻塞状态直到服务可用
    @param service: 被等待的服务话题名称
    @type service: str
    @param timeout: 超时时间
    @type timeout: double|rospy.Duration
    """
```

```
Copy
```

3.1.3 回旋函数

C++

在ROS程序中，频繁的使用了 `ros::spin()` 和 `ros::spinOnce()` 两个回旋函数，可以用于处理回调函数。

1.spinOnce()

```
/**
 * \brief 处理一轮回调
 *
 * 一般应用场景：
 *     在循环体内，处理所有可用的回调函数
 *
 */
ROSCPP_DECL void spinOnce();
Copy
```

2.spin()

```
/**
 * \brief 进入循环处理回调
 *
 */
ROSCPP_DECL void spin();
Copy
```

3.二者比较

相同点:二者都用于处理回调函数；

不同点:`ros::spin()` 是进入了循环执行回调函数，而 `ros::spinOnce()` 只会执行一次回调函数(没有循环)，在 `ros::spin()` 后的语句不会执行到，而 `ros::spinOnce()` 后的语句可以执行。

Python

```
def spin():
    """
    进入循环处理回调
    """
```

3.1.4 时间

ROS中时间相关的API是极其常用，比如:获取当前时刻、持续时间的设置、执行频率、休眠、定时器...都与时间相关。

C++

1.时刻

获取时刻，或是设置指定时刻:

```

ros::init(argc,argv,"hello_time");
ros::NodeHandle nh;//必须创建句柄,否则时间没有初始化,导致后续API调用失败
ros::Time right_now = ros::Time::now();//将当前时刻封装成对象
ROS_INFO("当前时刻:%.2f",right_now.toSec());//获取距离 1970年01月01日 00:00:00 的秒数
ROS_INFO("当前时刻:%d",right_now.sec);//获取距离 1970年01月01日 00:00:00 的秒数

ros::Time someTime(100,100000000);// 参数1:秒数 参数2:纳秒
ROS_INFO("时刻:%.2f",someTime.toSec()); //100.10
ros::Time someTime2(100.3);//直接传入 double 类型的秒数
ROS_INFO("时刻:%.2f",someTime2.toSec()); //100.30
Copy

```

2.持续时间

设置一个时间区间(间隔):

```

ROS_INFO("当前时刻:%.2f",ros::Time::now().toSec());
ros::Duration du(10);//持续10秒钟,参数是double类型的,以秒为单位
du.sleep();//按照指定的持续时间休眠
ROS_INFO("持续时间:%.2f",du.toSec());//将持续时间换算成秒
ROS_INFO("当前时刻:%.2f",ros::Time::now().toSec());
Copy

```

3.持续时间与时刻运算

为了方便使用,ROS中提供了时间与时刻的运算:

```

ROS_INFO("时间运算");
ros::Time now = ros::Time::now();
ros::Duration du1(10);
ros::Duration du2(20);
ROS_INFO("当前时刻:%.2f",now.toSec());
//1.time 与 duration 运算
ros::Time after_now = now + du1;
ros::Time before_now = now - du1;
ROS_INFO("当前时刻之后:%.2f",after_now.toSec());
ROS_INFO("当前时刻之前:%.2f",before_now.toSec());

//2.duration 之间相互运算
ros::Duration du3 = du1 + du2;
ros::Duration du4 = du1 - du2;
ROS_INFO("du3 = %.2f",du3.toSec());
ROS_INFO("du4 = %.2f",du4.toSec());
//PS: time 与 time 不可以运算
// ros::Time nn = now + before_now;//异常
Copy

```

4.设置运行频率

```

ros::Rate rate(1); //指定频率
while (true)
{
    ROS_INFO("-----code-----");
    rate.sleep(); //休眠, 休眠时间 = 1 / 频率。
}
Copy

```

5.定时器

ROS 中内置了专门的定时器，可以实现与 `ros::Rate` 类似的效果：

```

ros::NodeHandle nh; //必须创建句柄，否则时间没有初始化，导致后续API调用失败

// ROS 定时器
/**
 * \brief 创建一个定时器，按照指定频率调用回调函数。
 *
 * \param period 时间间隔
 * \param callback 回调函数
 * \param oneshot 如果设置为 true,只执行一次回调函数，设置为 false,就循环执行。
 * \param autostart 如果为true，返回已经启动的定时器,设置为 false，需要手动启动。
 */
//Timer createTimer(Duration period, const TimerCallback& callback, bool
oneshot = false,
//                          bool autostart = true) const;

// ros::Timer timer = nh.createTimer(ros::Duration(0.5),doSomething);
ros::Timer timer = nh.createTimer(ros::Duration(0.5),doSomething,true); //只执行一
次

// ros::Timer timer =
nh.createTimer(ros::Duration(0.5),doSomething,false,false); //需要手动启动
// timer.start();
ros::spin(); //必须 spin
Copy

```

定时器的回调函数：

```

void doSomething(const ros::TimerEvent &event){
    ROS_INFO("-----");
    ROS_INFO("event:%s",std::to_string(event.current_real.toSec()).c_str());
}
Copy

```

Python

1.时刻

获取时刻，或是设置指定时刻：

```

# 获取当前时刻
right_now = rospy.Time.now()
rospy.loginfo("当前时刻:%.2f",right_now.to_sec())

```

```

rospy.loginfo("当前时刻:%.2f",right_now.to_nsec())
# 自定义时刻
some_time1 = rospy.Time(1234.567891011)
some_time2 = rospy.Time(1234,567891011)
rospy.loginfo("设置时刻1:%.2f",some_time1.to_sec())
rospy.loginfo("设置时刻2:%.2f",some_time2.to_sec())

# 从时间创建对象
# some_time3 = rospy.Time.from_seconds(543.21)
some_time3 = rospy.Time.from_sec(543.21) # from_sec 替换了 from_seconds
rospy.loginfo("设置时刻3:%.2f",some_time3.to_sec())
Copy

```

2.持续时间

设置一个时间区间(间隔):

```

# 持续时间相关API
rospy.loginfo("持续时间测试开始.....")
du = rospy.Duration(3.3)
rospy.loginfo("du1 持续时间:%.2f",du.to_sec())
rospy.sleep(du) #休眠函数
rospy.loginfo("持续时间测试结束.....")
Copy

```

3.持续时间与时刻运算

为了方便使用，ROS中提供了时间与时刻的运算:

```

rospy.loginfo("时间运算")
now = rospy.Time.now()
du1 = rospy.Duration(10)
du2 = rospy.Duration(20)
rospy.loginfo("当前时刻:%.2f",now.to_sec())
before_now = now - du1
after_now = now + du1
dd = du1 + du2
# now = now + now #非法
rospy.loginfo("之前时刻:%.2f",before_now.to_sec())
rospy.loginfo("之后时刻:%.2f",after_now.to_sec())
rospy.loginfo("持续时间相加:%.2f",dd.to_sec())
Copy

```

4.设置运行频率

```

# 设置执行频率
rate = rospy.Rate(0.5)
while not rospy.is_shutdown():
    rate.sleep() #休眠
    rospy.loginfo("+++++")
Copy

```

5. 定时器

ROS 中内置了专门的定时器，可以实现与 `ros::Rate` 类似的效果：

```
#定时器设置
"""
def __init__(self, period, callback, oneshot=False, reset=False):
    Constructor.
    @param period: 回调函数的时间间隔
    @type period: rospy.Duration
    @param callback: 回调函数
    @type callback: function taking rospy.TimerEvent
    @param oneshot: 设置为True, 就只执行一次, 否则循环执行
    @type oneshot: bool
    @param reset: if True, timer is reset when rostime moved backward. [default:
False]
    @type reset: bool
"""
rospy.Timer(rospy.Duration(1), doMsg)
# rospy.Timer(rospy.Duration(1), doMsg, True) # 只执行一次
rospy.spin()
Copy
```

回调函数：

```
def doMsg(event):
    rospy.loginfo("+++++")
    rospy.loginfo("当前时刻:%s", str(event.current_real))
```

3.1.5 其他函数

在发布实现时，一般会循环发布消息，循环的判断条件一般由节点状态来控制，C++中可以通过 `ros::ok()` 来判断节点状态是否正常，而 python 中则通过 `rospy.is_shutdown()` 来实现判断，导致节点退出的原因主要有如下几种：

- 节点接收到了关闭信息，比如常用的 `ctrl + c` 快捷键就是关闭节点的信号；
- 同名节点启动，导致现有节点退出；
- 程序中的其他部分调用了节点关闭相关的API(C++中是 `ros::shutdown()`，python中是 `rospy.signal_shutdown()`)

另外，日志相关的函数也是极其常用的，在ROS中日志被划分成如下级别：

- **DEBUG(调试)**: 只在调试时使用，此类消息不会输出到控制台；
 - **INFO(信息)**: 标准消息，一般用于说明系统内正在执行的操作；
 - **WARN(警告)**: 提醒一些异常情况，但程序仍然可以执行；
 - **ERROR(错误)**: 提示错误信息，此类错误会影响程序运行；
 - **FATAL(严重错误)**: 此类错误将阻止节点继续运行。
-

C++

1.节点状态判断

```
/** \brief 检查节点是否已经退出
 *
 * ros::shutdown() 被调用且执行完毕后，该函数将会返回 false
 *
 * \return true 如果节点还健在，false 如果节点已经火化了。
 */
bool ok();
Copy
```

2.节点关闭函数

```
/**
 * 关闭节点
 */
void shutdown();
Copy
```

3.日志函数

使用示例

```
ROS_DEBUG("hello,DEBUG"); //不会输出
ROS_INFO("hello,INFO"); //默认白色字体
ROS_WARN("hello,WARN"); //默认黄色字体
ROS_ERROR("hello,ERROR"); //默认红色字体
ROS_FATAL("hello,FATAL"); //默认红色字体
Copy
```

Python

1.节点状态判断

```
def is_shutdown():
    """
    @return: True 如果节点已经被关闭
    @rtype: bool
    """
Copy
```

2.节点关闭函数

```
def signal_shutdown(reason):
    """
    关闭节点
    @param reason: 节点关闭的原因，是一个字符串
    @type reason: str
    """
Copy
def on_shutdown(h):
    """
```

节点被关闭时调用的函数

@param h: 关闭时调用的回调函数，此函数无参

@type h: fn()

"""

Copy

3.日志函数

使用示例

```
rospy.logdebug("hello,debug") #不会输出
rospy.loginfo("hello,info") #默认白色字体
rospy.logwarn("hello,warn") #默认黄色字体
rospy.logerr("hello,error") #默认红色字体
rospy.logfatal("hello,fatal") #默认红色字体
```

3.2 ROS中的头文件与源文件

本节主要介绍ROS的C++实现中，如何使用头文件与源文件的方式封装代码，具体内容如下：

1. 设置头文件，可执行文件作为源文件；
2. 分别设置头文件，源文件与可执行文件。

在ROS中关于头文件的使用，核心内容在于CMakeLists.txt文件的配置，不同的封装方式，配置上也有差异。

3.2.1 自定义头文件调用

需求:设计头文件，可执行文件本身作为源文件。

流程:

1. 编写头文件；
2. 编写可执行文件(同时也是源文件)；
3. 编辑配置文件并执行。

1.头文件

在功能包下的 include/功能包名 目录下新建头文件: hello.h，示例内容如下：

```
#ifndef _HELLO_H
#define _HELLO_H

namespace hello_ns{

class HelloPub {

public:
    void run();
};

}
```

```
#endif
Copy
```

注意:

在 VScode 中, 为了后续包含头文件时不抛出异常, 请配置 .vscode 下 c_cpp_properties.json 的 includepath 属性

```
"/home/用户/工作空间/src/功能包/include/**"
Copy
```

2.可执行文件

在 src 目录下新建文件:hello.cpp, 示例内容如下:

```
#include "ros/ros.h"
#include "test_head/hello.h"

namespace hello_ns {

void HelloPub::run(){
    ROS_INFO("自定义头文件的使用....");
}

}

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    ros::init(argc, argv, "test_head_node");
    hello_ns::HelloPub helloPub;
    helloPub.run();
    return 0;
}
Copy
```

3.配置文件

配置CMakeLists.txt文件, 头文件相关配置如下:

```
include_directories(
include
    ${catkin_INCLUDE_DIRS}
)
Copy
```

可执行配置文件配置方式与之前一致:

```

add_executable(hello src/hello.cpp)

add_dependencies(hello ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

target_link_libraries(hello
  ${catkin_LIBRARIES}
)
Copy

```

最后，编译并执行，控制台可以输出自定义的文本信息。

3.2.2 自定义源文件调用

需求:设计头文件与源文件，在可执行文件中包含头文件。

流程:

1. 编写头文件;
2. 编写源文件;
3. 编写可执行文件;
4. 编辑配置文件并执行。

1.头文件

头文件设置于 3.2.1 类似，在功能包下的 include/功能包名 目录下新建头文件: haha.h，示例内容如下:

```

#ifndef _HAHA_H
#define _HAHA_H

namespace hello_ns {

class My {

public:
    void run();

};

}

#endif
Copy

```

注意:

在 VScode 中，为了后续包含头文件时不抛出异常，请配置 .vscode 下 c_cpp_properties.json 的 includepath属性

```

"/home/用户/工作空间/src/功能包/include/**"
Copy

```

2.源文件

在 src 目录下新建文件:haha.cpp，示例内容如下:

```
#include "test_head_src/haha.h"
#include "ros/ros.h"

namespace hello_ns{

void My::run(){
    ROS_INFO("hello,head and src ...");
}

}

Copy
```

3.可执行文件

在 src 目录下新建文件: use_head.cpp，示例内容如下:

```
#include "ros/ros.h"
#include "test_head_src/haha.h"

int main(int argc, char *argv[])
{
    ros::init(argc,argv,"hahah");
    hello_ns::My my;
    my.run();
    return 0;
}

Copy
```

4.配置文件

头文件与源文件相关配置:

```
include_directories(
include
    ${catkin_INCLUDE_DIRS}
)

## 声明C++库
add_library(head
    include/test_head_src/haha.h
    src/haha.cpp
)

add_dependencies(head ${${PROJECT_NAME}_EXPORTED_TARGETS}
    ${catkin_EXPORTED_TARGETS})

target_link_libraries(head
    ${catkin_LIBRARIES}
)

Copy
```

可执行文件配置:

```

add_executable(use_head src/use_head.cpp)

add_dependencies(use_head ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})

#此处需要添加之前设置的 head 库
target_link_libraries(use_head
    head
    ${catkin_LIBRARIES}
)

```

3.3 Python模块导入

与C++类似的，在Python中导入其他模块时，也需要相关处理。

需求:首先新建一个Python文件A，再创建Python文件UseA，在UseA中导入A并调用A的实现。

实现:

1. 新建两个Python文件，使用 import 实现导入关系；
2. 添加可执行权限、编辑配置文件并执行UseA。

1.新建两个Python文件并使用import导入

文件A实现(包含一个变量):

```

#!/usr/bin/env python
num = 1000
Copy

```

文件B核心实现:

```

import os
import sys

path = os.path.abspath(".")
# 核心
sys.path.insert(0,path + "/src/plumbing_pub_sub/scripts")

import tools

....
....
    rospy.loginfo("num = %d",tools.num)
Copy

```

2.添加可执行权限，编辑配置文件并执行

此过程略。

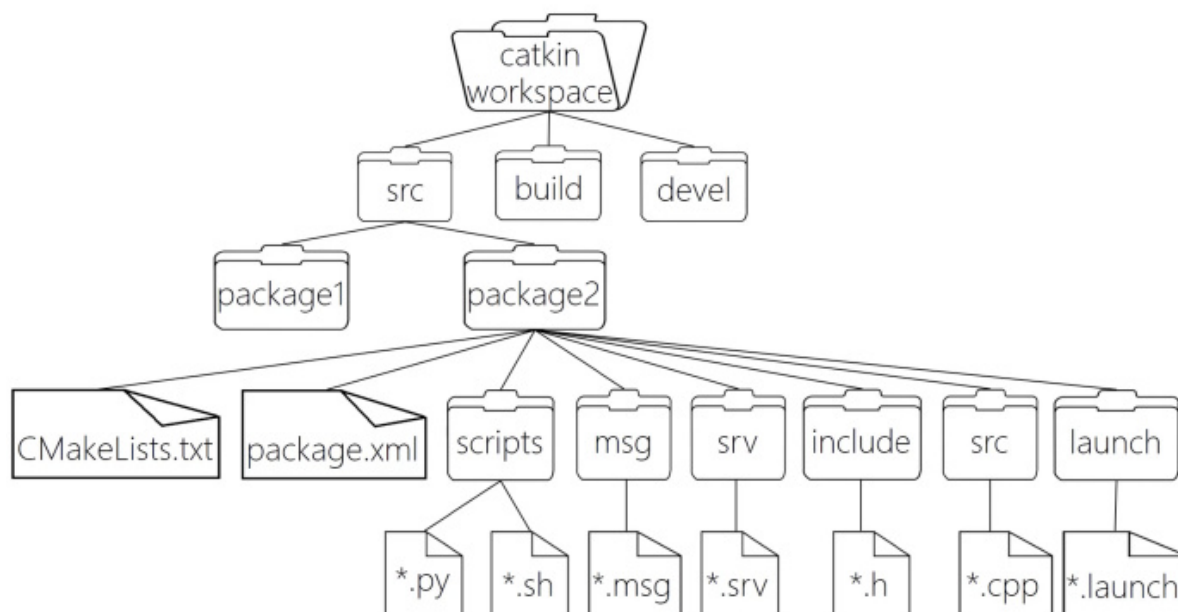
3.4 本章小结

本章内容相对比较简单，多加练习即可。

第 4 章 ROS运行管理

ROS是多进程(节点)的分布式框架，一个完整的ROS系统实现：

可能包含多台主机；
每台主机上又有多个工作空间(workspace)；
每个的工作空间中又包含多个功能包(package)；
每个功能包又包含多个节点(Node)，不同的节点都有自己的节点名称；
每个节点可能还会设置一个或多个话题(topic)...



在多级层深的ROS系统中，其实现与维护可能会出现一些问题，比如，如何关联不同的功能包，繁多的ROS节点应该如何启动？功能包、节点、话题、参数重名时应该如何处理？不同主机上的节点如何通信？

本章主要内容介绍在ROS中上述问题的解决策略(见本章目录)，预期达成学习目标也与上述问题对应：

- 掌握元功能包使用语法；
- 掌握launch文件的使用语法；
- 理解什么是ROS工作空间覆盖，以及存在什么安全隐患；
- 掌握节点名称重名时的处理方式；
- 掌握话题名称重名时的处理方式；
- 掌握参数名称重名时的处理方式；
- 能够实现ROS分布式通信。

4.1 ROS元功能包

场景:完成ROS中一个系统性的功能，可能涉及到多个功能包，比如实现了机器人导航模块，该模块下有地图、定位、路径规划...等不同的子级功能包。那么调用者安装该模块时，需要逐一的安装每一个功能包吗？

显而易见的，逐一安装功能包的效率低下，在ROS中，提供了一种方式可以将不同的功能包打包成一个功能包，当安装某个功能模块时，直接调用打包后的功能包即可，该包又称之为元功能包(metapackage)。

概念

MetaPackage是Linux的一个文件管理系统的概念。是ROS中的一个虚包，里面没有实质性的内容，但是它依赖了其他的软件包，通过这种方法可以把其他包组合起来，我们可以认为它是一本书的目录索引，告诉我们这个包集中有哪些子包，并且该去哪里下载。

例如：

- `sudo apt install ros-noetic-desktop-full` 命令安装ros时就使用了元功能包，该元功能包依赖于ROS中的其他一些功能包，安装该包时会一并安装依赖。

还有一些常见的MetaPackage：navigation moveit! turtlebot3

作用

方便用户的安装，我们只需要这一个包就可以把其他相关的软件包组织到一起安装了。

实现

首先:新建一个功能包

然后:修改`package.xml` ,内容如下:

```
<exec_depend>被集成的功能包</exec_depend>
.....
<export>
  <metapackage />
</export>
Copy
```

最后:修改 `CMakeLists.txt`,内容如下:

```
cmake_minimum_required(VERSION 3.0.2)
project(demo)
find_package(catkin REQUIRED)
catkin_metapackage()
Copy
```

PS:CMakeLists.txt 中不可以有换行。

4.2 ROS节点运行管理launch文件

关于 launch 文件的使用我们已经不陌生了，在第一章内容中，就曾经介绍到:

一个程序中可能需要启动多个节点，比如:ROS 内置的小乌龟案例，如果要控制乌龟运动，要启动多个窗口，分别启动 roscore、乌龟界面节点、键盘控制节点。如果每次都调用 `roslaunch` 逐一启动，显然效率低下，如何优化？

采用的优化策略便是使用`roslaunch` 命令集合 launch 文件启动管理节点，并且在后续教程中，也多次使用到了 launch 文件。

概念

launch 文件是一个 XML 格式的文件，可以启动本地和远程的多个节点，还可以在参数服务器中设置参数。

作用

简化节点的配置与启动，提高ROS程序的启动效率。

使用

以 turtlesim 为例演示

1.新建launch文件

在功能包下添加 launch目录, 目录下新建 xxxx.launch 文件，编辑 launch 文件

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node" name="myTurtle"
output="screen" />
  <node pkg="turtlesim" type="turtle_teleop_key" name="myTurtleContro"
output="screen" />
</launch>
Copy
```

2.调用 launch 文件

```
roslaunch 包名 xxx.launch
Copy
```

注意:roslaunch 命令执行launch文件时，首先会判断是否启动了 roscore,如果启动了，则不再启动，否则，会自动调用 roscore

PS:本节主要介绍launch文件的使用语法，launch 文件中的标签，以及不同标签的一些常用属性。

4.2.1 launch文件标签之launch

<launch> 标签是所有 launch 文件的根标签，充当其他标签的容器

1.属性

- deprecated = "弃用声明"

告知用户当前 launch 文件已经弃用

2.子级标签

所有其它标签都是launch的子级

4.2.2 launch文件标签之node

<node> 标签用于指定 ROS 节点，是最常见的标签，需要注意的是: roslaunch 命令不能保证按照 node 的声明顺序来启动节点(节点的启动是多进程的)

1.属性

- pkg="包名"
节点所属的包
- type="nodeType"
节点类型(与之相同名称的可执行文件)
- name="nodeName"
节点名称(在 ROS 网络拓扑中节点的名称)
- args="xxx xxx xxx" (可选)
将参数传递给节点
- machine="机器名"
在指定机器上启动节点
- respawn="true | false" (可选)
如果节点退出，是否自动重启
- respawn_delay=" N" (可选)
如果 respawn 为 true, 那么延迟 N 秒后启动节点
- required="true | false" (可选)
该节点是否必须，如果为 true,那么如果该节点退出，将杀死整个 roslaunch
- ns="xxx" (可选)
在指定命名空间 xxx 中启动节点
- clear_params="true | false" (可选)
在启动前，删除节点的私有空间的所有参数
- output="log | screen" (可选)
日志发送目标，可以设置为 log 日志文件，或 screen 屏幕,默认是 log

2.子级标签

- env 环境变量设置
- remap 重映射节点名称
- rosparam 参数设置
- param 参数设置

4.2.3 launch文件标签之include

`include` 标签用于将另一个 xml 格式的 launch 文件导入到当前文件

1.属性

- file="\$(find 包名)/xxx/xxx.launch"
要包含的文件路径
- ns="xxx" (可选)
在指定命名空间导入文件

2.子级标签

- env 环境变量设置
- arg 将参数传递给被包含的文件

4.2.4 launch文件标签之remap

用于话题重命名

1.属性

- from="xxx"
原始话题名称
- to="yyy"
目标名称

2.子级标签

- 无

4.2.5 launch文件标签之param

`<param>` 标签主要用于在参数服务器上设置参数，参数源可以在标签中通过 value 指定，也可以通过外部文件加载，在 `<node>` 标签中时，相当于私有命名空间。

1.属性

- name="命名空间/参数名"
参数名称，可以包含命名空间
- value="xxx" (可选)
定义参数值，如果此处省略，必须指定外部文件作为参数源
- type="str | int | double | bool | yaml" (可选)
指定参数类型，如果未指定，roslaunch 会尝试确定参数类型，规则如下：
 - 如果包含 '.' 的数字解析为浮点型，否则为整型
 - "true" 和 "false" 是 bool 值(不区分大小写)
 - 其他是字符串

2.子级标签

- 无

4.2.6 launch文件标签之rosparam

`<rosparam>` 标签可以从 YAML 文件导入参数，或将参数导出到 YAML 文件，也可以用来删除参数，`<rosparam>` 标签在 `<node>` 标签中时被视为私有。

1.属性

- command="load | dump | delete" (可选, 默认 load)
加载、导出或删除参数
- file="\$(find xxxxx)/xxx/yyy...."
加载或导出到的 yaml 文件
- param="参数名称"
- ns="命名空间" (可选)

2.子级标签

- 无

4.2.7 launch文件标签之group

`<group>` 标签可以对节点分组, 具有 ns 属性, 可以让节点归属某个命名空间

1.属性

- ns="名称空间" (可选)
- clear_params="true | false" (可选)
启动前, 是否删除组名称空间的所有参数(慎用....此功能危险)

2.子级标签

- 除了launch 标签外的其他标签

4.2.8 launch文件标签之arg

`<arg>` 标签是用于动态传参, 类似于函数的参数, 可以增强launch文件的灵活性

1.属性

- name="参数名称"
- default="默认值" (可选)
- value="数值" (可选)
不可以与 default 并存
- doc="描述"
参数说明

2.子级标签

- 无

3.示例

- launch文件传参语法实现,hello.lcaunch

```
<launch>
  <arg name="xxx" />
  <param name="param" value="$(arg xxx)" />
</launch>
Copy
```

- 命令行调用launch传参

```
roslaunch hello.launch xxx:=值
```

4.3 ROS工作空间覆盖

所谓工作空间覆盖，是指不同工作空间中，存在重名的功能包的情形。

ROS 开发中，会自定义工作空间且自定义工作空间可以同时存在多个，可能会出现一种情况：虽然特定工作空间内的功能包不能重名，但是自定义工作空间的功能包与内置的功能包可以重名或者不同的自定义的工作空间中也可以出现重名的功能包，那么调用该名称功能包时，会调用哪一个呢？比如：自定义工作空间A存在功能包 turtlesim，自定义工作空间B也存在功能包 turtlesim，当然系统内置空间也存在turtlesim，如果调用turtlesim包，会调用哪个工作空间中的呢？

实现

0.新建工作空间A与工作空间B，两个工作空间中都创建功能包: turtlesim。

1.在 ~/.bashrc 文件下**追加**当前工作空间的 bash 格式如下：

```
source /home/用户/路径/工作空间A/devel/setup.bash
source /home/用户/路径/工作空间B/devel/setup.bash
Copy
```

2.新开命令行: `source ~/.bashrc` 加载环境变量

3.查看ROS环境环境变量 `echo $ROS_PACKAGE_PATH`

结果:自定义工作空间B:自定义空间A:系统内置空间

4.调用命令: `roscd turtlesim` 会进入自定义工作空间B

原因

ROS 会解析 .bashrc 文件，并生成 ROS_PACKAGE_PATH ROS包路径，该变量中按照 .bashrc 中配置设置工作空间优先级，在设置时需要遵循一定的原则:ROS_PACKAGE_PATH 中的值，和 .bashrc 的配置顺序相反--->后配置的优先级更高，如果更改自定义空间A与自定义空间B的source顺序，那么调用时，将进入工作空间A。

结论

功能包重名时，会按照 ROS_PACKAGE_PATH 查找，配置在前的会优先执行。

隐患

存在安全隐患，比如当前工作空间B优先级更高，意味着当程序调用 turtlesim 时，不会调用工作空间A也不会调用系统内置的 turtlesim，如果工作空间A在实现时有其他功能包依赖于自身的 turtlesim，而按照ROS工作空间覆盖的涉及原则，那么实际执行时将会调用工作空间B的turtlesim，从而导致执行异常，出现安全隐患。

BUG 说明:

当在 .bashrc 文件中 source 多个工作空间后,可能出现的情况,在 ROS PACKAGE PATH 中只包含两个工作空间,可以删除自定义工作空间的 build 与 devel 目录,重新 catkin_make,然后重新载入 .bashrc 文件,问题解决。

4.4 ROS节点名称重名

场景:ROS 中创建的节点是有名称的,C++初始化节点时通过 API: `ros::init(argc,argv,"xxx");` 来定义节点名称,在Python中初始化节点则通过 `rospy.init_node("yyy")` 来定义节点名称。在ROS的网络拓扑中,是不可以出现重名的节点的,因为假设可以重名存在,那么调用时会产生混淆,这也就意味着,不可以启动重名节点或者同一个节点启动多次,的确,在ROS中如果启动重名节点的话,之前已经存在的节点会被直接关闭,但是如果有这种需求的话,怎么优化呢?

在ROS中给出的解决策略是使用命名空间或名称重映射。

命名空间就是为名称添加前缀,名称重映射是为名称起别名。这两种策略都可以解决节点重名问题,两种策略的实现途径有多种:

- rosrun 命令
- launch 文件
- 编码实现

以上三种途径都可以通过命名空间或名称重映射的方式,来避免节点重名,本节将对三者的使用逐一演示,三者要实现的需求类似。

案例

启动两个 turtlesim_node 节点,当然如果直接打开两个终端,直接启动,那么第一次启动的节点会关闭,并给出提示:

```
[ WARN] [1578812836.351049332]: Shutdown request received.  
[ WARN] [1578812836.351207362]: Reason given for shutdown: [new node registered  
with same name]  
Copy
```

因为两个节点不能重名,接下来将会介绍解决重名问题的多种方案。

4.4.1 rosrun设置命名空间与重映射

1. rosrun设置命名空间

1.1 设置命名空间演示

语法: rosrun 包名 节点名 __ns:=新名称

```
roslaunch turtlesim turtlesim_node __ns:=/xxx  
Copy  
roslaunch turtlesim turtlesim_node __ns:=/yyy  
Copy
```

两个节点都可以正常运行

1.2运行结果

`rostopic list` 查看节点信息,显示结果:

```
/xxx/turtlesim  
/yyy/turtlesim  
Copy
```

2. rosrun 名称重映射

2.1 为节点起别名

语法: `roslaunch 包名 节点名 __name:=新名称`

```
roslaunch turtlesim turtlesim_node __name:=t1 | roslaunch turtlesim turtlesim_node  
/turtlesim:=t1(不适用于python)  
Copy  
roslaunch turtlesim turtlesim_node __name:=t2 | roslaunch turtlesim turtlesim_node  
/turtlesim:=t2(不适用于python)  
Copy
```

两个节点都可以运行

2.2 运行结果

`rostopic list` 查看节点信息,显示结果:

```
/t1  
/t2  
Copy
```

3. roslaunch 命名空间与名称重映射叠加

3.1 设置命名空间同时名称重映射

语法: `roslaunch 包名 节点名 ns:=新名称 name:=新名称`

```
roslaunch turtlesim turtlesim_node __ns:=/xxx __name:=tn  
Copy
```

3.2 运行结果

`rostopic list` 查看节点信息,显示结果:

```
/xxx/tn  
Copy
```

使用环境变量也可以设置命名空间,启动节点前在终端键入如下命令:

```
export ROS_NAMESPACE=xxxx
```

4.4.2 launch文件设置命名空间与重映射

介绍 launch 文件的使用语法时，在 node 标签中有两个属性: name 和 ns，二者分别是用于实现名称重映射与命名空间设置的。使用launch文件设置命名空间与名称重映射也比较简单。

1.launch文件

```
<launch>

  <node pkg="turtlesim" type="turtlesim_node" name="t1" />
  <node pkg="turtlesim" type="turtlesim_node" name="t2" />
  <node pkg="turtlesim" type="turtlesim_node" name="t1" ns="hello"/>

</launch>
Copy
```

在 node 标签中，name 属性是必须的，ns 可选。

2.运行

`rostopic list` 查看节点信息,显示结果:

```
/t1
/t2
/t1/hello
```

4.4.3 编码设置命名空间与重映射

如果自定义节点实现，那么可以更灵活的设置命名空间与重映射实现。

1.C++ 实现:重映射

1.1名称别名设置

核心代码: `ros::init(argc,argv,"zhangsan",ros::init_options::AnonymousName);`

1.2执行

会在名称后面添加时间戳。

2.C++ 实现:命名空间

2.1命名空间设置

核心代码

```
std::map<std::string, std::string> map;
map["__ns"] = "xxxx";
ros::init(map,"wangqiang");
Copy
```

2.2执行

节点名称设置了命名空间。

3.Python 实现:重映射

3.1名称别名设置

核心代码: `rospy.init_node("lisi",anonymous=True)`

3.2执行

会在节点名称后缀时间戳。

4.5 ROS话题名称设置

在ROS中节点名称可能出现重名的情况，同理话题名称也可能重名。

在 ROS 中节点终端，不同的节点之间通信都依赖于话题，话题名称也可能出现重复的情况，这种情况下，系统虽然不会抛出异常，但是可能导致订阅的消息非预期的，从而导致节点运行异常。这种情况下需要将两个节点的话题名称由相同修改为不同。

又或者，两个节点是可以通信的，两个节点之间使用了相同的消息类型，但是由于，话题名称不同，导致通信失败。这种情况下需要将两个节点的话题名称由不同修改为相同。

在实际应用中，按照逻辑，有些时候可能需要将相同的话题名称设置为不同，也有可能将不同的话题名设置为相同。在ROS中给出的解决策略与节点名称重命名类似，也是使用名称重映射或为名称添加前缀。根据前缀不同，有全局、相对、和私有三种类型之分。

- 全局(参数名称直接参考ROS系统，与节点命名空间同级)
- 相对(参数名称参考的是节点的命名空间，与节点名称同级)
- 私有(参数名称参考节点名称，是节点名称的子级)

名称重映射是为名称起别名，为名称添加前缀，该实现比节点重命名更复杂些，不单是使用命名空间作为前缀、还可以使用节点名称作为前缀。两种策略的实现途径有多种：

- `roslaunch` 命令
- `launch` 文件
- 编码实现

本节将对三者的使用逐一演示，三者要实现的需求类似。

案例

在ROS中提供了一个比较好用的键盘控制功能包: `ros-noetic-teleop-twist-keyboard`，该功能包，可以控制机器人的运动，作用类似于乌龟的键盘控制节点，可以使用 `sudo apt install ros-noetic-teleop-twist-keyboard` 来安装该功能包，然后执行: `roslaunch teleop_twist_keyboard teleop_twist_keyboard.py`，在启动乌龟显示节点，不过此时前者不能控制乌龟运动，因为，二者使用的话题名称不同，前者使用的是 `cmd_vel` 话题，后者使用的是 `/turtle1/cmd_vel` 话题。需要将话题名称修改为一致，才能使用，如何实现？

4.5.1 rosrun设置话题重映射

rosrun名称重映射语法: rosrun 包名 节点名 话题名:=新话题名称

实现teleop_twist_keyboard与乌龟显示节点通信方案由两种:

1.方案1

将 teleop_twist_keyboard 节点的话题设置为 /turtle1/cmd_vel

启动键盘控制节点: `roslaunch teleop_twist_keyboard teleop_twist_keyboard.py`
`/cmd_vel:=/turtle1/cmd_vel`

启动乌龟显示节点: `roslaunch turtlesim turtlesim_node`

二者可以实现正常通信

2.方案2

将乌龟显示节点的话题设置为 `/cmd_vel`

启动键盘控制节点: `roslaunch teleop_twist_keyboard teleop_twist_keyboard.py`

启动乌龟显示节点: `roslaunch turtlesim turtlesim_node /turtle1/cmd_vel:=/cmd_vel`

二者可以实现正常通信

4.5.2 launch文件设置话题重映射

launch 文件设置话题重映射语法:

```
<node pkg="xxx" type="xxx" name="xxx">
  <remap from="原话题" to="新话题" />
</node>
Copy
```

实现teleop_twist_keyboard与乌龟显示节点通信方案由两种:

1.方案1

将 teleop_twist_keyboard 节点的话题设置为 `/turtle1/cmd_vel`

```
<launch>

  <node pkg="turtlesim" type="turtlesim_node" name="t1" />
  <node pkg="teleop_twist_keyboard" type="teleop_twist_keyboard.py"
name="key">
    <remap from="/cmd_vel" to="/turtle1/cmd_vel" />
  </node>

</launch>
Copy
```

二者可以实现正常通信

2.方案2

将乌龟显示节点的话题设置为 `/cmd_vel`

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node" name="t1">
    <remap from="/turtle1/cmd_vel" to="/cmd_vel" />
  </node>
  <node pkg="teleop_twist_keyboard" type="teleop_twist_keyboard.py" name="key"
/>

</launch>
Copy
```

二者可以实现正常通信

4.5.3 编码设置话题名称

话题的名称与节点的命名空间、节点的名称是有一定关系的，话题名称大致可以分为三种类型：

- 全局(话题参考ROS系统，与节点命名空间平级)
- 相对(话题参考的是节点的命名空间，与节点名称平级)
- 私有(话题参考节点名称，是节点名称的子级)

结合编码演示具体关系。

1.C++ 实现

演示准备：

1.初始化节点设置一个节点名称

```
ros::init(argc,argv,"hello")
```

2.设置不同类型的话题

3.启动节点时，传递一个 `__ns:= xxx`

4.节点启动后，使用 `rostopic` 查看话题信息

1.1全局名称

格式:以 `/` 开头的名称，和节点名称无关

比如: `/xxx/yyy/zzz`

示例1: `ros::Publisher pub = nh.advertise<std_msgs::String>("/chatter",1000);`

结果1: `/chatter`

示例2: `ros::Publisher pub = nh.advertise<std_msgs::String>("/chatter/money",1000);`

结果2: `/chatter/money`

1.2相对名称

格式:非 / 开头的名称,参考命名空间(与节点名称平级)来确定话题名称

示例1: `ros::Publisher pub = nh.advertise<std_msgs::String>("chatter",1000);`

结果1: `xxx/chatter`

示例2: `ros::Publisher pub = nh.advertise<std_msgs::String>("chatter/money",1000);`

结果2: `xxx/chatter/money`

1.3私有名称

格式:以 ~ 开头的名称

示例1:

```
ros::NodeHandle nh("~");
ros::Publisher pub = nh.advertise<std_msgs::String>("chatter",1000);
```

结果1: `/xxx/hello/chatter`

示例2:

```
ros::NodeHandle nh("~");
ros::Publisher pub = nh.advertise<std_msgs::String>("chatter/money",1000);
```

结果2: `/xxx/hello/chatter/money`

PS:当使用 ~,而话题名称有时 / 开头时,那么话题名称是绝对的

示例3:

```
ros::NodeHandle nh("~");
ros::Publisher pub = nh.advertise<std_msgs::String>("/chatter/money",1000);
```

结果3: `/chatter/money`

2.Python 实现

演示准备:

1.初始化节点设置一个节点名称

```
rospy.init_node("hello")
```

2.设置不同类型的话题

3.启动节点时, 传递一个 __ns:= xxx

4.节点启动后, 使用 rostopic 查看话题信息

2.1全局名称

格式:以 / 开头的名称, 和节点名称无关

示例1: `pub = rospy.Publisher("/chatter",String,queue_size=1000)`

结果1: /chatter

示例2: `pub = rospy.Publisher("/chatter/money",String,queue_size=1000)`

结果2: /chatter/money

2.2相对名称

格式:非 / 开头的名称,参考命名空间(与节点名称平级)来确定话题名称

示例1: `pub = rospy.Publisher("chatter",String,queue_size=1000)`

结果1: xxx/chatter

示例2: `pub = rospy.Publisher("chatter/money",String,queue_size=1000)`

结果2: xxx/chatter/money

2.3私有名称

格式:以 ~ 开头的名称

示例1: `pub = rospy.Publisher("~chatter",String,queue_size=1000)`

结果1: /xxx/hello/chatter

示例2: `pub = rospy.Publisher("~chatter/money",String,queue_size=1000)`

结果2: /xxx/hello/chatter/money

4.6 ROS参数名称设置

在ROS中节点名称话题名称可能出现重名的情况, 同理参数名称也可能重名。

当参数名称重名时, 那么就会产生覆盖, 如何避免这种情况?

关于参数重名的处理, 没有重映射实现, 为了尽量的避免参数重名, 都是使用为参数名添加前缀的方式, 实现类似于话题名称, 有全局、相对、和私有三种类型之分。

- 全局(参数名称直接参考ROS系统, 与节点命名空间平级)
- 相对(参数名称参考的是节点的命名空间, 与节点名称平级)
- 私有(参数名称参考节点名称, 是节点名称的子级)

设置参数的方式也有三种:

- rosrun 命令
- launch 文件
- 编码实现

三种设置方式前面都已经有所涉及, 但是之前没有涉及命名问题, 本节将对三者命名的设置逐一演示。

案例

启动节点时，为参数服务器添加参数(需要注意参数名称设置)。

4.6.1 rosrun设置参数

roslaunch 在启动节点时，也可以设置参数：

语法：roslaunch 包名 节点名称 _参数名:=参数值

1.设置参数

启动乌龟显示节点，并设置参数 A = 100

```
roslaunch turtlesim turtlesim_node _A:=100
Copy
```

2.运行

rosparam list 查看节点信息,显示结果：

```
/turtlesim/A
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
Copy
```

结果显示，参数A前缀节点名称，也就是说roslaunch执行设置参数参数名使用的是私有模式

4.6.2 launch文件设置参数

通过 launch 文件设置参数的方式前面已经介绍过了，可以在 node 标签外，或 node 标签中通过 param 或 rosparam 来设置参数。在 node 标签外设置的参数是全局性质的，参考的是 /，在 node 标签中设置的参数是私有性质的，参考的是 /命名空间/节点名称。

1.设置参数

以 param 标签为例，设置参数

```
<launch>

  <param name="p1" value="100" />
  <node pkg="turtlesim" type="turtlesim_node" name="t1">
    <param name="p2" value="100" />
  </node>

</launch>
Copy
```

2.运行

rosparam list 查看节点信息,显示结果：

```
/p1
/t1/p1
Copy
```

运行结果与预期一致。

4.6.3 编码设置参数

编码的方式可以更方便的设置:全局、相对与私有参数。

1.C++实现

在 C++ 中, 可以使用 `ros::param` 或者 `ros::NodeHandle` 来设置参数。

1.1 `ros::param` 设置参数

设置参数调用API是`ros::param::set`, 该函数中, 参数1传入参数名称, 参数2是传入参数值, 参数1中参数名称设置时, 如果以 `/` 开头, 那么就是全局参数, 如果以 `~` 开头, 那么就是私有参数, 既不以 `/` 也不以 `~` 开头, 那么就是相对参数。代码示例:

```
ros::param::set("/set_A",100); //全局,和命名空间以及节点名称无关
ros::param::set("set_B",100); //相对,参考命名空间
ros::param::set("~set_C",100); //私有,参考命名空间与节点名称
Copy
```

运行时, 假设设置的 namespace 为 xxx, 节点名称为 yyy, 使用 `rosparam list` 查看:

```
/set_A
/xxx/set_B
/xxx/yyy/set_C
Copy
```

1.2 `ros::NodeHandle` 设置参数

设置参数时, 首先需要创建 `NodeHandle` 对象, 然后调用该对象的 `setParam` 函数, 该函数参数1为参数名, 参数2为要设置的参数值, 如果参数名以 `/` 开头, 那么就是全局参数, 如果参数名不以 `/` 开头, 那么, 该参数是相对参数还是私有参数与 `NodeHandle` 对象有关, 如果 `NodeHandle` 对象创建时如果是调用的默认的空参构造, 那么该参数是相对参数, 如果 `NodeHandle` 对象创建时是使用:

`ros::NodeHandle nh("~")`, 那么该参数就是私有参数。代码示例:

```
ros::NodeHandle nh;
nh.setParam("/nh_A",100); //全局,和命名空间以及节点名称无关

nh.setParam("nh_B",100); //相对,参考命名空间

ros::NodeHandle nh_private("~");
nh_private.setParam("nh_C",100); //私有,参考命名空间与节点名称
Copy
```

运行时, 假设设置的 namespace 为 xxx, 节点名称为 yyy, 使用 `rosparam list` 查看:

```
/nh_A
/xxx/nh_B
/xxx/yyy/nh_C
Copy
```

2.python实现

python 中关于参数设置的语法实现比 C++ 简洁一些，调用的API时 `rospy.set_param`，该函数中，参数1传入参数名称，参数2是传入参数值，参数1中参数名称设置时，如果以 `/` 开头，那么就是全局参数，如果以 `~` 开头，那么就是私有参数，既不以 `/` 也不以 `~` 开头，那么就是相对参数。代码示例：

```
rospy.set_param("/py_A",100) #全局,和命名空间以及节点名称无关
rospy.set_param("py_B",100) #相对,参考命名空间
rospy.set_param("~py_C",100) #私有,参考命名空间与节点名称
Copy
```

运行时，假设设置的 namespace 为 xxx，节点名称为 yyy，使用 `rosparam list` 查看：

```
/py_A
/xxx/py_B
/xxx/yyy/py_C
```

4.7 ROS分布式通信

ROS是一个分布式计算环境。一个运行中的ROS系统可以包含分布在多台计算机上多个节点。根据系统的配置方式，任何节点可能随时需要与任何其他节点进行通信。

因此，ROS对网络配置有某些要求：

- 所有端口上的所有机器之间必须有完整的双向连接。
- 每台计算机必须通过所有其他计算机都可以解析的名称来公告自己。

实现

1.准备

先要保证不同计算机处于同一网络中，最好分别设置固定IP，如果为虚拟机，需要将网络适配器改为桥接模式；

2.配置文件修改

分别修改不同计算机的 `/etc/hosts` 文件，在该文件中加入对方的IP地址和计算机名：

主机端：

```
从机的IP      从机计算机名
Copy
```

从机端：

```
主机的IP      主机计算机名
Copy
```

设置完毕，可以通过 ping 命令测试网络通信是否正常。

IP地址查看名: ifconfig

计算机名称查看: hostname

3.配置主机IP

配置主机的 IP 地址

~/bashrc 追加

```
export ROS_MASTER_URI=http://主机IP:11311
export ROS_HOSTNAME=主机IP
Copy
```

4.配置从机IP

配置从机的 IP 地址，从机可以有多台，每台都做如下设置:

~/bashrc 追加

```
export ROS_MASTER_URI=http://主机IP:11311
export ROS_HOSTNAME=从机IP
Copy
```

测试

1.主机启动 roscore(必须)

2.主机启动订阅节点，从机启动发布节点，测试通信是否正常

3.反向测试，主机启动发布节点，从机启动订阅节点，测试通信是否正常

4.8 本章小结

本章主要介绍了ROS的运行管理机制，内容如下:

- 如何通过元功能包关联工作空间下的不同功能包
- 使用 launch 文件来管理维护 ROS 中的节点
- 在 ROS 中重名是经常出现的，重名时会导致什么情况？以及怎么避免重名？
- 如何实现 ROS 分布式通信？

本章的重点是"重名"相关的内容:

- 包名重复，会导致覆盖。
- 节点名称重复，会导致先启动的节点关闭
- 话题名称重复，无语法异常，但是可能导致通信实现出现逻辑问题
- 参数名称重复，会导致参数设置的覆盖

解决重名问题的实现方案有两种:

- 重映射(重新起名字)
- 为命名添加前缀

本章介绍的内容还是偏向语法层面的实现，下一章将开始介绍ROS中内置的一些较为实用的组件。

第 5 章 ROS常用组件

在ROS中内置一些比较实用的工具，通过这些工具可以方便快捷的实现某个功能或调试程序，从而提高开发效率，本章主要介绍ROS中内置的如下组件：

- TF坐标变换，实现不同类型的坐标系之间的转换；
- rosbag 用于录制ROS节点的执行过程并可以重放该过程；
- rqt 工具箱，集成了多款图形化的调试工具。

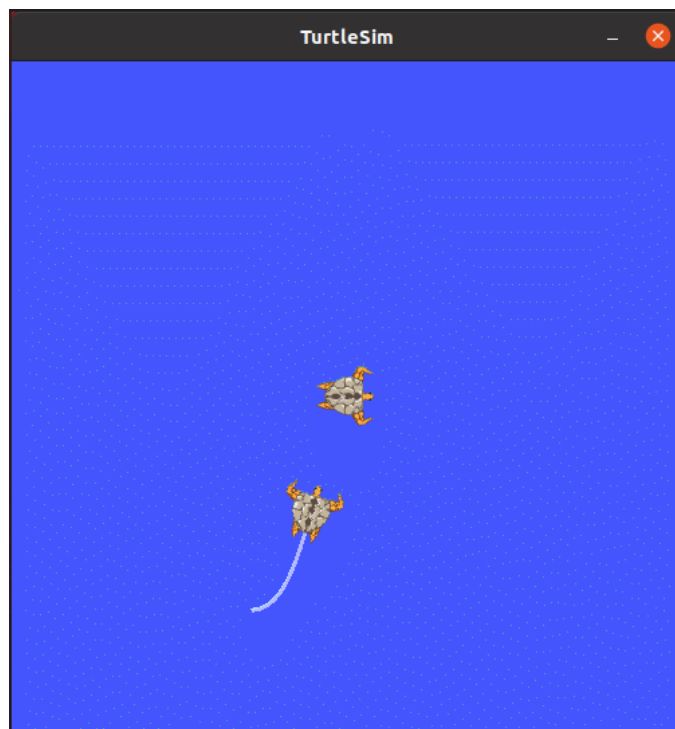
本章预期达成的学习目标：

- 了解 TF 坐标变换的概念以及应用场景；
- 能够独立完成TF案例:小乌龟跟随；
- 可以使用 rosbag 命令或编码的形式实现录制与回放；
- 能够熟练使用rqt中的图形化工具。

案例演示: 小乌龟跟随实现，该案例是ros中内置案例，终端下键入启动命令

```
roslaunch turtle_tf2 turtle_tf2_demo_cpp.launch`或`roslaunch turtle_tf2
turtle_tf2_demo.launch
```

键盘可以控制一只乌龟运动，另一只跟随运动。

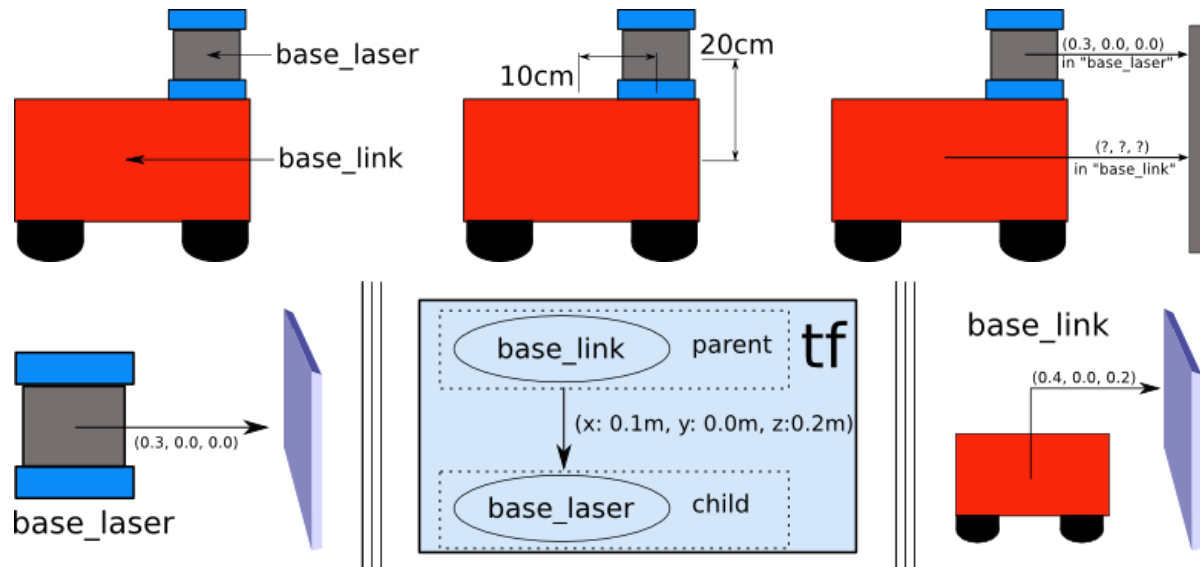


5.1 TF坐标变换

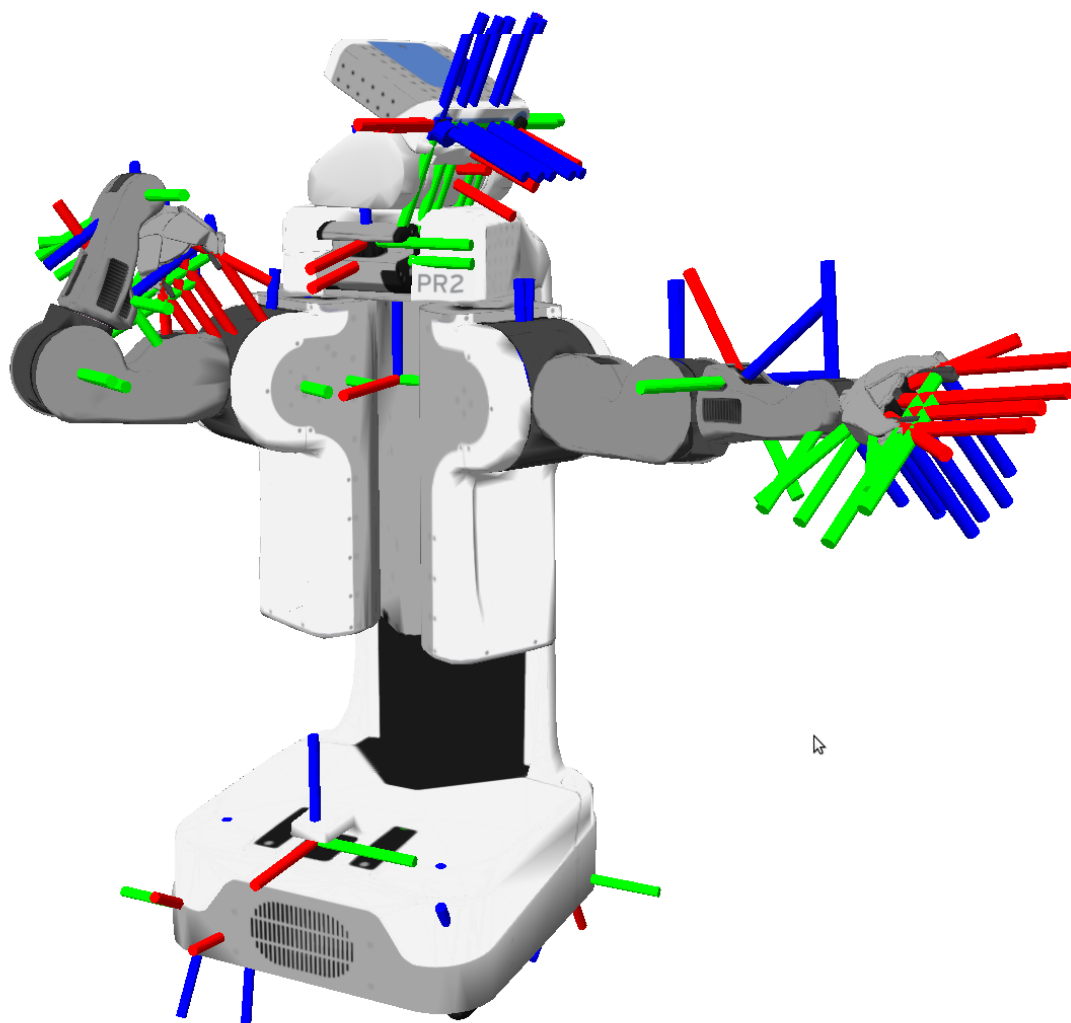
机器人系统上，有多个传感器，如激光雷达、摄像头等，有的传感器是可以感知机器人周边的物体方位(或者称之为:坐标，横向、纵向、高度的距离信息)的，以协助机器人定位障碍物，可以直接将物体相对该传感器的方位信息，等价于物体相对于机器人系统或机器人其它组件的方位信息吗？显示是不行的，这中间需要一个转换过程。更具体描述如下：

场景1:雷达与小车

现有一移动式机器人底盘，在底盘上安装了一雷达，雷达相对于底盘的偏移量已知，现雷达检测到一障碍物信息，获取到坐标分别为 (x,y,z) ，该坐标是以雷达为参考系的，如何将这个坐标转换成以小车为参考系的坐标呢？



场景2:现有一带机械臂的机器人(比如:PR2)需要夹取目标物，当前机器人头部摄像头可以探测到目标物的坐标 (x,y,z) ，不过该坐标是以摄像头为参考系的，而实际操作目标物的是机械臂的夹具，当前我们需要将该坐标转换成相对于机械臂夹具的坐标，这个过程如何实现？

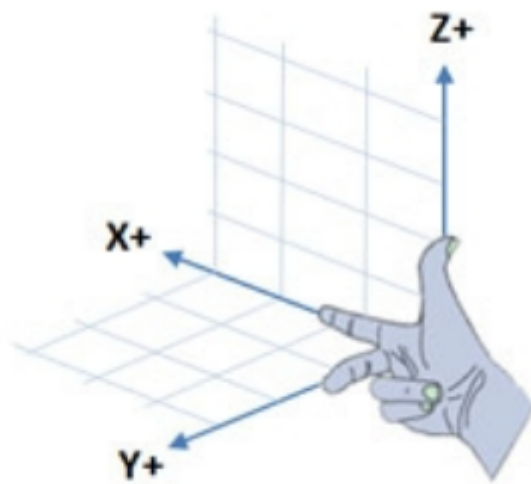


当然，根据我们高中学习的知识，在明确了不同坐标系之间的相对关系，就可以实现任何坐标点在不同坐标系之间的转换，但是该计算实现是较为常用的，且算法也有点复杂，因此在 ROS 中直接封装了相关的模块: 坐标变换(TF)。

概念

tf: Transform Frame,坐标变换

坐标系:ROS 中是通过坐标系统来标定物体的，确切的是通过右手坐标系来标定的。



作用

在 ROS 中用于实现不同坐标系之间的点或向量的转换。

案例

小乌龟跟随案例: 如本章引言部分演示。

说明

在ROS中坐标变换最初对应的是tf，不过在 hydro 版本开始, tf 被弃用，迁移到 tf2,后者更为简洁高效，tf2对应的常用功能包有:

tf2_geometry_msgs:可以将ROS消息转换成tf2消息。

tf2: 封装了坐标变换的常用消息。

tf2_ros:为tf2提供了roscpp和rospy绑定，封装了坐标变换常用的API。

5.1.1 坐标msg消息

订阅发布模型中数据载体 msg 是一个重要实现，首先需要了解一下，在坐标转换实现中常用的 msg: geometry_msgs/TransformStamped 和 geometry_msgs/PointStamped

前者用于传输坐标系相关位置信息，后者用于传输某个坐标系内坐标点的信息。在坐标变换中，频繁的需要使用到坐标系的相对关系以及坐标点信息。

1.geometry_msgs/TransformStamped

命令行键入: `rosmmsg info geometry_msgs/TransformStamped`

```
std_msgs/Header header          #头信息
  uint32 seq                    #|-- 序列号
  time stamp                    #|-- 时间戳
  string frame_id               #|-- 坐标 ID
string child_frame_id           #子坐标系的 id
geometry_msgs/Transform transform #坐标信息
  geometry_msgs/Vector3 translation #偏移量
    float64 x                    #|-- X 方向的偏移量
    float64 y                    #|-- Y 方向的偏移量
    float64 z                    #|-- Z 方向上的偏移量
  geometry_msgs/Quaternion rotation #四元数
    float64 x
    float64 y
    float64 z
    float64 w
Copy
```

四元数用于表示坐标的相对姿态

2.geometry_msgs/PointStamped

命令行键入: `rosmmsg info geometry_msgs/PointStamped`

```
std_msgs/Header header          #头
  uint32 seq                    #|-- 序号
  time stamp                    #|-- 时间戳
  string frame_id               #|-- 所属坐标系的 id
geometry_msgs/Point point        #点坐标
  float64 x                    #|-- x y z 坐标
  float64 y
  float64 z
Copy
```

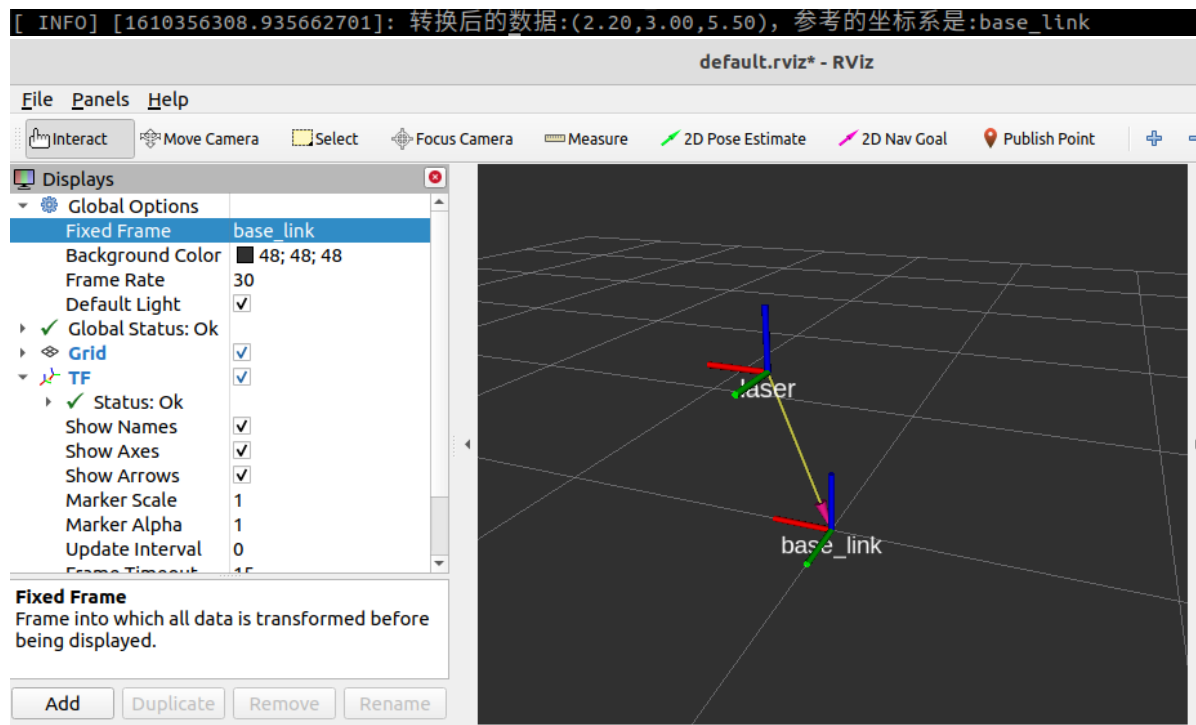
5.1.2 静态坐标变换

所谓静态坐标变换，是指两个坐标系之间的相对位置是固定的。

需求描述:

现有一机器人模型，核心构成包含主体与雷达，各对应一坐标系，坐标系的原点分别位于主体与雷达的物理中心，已知雷达原点相对于主体原点位移关系如下: x 0.2 y0.0 z0.5。当前雷达检测到一障碍物，在雷达坐标系中障碍物的坐标为 (2.0 3.0 5.0),请问，该障碍物相对于主体的坐标是多少？

结果演示:



实现分析:

1. 坐标系相对关系, 可以通过发布方发布
2. 订阅方, 订阅到发布的坐标系相对关系, 再传入坐标点信息(可以写死), 然后借助于 tf 实现坐标变换, 并将结果输出

实现流程:C++ 与 Python 实现流程一致

1. 新建功能包, 添加依赖
2. 编写发布方实现
3. 编写订阅方实现
4. 执行并查看结果

方案A:C++实现

1.创建功能包

创建项目功能包依赖于 tf2、tf2_ros、tf2_geometry_msgs、roscpp rospy std_msgs geometry_msgs

2.发布方

```
/*
    静态坐标变换发布方:
        发布关于 laser 坐标系的位置信息

    实现流程:
        1. 包含头文件
        2. 初始化 ROS 节点
        3. 创建静态坐标转换广播器
        4. 创建坐标系信息
        5. 广播器发布坐标系信息
        6. spin()
*/
```

```
// 1.包含头文件
#include "ros/ros.h"
#include "tf2_ros/static_transform_broadcaster.h"
#include "geometry_msgs/TransformStamped.h"
#include "tf2/LinearMath/Quaternion.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ROS 节点
    ros::init(argc, argv, "static_brocast");
    // 3.创建静态坐标转换广播器
    tf2_ros::StaticTransformBroadcaster broadcaster;
    // 4.创建坐标系信息
    geometry_msgs::TransformStamped ts;
    //----设置头信息
    ts.header.seq = 100;
    ts.header.stamp = ros::Time::now();
    ts.header.frame_id = "base_link";
    //----设置子级坐标系
    ts.child_frame_id = "laser";
    //----设置子级相对于父级的偏移量
    ts.transform.translation.x = 0.2;
    ts.transform.translation.y = 0.0;
    ts.transform.translation.z = 0.5;
    //----设置四元数:将 欧拉角数据转换成四元数
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,0);
    ts.transform.rotation.x = qtn.getX();
    ts.transform.rotation.y = qtn.getY();
    ts.transform.rotation.z = qtn.getZ();
    ts.transform.rotation.w = qtn.getW();
    // 5.广播器发布坐标系信息
    broadcaster.sendTransform(ts);
    ros::spin();
    return 0;
}
Copy
```

配置文件此处略。

3.订阅方

```
/*
    订阅坐标系信息，生成一个相对于 子级坐标系的坐标点数据，转换成父级坐标系中的坐标点

    实现流程：
        1.包含头文件
        2.初始化 ROS 节点
        3.创建 TF 订阅节点
        4.生成一个坐标点(相对于子级坐标系)
        5.转换坐标点(相对于父级坐标系)
        6.spin()
*/
//1.包含头文件
#include "ros/ros.h"
#include "tf2_ros/transform_listener.h"
```

```

#include "tf2_ros/buffer.h"
#include "geometry_msgs/PointStamped.h"
#include "tf2_geometry_msgs/tf2_geometry_msgs.h" //注意：调用 transform 必须包含该头文件

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ROS 节点
    ros::init(argc, argv, "tf_sub");
    ros::NodeHandle nh;
    // 3.创建 TF 订阅节点
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);

    ros::Rate r(1);
    while (ros::ok())
    {
        // 4.生成一个坐标点(相对于子级坐标系)
        geometry_msgs::PointStamped point_laser;
        point_laser.header.frame_id = "laser";
        point_laser.header.stamp = ros::Time::now();
        point_laser.point.x = 1;
        point_laser.point.y = 2;
        point_laser.point.z = 7.3;
        // 5.转换坐标点(相对于父级坐标系)
        //新建一个坐标点，用于接收转换结果
        //-----使用 try 语句或休眠，否则可能由于缓存接收延迟而导致坐标转换失败---
        -----
        try
        {
            geometry_msgs::PointStamped point_base;
            point_base = buffer.transform(point_laser, "base_link");
            ROS_INFO("转换后的数据: (%.2f, %.2f, %.2f), 参考的坐标系是: ", point_base.point.x, point_base.point.y, point_base.point.z, point_base.header.frame_id.c_str());

        }
        catch(const std::exception& e)
        {
            // std::cerr << e.what() << '\n';
            ROS_INFO("程序异常.....");
        }

        r.sleep();
        ros::spinOnce();
    }

    return 0;
}
Copy

```

配置文件此处略。

4.执行

可以使用命令行或launch文件的方式分别启动发布节点与订阅节点，如果程序无异常，控制台将输出，坐标转换后的结果。

方案B:Python实现

1.创建功能包

创建项目功能包依赖于 tf2、tf2_ros、tf2_geometry_msgs、roscpp rospy std_msgs geometry_msgs

2.发布方

```
#!/usr/bin/env python
"""
    静态坐标变换发布方：
        发布关于 laser 坐标系的位置信息
    实现流程：
        1. 导包
        2. 初始化 ROS 节点
        3. 创建 静态坐标广播器
        4. 创建并组织被广播的消息
        5. 广播器发送消息
        6. spin
"""
# 1. 导包
import rospy
import tf2_ros
import tf
from geometry_msgs.msg import TransformStamped

if __name__ == "__main__":
    # 2. 初始化 ROS 节点
    rospy.init_node("static_tf_pub_p")
    # 3. 创建 静态坐标广播器
    broadcaster = tf2_ros.StaticTransformBroadcaster()
    # 4. 创建并组织被广播的消息
    tfs = TransformStamped()
    # --- 头信息
    tfs.header.frame_id = "world"
    tfs.header.stamp = rospy.Time.now()
    tfs.header.seq = 101
    # --- 子坐标系
    tfs.child_frame_id = "radar"
    # --- 坐标系相对信息
    # ----- 偏移量
    tfs.transform.translation.x = 0.2
    tfs.transform.translation.y = 0.0
    tfs.transform.translation.z = 0.5
    # ----- 四元数
    qtn = tf.transformations.quaternion_from_euler(0,0,0)
    tfs.transform.rotation.x = qtn[0]
    tfs.transform.rotation.y = qtn[1]
    tfs.transform.rotation.z = qtn[2]
    tfs.transform.rotation.w = qtn[3]
```

```
# 5.广播器发送消息
broadcaster.sendTransform(tfs)
# 6.spin
rospy.spin()
Copy
```

权限设置以及配置文件此处略。

3.订阅方

```
#!/usr/bin/env python
"""
    订阅坐标系信息，生成一个相对于 子级坐标系的坐标点数据，
    转换成父级坐标系中的坐标点

    实现流程：
        1.导包
        2.初始化 ROS 节点
        3.创建 TF 订阅对象
        4.创建一个 radar 坐标系中的坐标点
        5.调研订阅对象的 API 将 4 中的点坐标转换成相对于 world 的坐标
        6.spin
"""
# 1.导包
import rospy
import tf2_ros
# 不要使用 geometry_msgs,需要使用 tf2 内置的消息类型
from tf2_geometry_msgs import PointStamped
# from geometry_msgs.msg import PointStamped

if __name__ == "__main__":
    # 2.初始化 ROS 节点
    rospy.init_node("static_sub_tf_p")
    # 3.创建 TF 订阅对象
    buffer = tf2_ros.Buffer()
    listener = tf2_ros.TransformListener(buffer)

    rate = rospy.Rate(1)
    while not rospy.is_shutdown():
        # 4.创建一个 radar 坐标系中的坐标点
        point_source = PointStamped()
        point_source.header.frame_id = "radar"
        point_source.header.stamp = rospy.Time.now()
        point_source.point.x = 10
        point_source.point.y = 2
        point_source.point.z = 3

        try:
            # 5.调研订阅对象的 API 将 4 中的点坐标转换成相对于 world 的坐标
            point_target = buffer.transform(point_source,"world")
            rospy.loginfo("转换结果:x = %.2f, y = %.2f, z = %.2f",
                          point_target.point.x,
                          point_target.point.y,
                          point_target.point.z)
        except Exception as e:
```

```
rospy.logerr("异常:%s",e)

# 6.spin
rate.sleep()

Copy
```

权限设置以及配置文件此处略。

PS: 在 tf2 的 python 实现中, tf2 已经封装了一些消息类型, 不可以使用 geometry_msgs.msg 中的类型

4.执行

可以使用命令行或launch文件的方式分别启动发布节点与订阅节点, 如果程序无异常, 控制台将输出, 坐标转换后的结果。

补充1:

当坐标系之间的相对位置固定时, 那么所需参数也是固定的: 父系坐标名称、子级坐标系名称、x偏移量、y偏移量、z偏移量、x 翻滚角度、y俯仰角度、z偏航角度, 实现逻辑相同, 参数不同, 那么 ROS 系统就已经封装好了专门的节点, 使用方式如下:

```
roslaunch tf2_ros static_transform_publisher x偏移量 y偏移量 z偏移量 z偏航角度 y俯仰角度 x翻滚角度 父级坐标系 子级坐标系
```

示例: `roslaunch tf2_ros static_transform_publisher 0.2 0 0.5 0 0 0 /base_link /laser`

也建议使用该种方式直接实现静态坐标系相对信息发布。

补充2:

可以借助于rviz显示坐标系关系, 具体操作:

- 新建窗口输入命令:rviz;
- 在启动的 rviz 中设置Fixed Frame 为 base_link;
- 点击左下的 add 按钮, 在弹出的窗口中选择 TF 组件, 即可显示坐标关系。

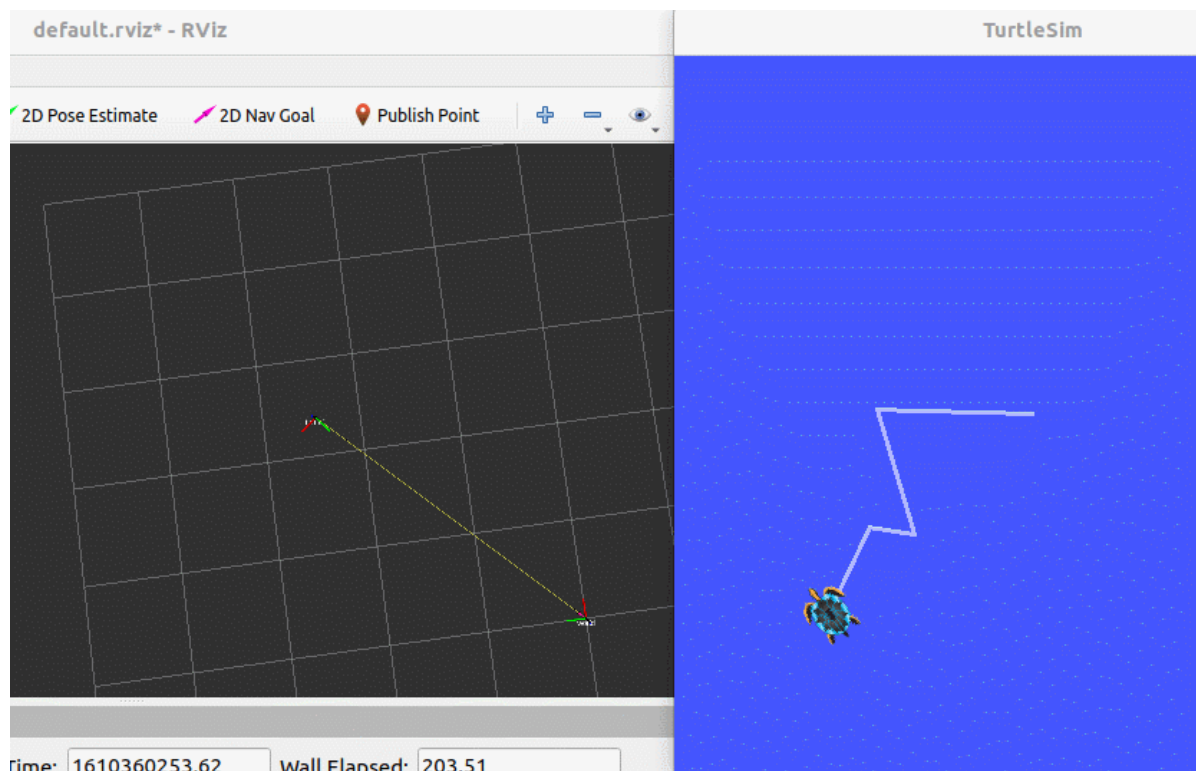
5.1.3 动态坐标变换

所谓动态坐标变换, 是指两个坐标系之间的相对位置是变化的。

需求描述:

启动 turtlesim_node,该节点中窗体有一个世界坐标系(左下角为坐标系原点), 乌龟是另一个坐标系, 键盘控制乌龟运动, 将两个坐标系的相对位置动态发布。

结果演示:



实现分析:

1. 乌龟本身不但可以看作坐标系，也是世界坐标系中的一个坐标点
2. 订阅 `turtle1/pose`, 可以获取乌龟在世界坐标系的 `x` 坐标、`y` 坐标、偏移量以及线速度和角速度
3. 将 `pose` 信息转换成 坐标系相对信息并发布

实现流程:C++ 与 Python 实现流程一致

1. 新建功能包，添加依赖
2. 创建坐标相对关系发布方(同时需要订阅乌龟位姿信息)
3. 创建坐标相对关系订阅方
4. 执行

方案A:C++实现

1.创建功能包

创建项目功能包依赖于 `tf2`、`tf2_ros`、`tf2_geometry_msgs`、`roscpp` `rospy` `std_msgs` `geometry_msgs`、`turtlesim`

2.发布方

/*

动态的坐标系相对姿态发布(一个坐标系相对于另一个坐标系的相对姿态是不断变动的)

需求：启动 `turtlesim_node`, 该节点中窗体有一个世界坐标系(左下角为坐标系原点)，乌龟是另一个坐标系，键盘

控制乌龟运动，将两个坐标系的相对位置动态发布

实现分析：

1. 乌龟本身不但可以看作坐标系，也是世界坐标系中的一个坐标点
2. 订阅 `turtle1/pose`, 可以获取乌龟在世界坐标系的 `x` 坐标、`y` 坐标、偏移量以及线速度和角速度
3. 将 `pose` 信息转换成 坐标系相对信息并发布

实现流程：

1. 包含头文件
2. 初始化 ROS 节点
3. 创建 ROS 句柄
4. 创建订阅对象
5. 回调函数处理订阅到的数据(实现TF广播)
 - 5-1. 创建 TF 广播器
 - 5-2. 创建 广播的数据(通过 pose 设置)
 - 5-3. 广播器发布数据
6. spin

```
*/
// 1. 包含头文件
#include "ros/ros.h"
#include "turtlesim/Pose.h"
#include "tf2_ros/transform_broadcaster.h"
#include "geometry_msgs/TransformStamped.h"
#include "tf2/LinearMath/Quaternion.h"

void doPose(const turtlesim::Pose::ConstPtr& pose){
    // 5-1. 创建 TF 广播器
    static tf2_ros::TransformBroadcaster broadcaster;
    // 5-2. 创建 广播的数据(通过 pose 设置)
    geometry_msgs::TransformStamped tfs;
    // |----头设置
    tfs.header.frame_id = "world";
    tfs.header.stamp = ros::Time::now();

    // |----坐标系 ID
    tfs.child_frame_id = "turtle1";

    // |----坐标系相对信息设置
    tfs.transform.translation.x = pose->x;
    tfs.transform.translation.y = pose->y;
    tfs.transform.translation.z = 0.0; // 二维实现, pose 中没有z, z 是 0
    // |----- 四元数设置
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,pose->theta);
    tfs.transform.rotation.x = qtn.getX();
    tfs.transform.rotation.y = qtn.getY();
    tfs.transform.rotation.z = qtn.getZ();
    tfs.transform.rotation.w = qtn.getW();

    // 5-3. 广播器发布数据
    broadcaster.sendTransform(tfs);
}

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2. 初始化 ROS 节点
    ros::init(argc, argv, "dynamic_tf_pub");
    // 3. 创建 ROS 句柄
    ros::NodeHandle nh;
    // 4. 创建订阅对象
    ros::Subscriber sub = nh.subscribe<turtlesim::Pose>
("/turtle1/pose", 1000, doPose);
    // 5. 回调函数处理订阅到的数据(实现TF广播)
```

```

//
// 6.spin
ros::spin();
return 0;
}
Copy

```

配置文件此处略。

3.订阅方

```

//1.包含头文件
#include "ros/ros.h"
#include "tf2_ros/transform_listener.h"
#include "tf2_ros/buffer.h"
#include "geometry_msgs/PointStamped.h"
#include "tf2_geometry_msgs/tf2_geometry_msgs.h" //注意：调用 transform 必须包含该头文件

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ROS 节点
    ros::init(argc, argv, "dynamic_tf_sub");
    ros::NodeHandle nh;
    // 3.创建 TF 订阅节点
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);

    ros::Rate r(1);
    while (ros::ok())
    {
        // 4.生成一个坐标点(相对于子级坐标系)
        geometry_msgs::PointStamped point_laser;
        point_laser.header.frame_id = "turtle1";
        point_laser.header.stamp = ros::Time();
        point_laser.point.x = 1;
        point_laser.point.y = 1;
        point_laser.point.z = 0;
        // 5.转换坐标点(相对于父级坐标系)
        //新建一个坐标点，用于接收转换结果
        //-----使用 try 语句或休眠，否则可能由于缓存接收延迟而导致坐标转换失败---
        -----
        try
        {
            geometry_msgs::PointStamped point_base;
            point_base = buffer.transform(point_laser, "world");
            ROS_INFO("坐标点相对于 world 的坐标为：
(%.2f,%.2f,%.2f)", point_base.point.x, point_base.point.y, point_base.point.z);

        }
        catch(const std::exception& e)
        {
            // std::cerr << e.what() << '\n';
            ROS_INFO("程序异常:%s", e.what());
        }
    }
}

```

```

        r.sleep();
        ros::spinOnce();
    }

    return 0;
}
Copy

```

配置文件此处略。

4.执行

可以使用命令行或launch文件的方式分别启动发布节点与订阅节点，如果程序无异常，与演示结果类似。

可以使用 rviz 查看坐标系相对关系。

方案B:Python实现

1.创建功能包

创建项目功能包依赖于 tf2、tf2_ros、tf2_geometry_msgs、roscpp rospy std_msgs geometry_msgs、turtlesim

2.发布方

```

#!/usr/bin/env python
"""
    动态的坐标系相对姿态发布(一个坐标系相对于另一个坐标系的相对姿态是不断变动的)

    需求：启动 turtlesim_node,该节点中窗体有一个世界坐标系(左下角为坐标系原点)，乌龟是另一个
    坐标系，键盘
    控制乌龟运动，将两个坐标系的相对位置动态发布

    实现分析：
    1.乌龟本身不但可以看作坐标系，也是世界坐标系中的一个坐标点
    2.订阅 turtlesim/pose,可以获取乌龟在世界坐标系的 x坐标、y坐标、偏移量以及线速度和角速度
    3.将 pose 信息转换成 坐标系相对信息并发布

    实现流程：
    1.导包
    2.初始化 ROS 节点
    3.订阅 /turtlesim/pose 话题消息
    4.回调函数处理
        4-1.创建 TF 广播器
        4-2.创建 广播的数据(通过 pose 设置)
        4-3.广播器发布数据
    5.spin
"""
# 1.导包
import rospy
import tf2_ros
import tf
from turtlesim.msg import Pose
from geometry_msgs.msg import TransformStamped

```

```

#      4.回调函数处理
def doPose(pose):
    #      4-1.创建 TF 广播器
    broadcaster = tf2_ros.TransformBroadcaster()
    #      4-2.创建 广播的数据(通过 pose 设置)
    tfs = TransformStamped()
    tfs.header.frame_id = "world"
    tfs.header.stamp = rospy.Time.now()
    tfs.child_frame_id = "turtle1"
    tfs.transform.translation.x = pose.x
    tfs.transform.translation.y = pose.y
    tfs.transform.translation.z = 0.0
    qtn = tf.transformations.quaternion_from_euler(0,0,pose.theta)
    tfs.transform.rotation.x = qtn[0]
    tfs.transform.rotation.y = qtn[1]
    tfs.transform.rotation.z = qtn[2]
    tfs.transform.rotation.w = qtn[3]
    #      4-3.广播器发布数据
    broadcaster.sendTransform(tfs)

if __name__ == "__main__":
    # 2.初始化 ROS 节点
    rospy.init_node("dynamic_tf_pub_p")
    # 3.订阅 /turtle1/pose 话题消息
    sub = rospy.Subscriber("/turtle1/pose", Pose, doPose)
    #      4.回调函数处理
    #      4-1.创建 TF 广播器
    #      4-2.创建 广播的数据(通过 pose 设置)
    #      4-3.广播器发布数据
    #      5.spin
    rospy.spin()
Copy

```

权限设置以及配置文件此处略。

3.订阅方

```

#!/usr/bin/env python
"""

```

动态的坐标系相对姿态发布(一个坐标系相对于另一个坐标系的相对姿态是不断变动的)

需求：启动 `turtlesim_node`, 该节点中窗体有一个世界坐标系(左下角为坐标系原点)，乌龟是另一个坐标系，键盘

控制乌龟运动，将两个坐标系的相对位置动态发布

实现分析：

1. 乌龟本身不但可以看作坐标系，也是世界坐标系中的一个坐标点
2. 订阅 `turtle1/pose`, 可以获取乌龟在世界坐标系的 `x`坐标、`y`坐标、偏移量以及线速度和角速度

度

3. 将 `pose` 信息转换成 坐标系相对信息并发布

实现流程：

1. 导包
2. 初始化 ROS 节点
3. 创建 TF 订阅对象
4. 处理订阅的数据

```

"""
# 1.导包
import rospy
import tf2_ros
# 不要使用 geometry_msgs,需要使用 tf2 内置的消息类型
from tf2_geometry_msgs import PointStamped
# from geometry_msgs.msg import PointStamped

if __name__ == "__main__":
    # 2.初始化 ROS 节点
    rospy.init_node("static_sub_tf_p")
    # 3.创建 TF 订阅对象
    buffer = tf2_ros.Buffer()
    listener = tf2_ros.TransformListener(buffer)

    rate = rospy.Rate(1)
    while not rospy.is_shutdown():
        # 4.创建一个 radar 坐标系中的坐标点
        point_source = PointStamped()
        point_source.header.frame_id = "turtle1"
        point_source.header.stamp = rospy.Time.now()
        point_source.point.x = 10
        point_source.point.y = 2
        point_source.point.z = 3

        try:
            # 5.调研订阅对象的 API 将 4 中的点坐标转换成相对于 world 的坐标
            point_target =
buffer.transform(point_source,"world",rospy.Duration(1))
            rospy.loginfo("转换结果:x = %.2f, y = %.2f, z = %.2f",
                            point_target.point.x,
                            point_target.point.y,
                            point_target.point.z)
        except Exception as e:
            rospy.logerr("异常:%s",e)

        # 6.spin
        rate.sleep()
Copy

```

权限设置以及配置文件此处略。

4.执行

可以使用命令行或launch文件的方式分别启动发布节点与订阅节点，如果程序无异常，与演示结果类似。

可以使用 rviz 查看坐标系相对关系。

5.1.4 多坐标变换

需求描述:

现有坐标系统，父级坐标系统 world,下有两子级系统 son1，son2，son1 相对于 world，以及 son2 相对于 world 的关系是已知的，求 son1原点在 son2中的坐标，又已知在 son1中一点的坐标，要求求出该点在 son2 中的坐标

实现分析:

1. 首先, 需要发布 son1 相对于 world, 以及 son2 相对于 world 的坐标消息
2. 然后, 需要订阅坐标发布消息, 并取出订阅的消息, 借助于 tf2 实现 son1 和 son2 的转换
3. 最后, 还要实现坐标点的转换

实现流程:C++ 与 Python 实现流程一致

1. 新建功能包, 添加依赖
2. 创建坐标相对关系发布方(需要发布两个坐标相对关系)
3. 创建坐标相对关系订阅方
4. 执行

方案A:C++实现

1.创建功能包

创建项目功能包依赖于 tf2、tf2_ros、tf2_geometry_msgs、roscpp rospy std_msgs geometry_msgs、turtlesim

2.发布方

为了方便, 使用静态坐标变换发布

```
<launch>
  <node pkg="tf2_ros" type="static_transform_publisher" name="son1" args="0.2
0.8 0.3 0 0 0 /world /son1" output="screen" />
  <node pkg="tf2_ros" type="static_transform_publisher" name="son2" args="0.5
0 0 0 0 0 /world /son2" output="screen" />
</launch>
Copy
```

3.订阅方

```
/*
```

需求:

现有坐标系统, 父级坐标系统 world, 下有两子级系统 son1, son2, son1 相对于 world, 以及 son2 相对于 world 的关系是已知的, 求 son1 与 son2 中的坐标关系, 又已知在 son1 中一点的坐标, 要求求出该点在 son2 中的坐标

实现流程:

1. 包含头文件
2. 初始化 ros 节点
3. 创建 ros 句柄
4. 创建 TF 订阅对象
5. 解析订阅信息中获取 son1 坐标系原点在 son2 中的坐标
解析 son1 中的点相对于 son2 的坐标
6. spin

```
*/
```

```
//1.包含头文件
```

```
#include "ros/ros.h"
```

```
#include "tf2_ros/transform_listener.h"
```

```
#include "tf2/LinearMath/Quaternion.h"
```

```
#include "tf2_geometry_msgs/tf2_geometry_msgs.h"
```

```
#include "geometry_msgs/TransformStamped.h"
```

```

#include "geometry_msgs/PointStamped.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ros 节点
    ros::init(argc,argv,"sub_frames");
    // 3.创建 ros 句柄
    ros::NodeHandle nh;
    // 4.创建 TF 订阅对象
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);
    // 5.解析订阅信息中获取 son1 坐标系原点在 son2 中的坐标
    ros::Rate r(1);
    while (ros::ok())
    {
        try
        {
            // 解析 son1 中的点相对于 son2 的坐标
            geometry_msgs::TransformStamped tfs =
buffer.lookupTransform("son2","son1",ros::Time(0));
            ROS_INFO("Son1 相对于 Son2 的坐标关系:父坐标系
ID=%s",tfs.header.frame_id.c_str());
            ROS_INFO("Son1 相对于 Son2 的坐标关系:子坐标系
ID=%s",tfs.child_frame_id.c_str());
            ROS_INFO("Son1 相对于 Son2 的坐标关系:x=%.2f,y=%.2f,z=%.2f",
                tfs.transform.translation.x,
                tfs.transform.translation.y,
                tfs.transform.translation.z
                );

            // 坐标点解析
            geometry_msgs::PointStamped ps;
            ps.header.frame_id = "son1";
            ps.header.stamp = ros::Time::now();
            ps.point.x = 1.0;
            ps.point.y = 2.0;
            ps.point.z = 3.0;

            geometry_msgs::PointStamped psAtSon2;
            psAtSon2 = buffer.transform(ps,"son2");
            ROS_INFO("在 son2 中的坐标:x=%.2f,y=%.2f,z=%.2f",
                psAtSon2.point.x,
                psAtSon2.point.y,
                psAtSon2.point.z
                );
        }
        catch(const std::exception& e)
        {
            // std::cerr << e.what() << '\n';
            ROS_INFO("异常信息:%s",e.what());
        }

        r.sleep();
        // 6.spin
        ros::spinOnce();
    }
    return 0;
}

```

```
}  
Copy
```

配置文件此处略。

4.执行

可以使用命令行或launch文件的方式分别启动发布节点与订阅节点，如果程序无异常，将输出换算后的结果。

方案B:Python实现

1.创建功能包

创建项目功能包依赖于 tf2、tf2_ros、tf2_geometry_msgs、roscpp rospy std_msgs geometry_msgs、turtlesim

2.发布方

为了方便，使用静态坐标变换发布

```
<launch>  
  <node pkg="tf2_ros" type="static_transform_publisher" name="son1" args="0.2  
0.8 0.3 0 0 0 /world /son1" output="screen" />  
  <node pkg="tf2_ros" type="static_transform_publisher" name="son2" args="0.5  
0 0 0 0 0 /world /son2" output="screen" />  
</launch>  
Copy
```

3.订阅方

```
#!/usr/bin/env python  
"""  
    需求：  
        现有坐标系统，父级坐标系统 world, 下有两子级系统 son1, son2,  
        son1 相对于 world, 以及 son2 相对于 world 的关系是已知的，  
        求 son1 与 son2中的坐标关系，又已知在 son1中一点的坐标，要求求出该点在 son2 中的坐标  
    实现流程：  
        1. 导包  
        2. 初始化 ROS 节点  
        3. 创建 TF 订阅对象  
        4. 调用 API 求出 son1 相对于 son2 的坐标关系  
        5. 创建一依赖于 son1 的坐标点，调用 API 求出该点在 son2 中的坐标  
        6.spin  
    """  
  
# 1.导包  
import rospy  
import tf2_ros  
from geometry_msgs.msg import TransformStamped  
from tf2_geometry_msgs import PointStamped  
  
if __name__ == "__main__":  
  
    # 2.初始化 ROS 节点
```

```

rospy.init_node("frames_sub_p")
# 3.创建 TF 订阅对象
buffer = tf2_ros.Buffer()
listener = tf2_ros.TransformListener(buffer)

rate = rospy.Rate(1)
while not rospy.is_shutdown():

    try:
        # 4.调用 API 求出 son1 相对于 son2 的坐标关系
        #lookup_transform(self, target_frame, source_frame, time,
        timeout=rospy.Duration(0.0)):
        tfs = buffer.lookup_transform("son2","son1",rospy.Time(0))
        rospy.loginfo("son1 与 son2 相对关系:")
        rospy.loginfo("父级坐标系:%s",tfs.header.frame_id)
        rospy.loginfo("子级坐标系:%s",tfs.child_frame_id)
        rospy.loginfo("相对坐标:x=%.2f, y=%.2f, z=%.2f",
                        tfs.transform.translation.x,
                        tfs.transform.translation.y,
                        tfs.transform.translation.z,
                    )
        # 5.创建一依赖于 son1 的坐标点, 调用 API 求出该点在 son2 中的坐标
        point_source = PointStamped()
        point_source.header.frame_id = "son1"
        point_source.header.stamp = rospy.Time.now()
        point_source.point.x = 1
        point_source.point.y = 1
        point_source.point.z = 1

        point_target =
buffer.transform(point_source,"son2",rospy.Duration(0.5))

        rospy.loginfo("point_target 所属的坐标
系:%s",point_target.header.frame_id)
        rospy.loginfo("坐标点相对于 son2 的坐标:(%.2f,%.2f,%.2f)",
                        point_target.point.x,
                        point_target.point.y,
                        point_target.point.z
                    )

    except Exception as e:
        rospy.logerr("错误提示:%s",e)

    rate.sleep()
# 6.spin
# rospy.spin()
Copy

```

权限设置以及配置文件此处略。

4.执行

可以使用命令行或launch文件的方式分别启动发布节点与订阅节点，如果程序无异常，将输出换算后的结果。

5.1.5 坐标系关系查看

在机器人系统中，涉及的坐标系有多个，为了方便查看，ros 提供了专门的工具，可以用于生成显示坐标系关系的 pdf 文件，该文件包含树形结构的坐标系图谱。

6.1 准备

首先调用 `rospack find tf2_tools` 查看是否包含该功能包，如果没有，请使用如下命令安装：

```
sudo apt install ros-noetic-tf2-tools  
Copy
```

6.2 使用

6.2.1 生成 pdf 文件

启动坐标系广播程序之后，运行如下命令：

```
roslaunch tf2_tools view_frames.py  
Copy
```

会产生类似于下面的日志信息：

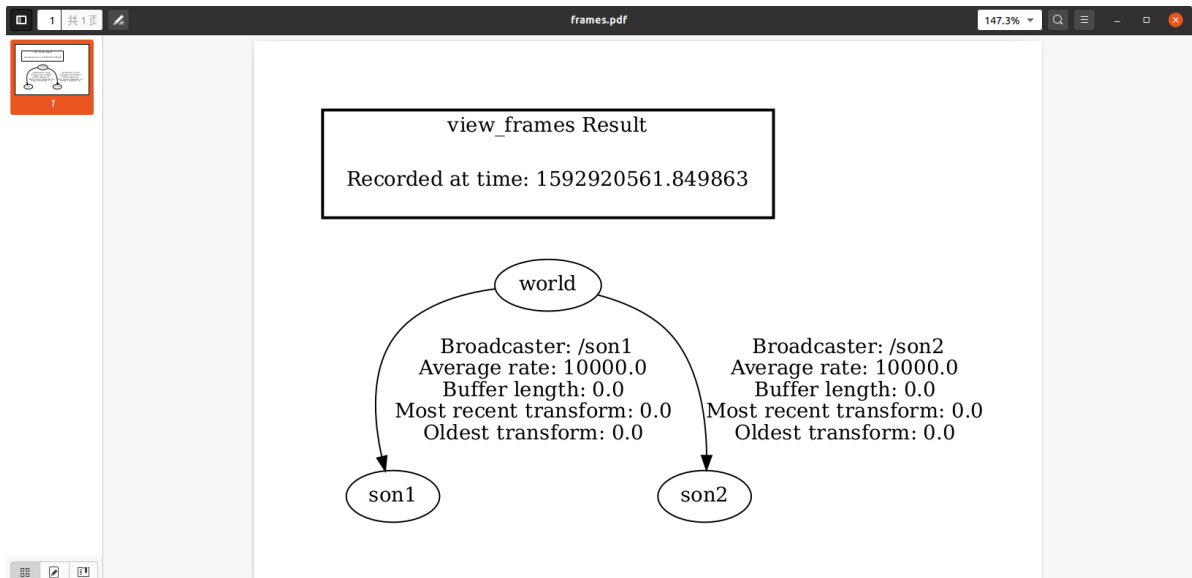
```
[INFO] [1592920556.827549]: Listening to tf data during 5 seconds...  
[INFO] [1592920561.841536]: Generating graph in frames.pdf file...  
Copy
```

查看当前目录会生成一个 frames.pdf 文件

6.2.2 查看文件

可以直接进入目录打开文件，或者调用命令查看文件：`evince frames.pdf`

内容如图所示：

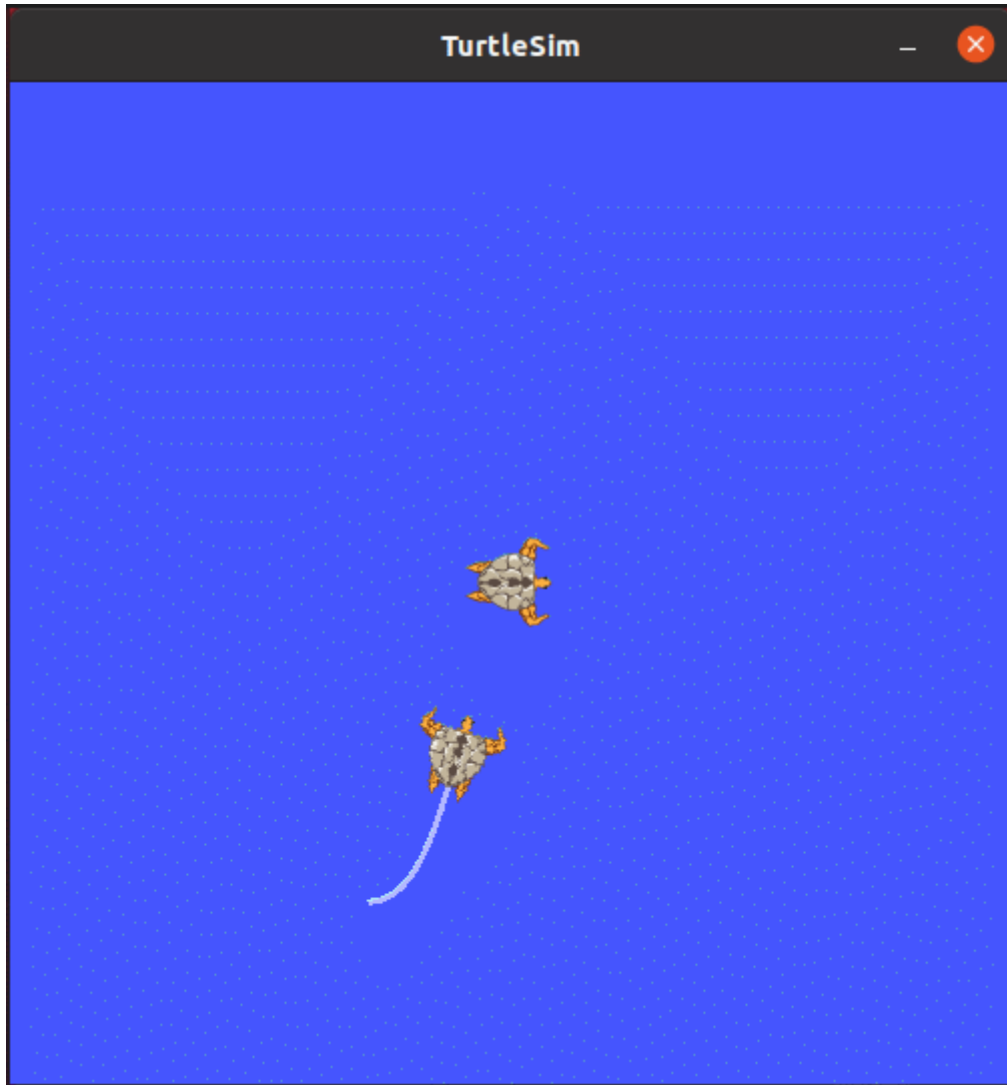


5.1.6 TF坐标变换实操

需求描述:

程序启动之初: 产生两只乌龟, 中间的乌龟(A) 和 左下乌龟(B), B 会自动运行至A的位置, 并且键盘控制时, 只是控制 A 的运动, 但是 B 可以跟随 A 运行

结果演示:



实现分析:

乌龟跟随实现的核心, 是乌龟A和B都要发布相对世界坐标系的坐标信息, 然后, 订阅到该信息需要转换获取A相对于B坐标系的信息, 最后, 再生成速度信息, 并控制B运动。

1. 启动乌龟显示节点
2. 在乌龟显示窗体中生成一只新的乌龟(需要使用服务)
3. 编写两只乌龟发布坐标信息的节点
4. 编写订阅节点订阅坐标信息并生成新的相对关系生成速度信息

实现流程:C++ 与 Python 实现流程一致

1. 新建功能包, 添加依赖
2. 编写服务客户端, 用于生成一只新的乌龟
3. 编写发布方, 发布两只乌龟的坐标信息
4. 编写订阅方, 订阅两只乌龟信息, 生成速度信息并发布
5. 运行

准备工作:

1.了解如何创建第二只乌龟，且不受键盘控制

创建第二只乌龟需要使用rosservice,话题使用的是 spawn

```
rosservice call /spawn "x: 1.0
y: 1.0
theta: 1.0
name: 'turtle_flow'"
name: "turtle_flow"
Copy
```

键盘是无法控制第二只乌龟运动的，因为使用的话题: /第二只乌龟名称/cmd_vel,对应的要控制乌龟运动必须发布对应的话题消息

2.了解如何获取两只乌龟的坐标

是通过话题 /乌龟名称/pose 来获取的

```
x: 1.0 //x坐标
y: 1.0 //y坐标
theta: -1.21437060833 //角度
linear_velocity: 0.0 //线速度
angular_velocity: 1.0 //角速度
Copy
```

方案A:C++实现

1.创建功能包

创建项目功能包依赖于 tf2、tf2_ros、tf2_geometry_msgs、roscpp rospy std_msgs geometry_msgs、turtlesim

2.服务客户端(生成乌龟)

```
/*
    创建第二只小乌龟
*/
#include "ros/ros.h"
#include "turtlesim/Spawn.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");

    //执行初始化
    ros::init(argc, argv, "create_turtle");
    //创建节点
    ros::NodeHandle nh;
    //创建服务客户端
    ros::ServiceClient client = nh.serviceClient<turtlesim::Spawn>("/spawn");

    ros::service::waitForService("/spawn");
    turtlesim::Spawn spawn;
    spawn.request.name = "turtle2";
    spawn.request.x = 1.0;
```

```

spawn.request.y = 2.0;
spawn.request.theta = 3.12415926;
bool flag = client.call(spawn);
if (flag)
{
    ROS_INFO("乌龟%s创建成功!", spawn.response.name.c_str());
}
else
{
    ROS_INFO("乌龟2创建失败!");
}

ros::spin();

return 0;
}
Copy

```

配置文件此处略。

3.发布方(发布两只乌龟的坐标信息)

可以订阅乌龟的位姿信息，然后再转换成坐标信息，两只乌龟的实现逻辑相同，只是订阅的话题名称，生成的坐标信息等稍有差异，可以将差异部分通过参数传入：

- 该节点需要启动两次
- 每次启动时都需要传入乌龟节点名称(第一次是 turtle1 第二次是 turtle2)

```

/*
    该文件实现:需要订阅 turtle1 和 turtle2 的 pose，然后广播相对 world 的坐标系信息

    注意：订阅的两只 turtle,除了命名空间(turtle1 和 turtle2)不同外，
    其他的话题名称和实现逻辑都是一样的，
    所以我们可以将所需的命名空间通过 args 动态传入

    实现流程：
    1.包含头文件
    2.初始化 ros 节点
    3.解析传入的命名空间
    4.创建 ros 句柄
    5.创建订阅对象
    6.回调函数处理订阅的 pose 信息
        6-1.创建 TF 广播器
        6-2.将 pose 信息转换成 TransformStamped
        6-3.发布
    7.spin

*/
//1.包含头文件
#include "ros/ros.h"
#include "turtlesim/Pose.h"
#include "tf2_ros/transform_broadcaster.h"
#include "tf2/LinearMath/Quaternion.h"
#include "geometry_msgs/TransformStamped.h"
//保存乌龟名称
std::string turtle_name;

```

```

void doPose(const turtlesim::Pose::ConstPtr& pose){
    // 6-1.创建 TF 广播器 ----- 注意 static
    static tf2_ros::TransformBroadcaster broadcaster;
    // 6-2.将 pose 信息转换成 TransformStamped
    geometry_msgs::TransformStamped tfs;
    tfs.header.frame_id = "world";
    tfs.header.stamp = ros::Time::now();
    tfs.child_frame_id = turtle_name;
    tfs.transform.translation.x = pose->x;
    tfs.transform.translation.y = pose->y;
    tfs.transform.translation.z = 0.0;
    tf2::Quaternion qtn;
    qtn.setRPY(0,0,pose->theta);
    tfs.transform.rotation.x = qtn.getX();
    tfs.transform.rotation.y = qtn.getY();
    tfs.transform.rotation.z = qtn.getZ();
    tfs.transform.rotation.w = qtn.getW();
    // 6-3.发布
    broadcaster.sendTransform(tfs);
}

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ros 节点
    ros::init(argc,argv,"pub_tf");
    // 3.解析传入的命名空间
    if (argc != 2)
    {
        ROS_ERROR("请传入正确的参数");
    } else {
        turtle_name = argv[1];
        ROS_INFO("乌龟 %s 坐标发送启动",turtle_name.c_str());
    }

    // 4.创建 ros 句柄
    ros::NodeHandle nh;
    // 5.创建订阅对象
    ros::Subscriber sub = nh.subscribe<turtlesim::Pose>(turtle_name +
"/pose",1000,doPose);
    // 6.回调函数处理订阅的 pose 信息
    // 6-1.创建 TF 广播器
    // 6-2.将 pose 信息转换成 TransformStamped
    // 6-3.发布
    // 7.spin
    ros::spin();
    return 0;
}
Copy

```

配置文件此处略。

4.订阅方(解析坐标信息并生成速度信息)

```
/*
    订阅 turtle1 和 turtle2 的 TF 广播信息，查找并转换时间最近的 TF 信息
    将 turtle1 转换成相对 turtle2 的坐标，在计算线速度和角速度并发布

    实现流程：
        1.包含头文件
        2.初始化 ros 节点
        3.创建 ros 句柄
        4.创建 TF 订阅对象
        5.处理订阅到的 TF
        6.spin

*/
//1.包含头文件
#include "ros/ros.h"
#include "tf2_ros/transform_listener.h"
#include "geometry_msgs/TransformStamped.h"
#include "geometry_msgs/Twist.h"

int main(int argc, char *argv[])
{
    setlocale(LC_ALL, "");
    // 2.初始化 ros 节点
    ros::init(argc,argv,"sub_TF");
    // 3.创建 ros 句柄
    ros::NodeHandle nh;
    // 4.创建 TF 订阅对象
    tf2_ros::Buffer buffer;
    tf2_ros::TransformListener listener(buffer);
    // 5.处理订阅到的 TF

    // 需要创建发布 /turtle2/cmd_vel 的 publisher 对象

    ros::Publisher pub = nh.advertise<geometry_msgs::Twist>
("/turtle2/cmd_vel",1000);

    ros::Rate rate(10);
    while (ros::ok())
    {
        try
        {
            //5-1.先获取 turtle1 相对 turtle2 的坐标信息
            geometry_msgs::TransformStamped tfs =
buffer.lookupTransform("turtle2","turtle1",ros::Time(0));

            //5-2.根据坐标信息生成速度信息 -- geometry_msgs/Twist.h
            geometry_msgs::Twist twist;
            twist.linear.x = 0.5 * sqrt(pow(tfs.transform.translation.x,2) +
pow(tfs.transform.translation.y,2));
            twist.angular.z = 4 *
atan2(tfs.transform.translation.y,tfs.transform.translation.x);

            //5-3.发布速度信息 -- 需要提前创建 publish 对象
            pub.publish(twist);
        }
    }
}
```

```

        catch(const std::exception& e)
        {
            // std::cerr << e.what() << '\n';
            ROS_INFO("错误提示:%s",e.what());
        }

        rate.sleep();
        // 6.spin
        ros::spinOnce();
    }

    return 0;
}
Copy

```

配置文件此处略。

5.运行

使用 launch 文件组织需要运行的节点，内容示例如下：

```

<!--
    tf2 实现小乌龟跟随案例
-->
<launch>
    <!-- 启动乌龟节点与键盘控制节点 -->
    <node pkg="turtlesim" type="turtlesim_node" name="turtle1" output="screen" />
    <node pkg="turtlesim" type="turtle_teleop_key" name="key_control"
    output="screen" />
    <!-- 启动创建第二只乌龟的节点 -->
    <node pkg="demo_tf2_test" type="Test01_Create_Turtle2" name="turtle2"
    output="screen" />
    <!-- 启动两个坐标发布节点 -->
    <node pkg="demo_tf2_test" type="Test02_TF2_Caster" name="caster1"
    output="screen" args="turtle1" />
    <node pkg="demo_tf2_test" type="Test02_TF2_Caster" name="caster2"
    output="screen" args="turtle2" />
    <!-- 启动坐标转换节点 -->
    <node pkg="demo_tf2_test" type="Test03_TF2_Listener" name="listener"
    output="screen" />
</launch>
Copy

```

方案B:Python实现

1.创建功能包

创建项目功能包依赖于 tf2、tf2_ros、tf2_geometry_msgs、roscpp rospy std_msgs geometry_msgs、turtlesim

2.服务客户端(生成乌龟)

```
#!/usr/bin/env python
"""
    调用 service 服务在窗体指定位置生成一只乌龟
    流程：
        1.导包
        2.初始化 ros 节点
        3.创建服务客户端
        4.等待服务启动
        5.创建请求数据
        6.发送请求并处理响应
"""

#1.导包
import rospy
from turtlesim.srv import Spawn, SpawnRequest, SpawnResponse

if __name__ == "__main__":
    # 2.初始化 ros 节点
    rospy.init_node("turtle_spawn_p")
    # 3.创建服务客户端
    client = rospy.ServiceProxy("/spawn",Spawn)
    # 4.等待服务启动
    client.wait_for_service()
    # 5.创建请求数据
    req = SpawnRequest()
    req.x = 1.0
    req.y = 1.0
    req.theta = 3.14
    req.name = "turtle2"
    # 6.发送请求并处理响应
    try:
        response = client.call(req)
        rospy.loginfo("乌龟创建成功, 名字是:%s",response.name)
    except Exception as e:
        rospy.loginfo("服务调用失败....")

Copy
```

权限设置以及配置文件此处略。

3.发布方(发布两只乌龟的坐标信息)

```
#!/usr/bin/env python
"""
    该文件实现:需要订阅 turtle1 和 turtle2 的 pose, 然后广播相对 world 的坐标系信息

    注意: 订阅的两只 turtle,除了命名空间(turtle1 和 turtle2)不同外,
        其他的话题名称和实现逻辑都是一样的,
        所以我们可以将所需的命名空间通过 args 动态传入

    实现流程：
        1.导包
        2.初始化 ros 节点
        3.解析传入的命名空间
        4.创建订阅对象
        5.回调函数处理订阅的 pose 信息
"""
```

```

        5-1.创建 TF 广播器
        5-2.将 pose 信息转换成 TransformStamped
        5-3.发布
    6.spin
"""
# 1.导包
import rospy
import sys
from turtlesim.msg import Pose
from geometry_msgs.msg import TransformStamped
import tf2_ros
import tf_conversions

turtle_name = ""

def doPose(pose):
    # rospy.loginfo("x = %.2f",pose.x)
    #1.创建坐标系广播器
    broadcaster = tf2_ros.TransformBroadcaster()
    #2.将 pose 信息转换成 TransformStamped
    tfs = TransformStamped()
    tfs.header.frame_id = "world"
    tfs.header.stamp = rospy.Time.now()

    tfs.child_frame_id = turtle_name
    tfs.transform.translation.x = pose.x
    tfs.transform.translation.y = pose.y
    tfs.transform.translation.z = 0.0

    qtn = tf_conversions.transformations.quaternion_from_euler(0, 0, pose.theta)
    tfs.transform.rotation.x = qtn[0]
    tfs.transform.rotation.y = qtn[1]
    tfs.transform.rotation.z = qtn[2]
    tfs.transform.rotation.w = qtn[3]

    #3.广播器发布 tfs
    broadcaster.sendTransform(tfs)

if __name__ == "__main__":
    # 2.初始化 ros 节点
    rospy.init_node("sub_tfs_p")
    # 3.解析传入的命名空间
    rospy.loginfo("-----%d",len(sys.argv))
    if len(sys.argv) < 2:
        rospy.loginfo("请传入参数:乌龟的命名空间")
    else:
        turtle_name = sys.argv[1]
    rospy.loginfo("//////////乌龟:%s",turtle_name)

    rospy.Subscriber(turtle_name + "/pose",Pose,doPose)
    #      4.创建订阅对象
    #      5.回调函数处理订阅的 pose 信息
    #          5-1.创建 TF 广播器
    #          5-2.将 pose 信息转换成 TransformStamped
    #          5-3.发布
    #      6.spin
    rospy.spin()

```

权限设置以及配置文件此处略。

4.订阅方(解析坐标信息并生成速度信息)

```
#!/usr/bin/env python
"""
    订阅 turtle1 和 turtle2 的 TF 广播信息，查找并转换时间最近的 TF 信息
    将 turtle1 转换成相对 turtle2 的坐标，在计算线速度和角速度并发布

    实现流程：
        1.导包
        2.初始化 ros 节点
        3.创建 TF 订阅对象
        4.处理订阅到的 TF
            4-1.查找坐标系的相对关系
            4-2.生成速度信息，然后发布
"""
# 1.导包
import rospy
import tf2_ros
from geometry_msgs.msg import TransformStamped, Twist
import math

if __name__ == "__main__":
    # 2.初始化 ros 节点
    rospy.init_node("sub_tfs_p")
    # 3.创建 TF 订阅对象
    buffer = tf2_ros.Buffer()
    listener = tf2_ros.TransformListener(buffer)
    # 4.处理订阅到的 TF
    rate = rospy.Rate(10)
    # 创建速度发布对象
    pub = rospy.Publisher("/turtle2/cmd_vel", Twist, queue_size=1000)
    while not rospy.is_shutdown():

        rate.sleep()
        try:
            #def lookup_transform(self, target_frame, source_frame, time,
            timeout=rospy.Duration(0.0)):
            trans = buffer.lookup_transform("turtle2", "turtle1", rospy.Time(0))
            # rospy.loginfo("相对坐标: (%.2f, %.2f, %.2f)",
            #               trans.transform.translation.x,
            #               trans.transform.translation.y,
            #               trans.transform.translation.z
            #               )
            # 根据转变后的坐标计算出速度和角速度信息
            twist = Twist()
            # 间距 =  $x^2 + y^2$  然后开方
            twist.linear.x = 0.5 *
            math.sqrt(math.pow(trans.transform.translation.x, 2) +
            math.pow(trans.transform.translation.y, 2))
            twist.angular.z = 4 * math.atan2(trans.transform.translation.y,
            trans.transform.translation.x)

            pub.publish(twist)
```

```
except Exception as e:
    rospy.logwarn("警告:%s",e)
```

Copy

权限设置以及配置文件此处略。

5.运行

使用 launch 文件组织需要运行的节点，内容示例如下：

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node" name="turtle1" output="screen"
/>
  <node pkg="turtlesim" type="turtle_teleop_key" name="key_control"
output="screen"/>

  <node pkg="demo06_test_flow_p" type="test01_turtle_spawn_p.py"
name="turtle_spawn" output="screen"/>

  <node pkg="demo06_test_flow_p" type="test02_turtle_tf_pub_p.py"
name="tf_pub1" args="turtle1" output="screen"/>
  <node pkg="demo06_test_flow_p" type="test02_turtle_tf_pub_p.py"
name="tf_pub2" args="turtle2" output="screen"/>
  <node pkg="demo06_test_flow_p" type="test03_turtle_tf_sub_p.py"
name="tf_sub" output="screen"/>

</launch>
```

5.1.7 TF2与TF

1.TF2与TF比较_简介

- TF2已经替换了TF，TF2是TF的超集，建议学习 TF2 而非 TF
- TF2 功能包的增强了内聚性，TF 与 TF2 所依赖的功能包是不同的，TF 对应的是 `tf` 包，TF2 对应的是 `tf2` 和 `tf2_ros` 包，在 TF2 中不同类型的 API 实现做了分包处理。
- TF2 实现效率更高，比如在:TF2 的静态坐标实现、TF2 坐标变换监听器中的 Buffer 实现等

2.TF2与TF比较_静态坐标变换演示

接下来，我们通过静态坐标变换来演示TF2的实现效率。

2.1启动 TF2 与 TF 两个版本的静态坐标变换

TF2 版静态坐标变换:`roslaunch tf2_ros static_transform_publisher 0 0 0 0 0 0 /base_link /laser`

TF 版静态坐标变换:`roslaunch tf static_transform_publisher 0 0 0 0 0 0 /base_link /laser 100`

会发现，TF 版本的启动中最后多一个参数，该参数是指定发布频率

2.2运行结果比对

使用 `rostopic` 查看话题，包含 `/tf` 与 `/tf_static`，前者是 TF 发布的话题，后者是 TF2 发布的话题，分别调用命令打印二者的话题消息

```
rostopic echo /tf: 当前会循环输出坐标系信息
```

```
rostopic echo /tf_static: 坐标系信息只有一次
```

2.3结论

如果是静态坐标转换，那么不同坐标系之间的相对状态是固定的，既然是固定的，那么没有必要重复发布坐标系的转换消息，很显然的，tf2 实现较之于 tf 更为高效

5.3 rqt工具箱

之前，在 ROS 中使用了一些实用的工具，比如：ros_bag 用于录制与回放、tf2_tools 可以生成 TF 树 这些工具大大提高了开发的便利性，但是也存在一些问题：这些工具的启动和使用过程中涉及到一些命令操作，应用起来不够方便，在 ROS 中，提供了 rqt 工具箱，在调用工具时以图形化操作代替了命令操作，应用更便利，提高了操作效率，优化了用户体验。

概念

ROS 基于 QT 框架，针对机器人开发提供了一系列可视化的工具，这些工具的集合就是 rqt

作用

可以方便的实现 ROS 可视化调试，并且在同一窗口中打开多个部件，提高开发效率，优化用户体验。

组成

rqt 工具箱组成有三大部分

- rqt——核心实现，开发人员无需关注
- rqt_common_plugins——rqt 中常用的工具套件
- rqt_robot_plugins——运行中和机器人交互的插件(比如: rviz)

5.3.1 rqt安装启动与基本使用

1.安装

- 一般只要你安装的是 desktop-full 版本就会自带工具箱
- 如果需要安装可以以如下方式安装

```
$ sudo apt-get install ros-noetic-rqt
$ sudo apt-get install ros-noetic-rqt-common-plugins
Copy
```

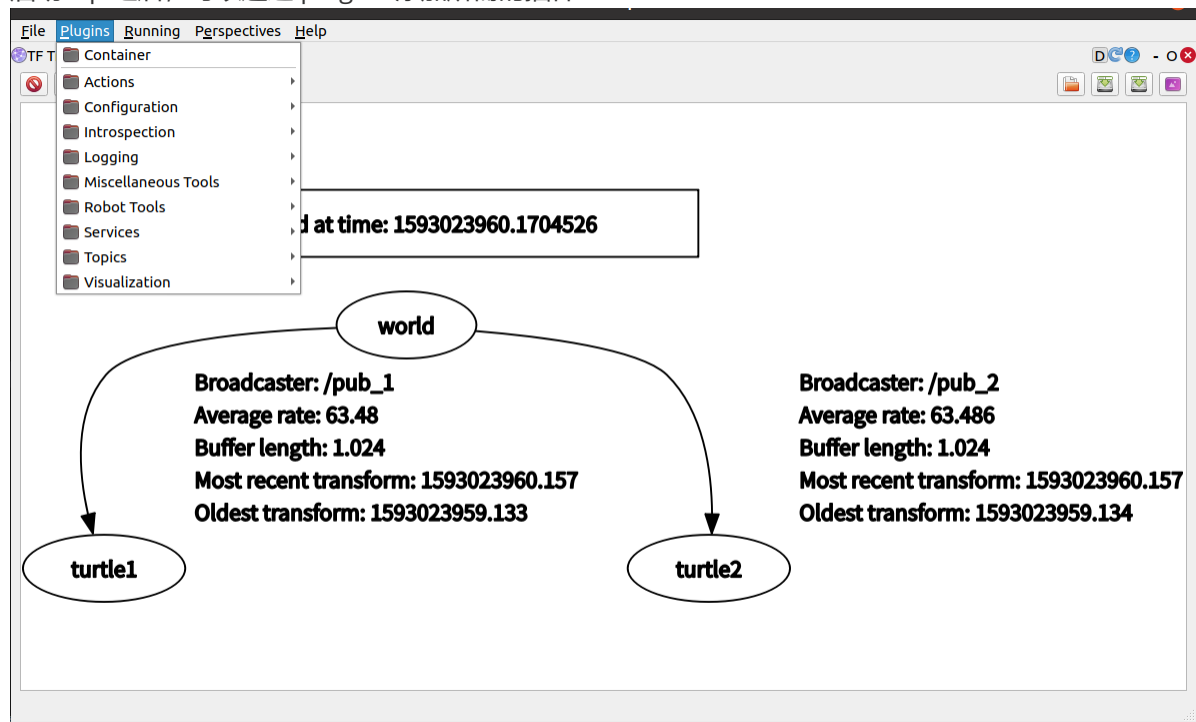
2.启动

rqt 的启动方式有两种:

- 方式1: `rqt`
- 方式2: `roslaunch rqt_gui rqt_gui`

3.基本使用

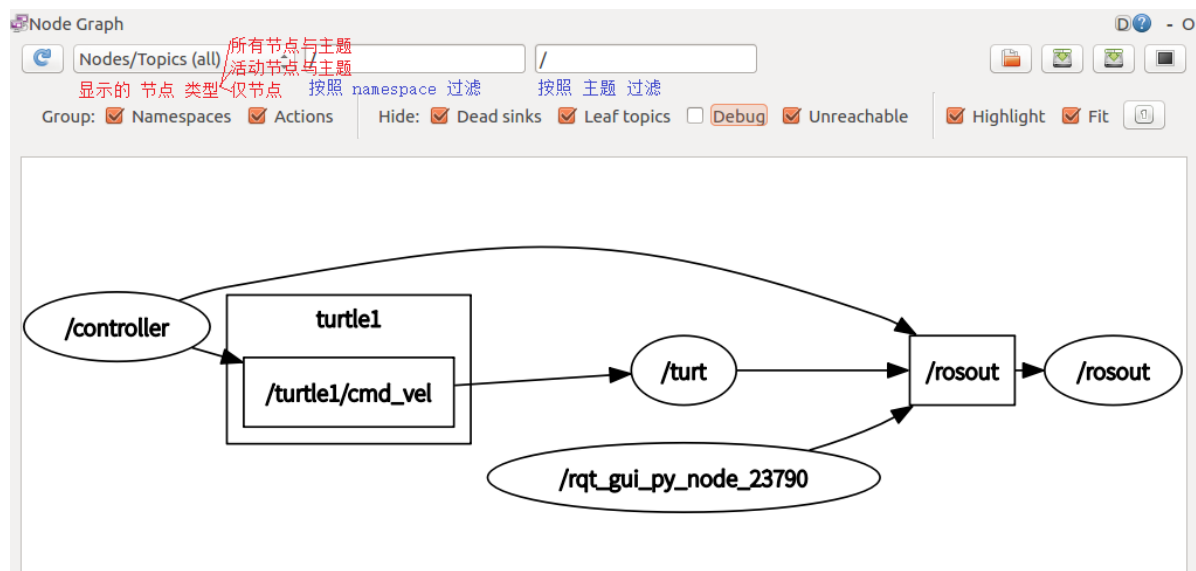
启动 rqt 之后, 可以通过 plugins 添加所需的插件



5.3.2 rqt常用插件:rqt_graph

简介:可视化显示计算图

启动:可以在 rqt 的 plugins 中添加, 或者使用 rqt_graph 启动



5.3.3 rqt常用插件:rqt_console

简介:rqt_console 是 ROS 中用于显示和过滤日志的图形化插件

准备:编写 Node 节点输出各个级别的日志信息

```
/*
ROS 节点:输出各种级别的日志信息
*/
```

```

#include "ros/ros.h"

int main(int argc, char *argv[])
{
    ros::init(argc,argv,"log_demo");
    ros::NodeHandle nh;

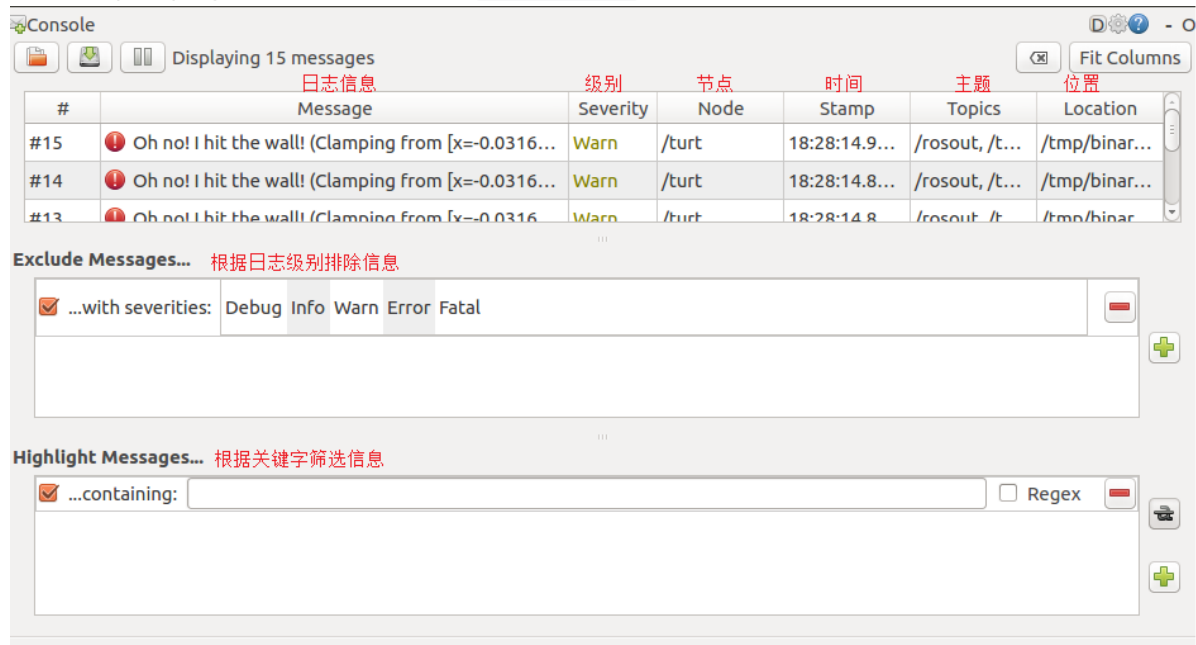
    ros::Rate r(0.3);
    while (ros::ok())
    {
        ROS_DEBUG("Debug message d");
        ROS_INFO("Info message oooooooooooooo");
        ROS_WARN("warn message wwwwww");
        ROS_ERROR("Error message EEEEEEEEEEEEEEEEEEE");
        ROS_FATAL("Fatal message FFFFFFFFFFFFFFFFFFFFFFFFFF");
        r.sleep();
    }

    return 0;
}
Copy

```

启动:

可以在 rqt 的 plugins 中添加, 或者使用 rqt_console 启动

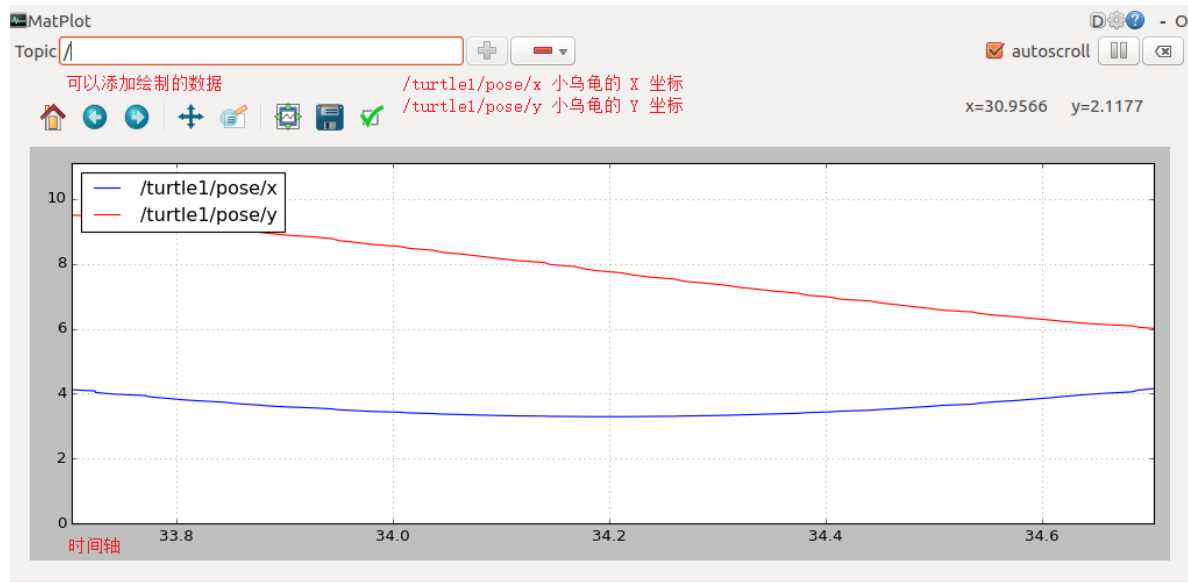


5.3.4 rqt常用插件:rqt_plot

简介:图形绘制插件, 可以以 2D 绘图的方式绘制发布在 topic 上的数据

准备:启动 turtlesim 乌龟节点与键盘控制节点, 通过 rqt_plot 获取乌龟位姿

启动:可以在 rqt 的 plugins 中添加, 或者使用 rqt_plot 启动



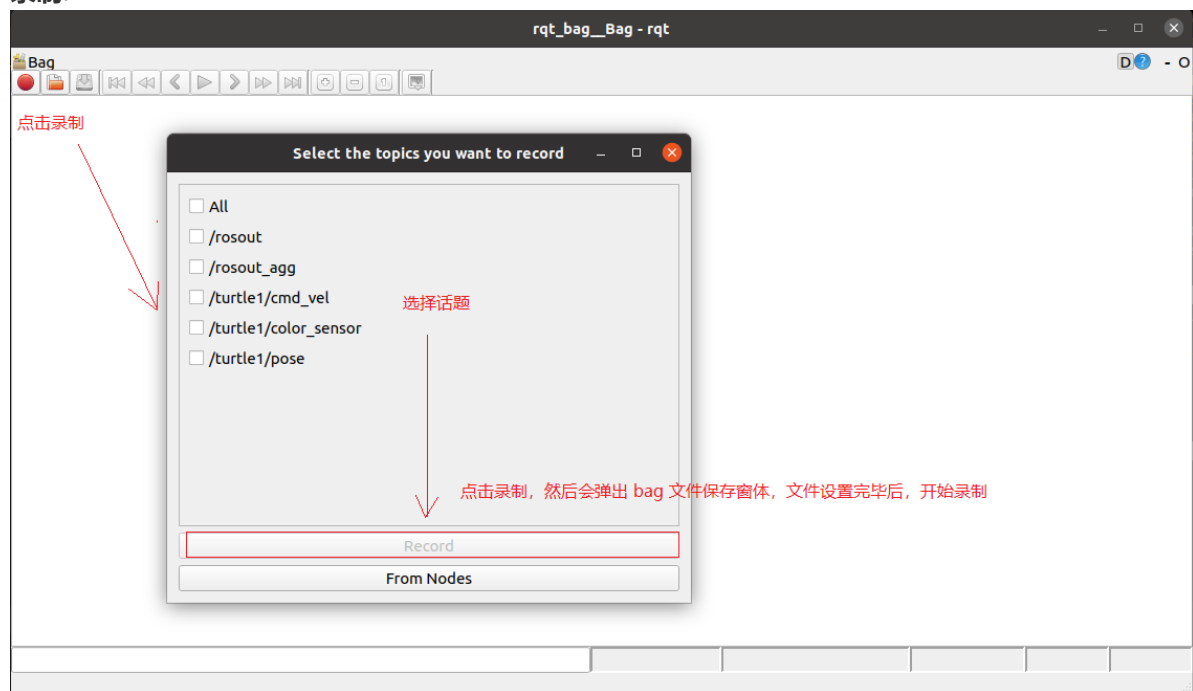
5.3.5 rqt常用插件:rqt_bag

简介:录制和重放 bag 文件的图形化插件

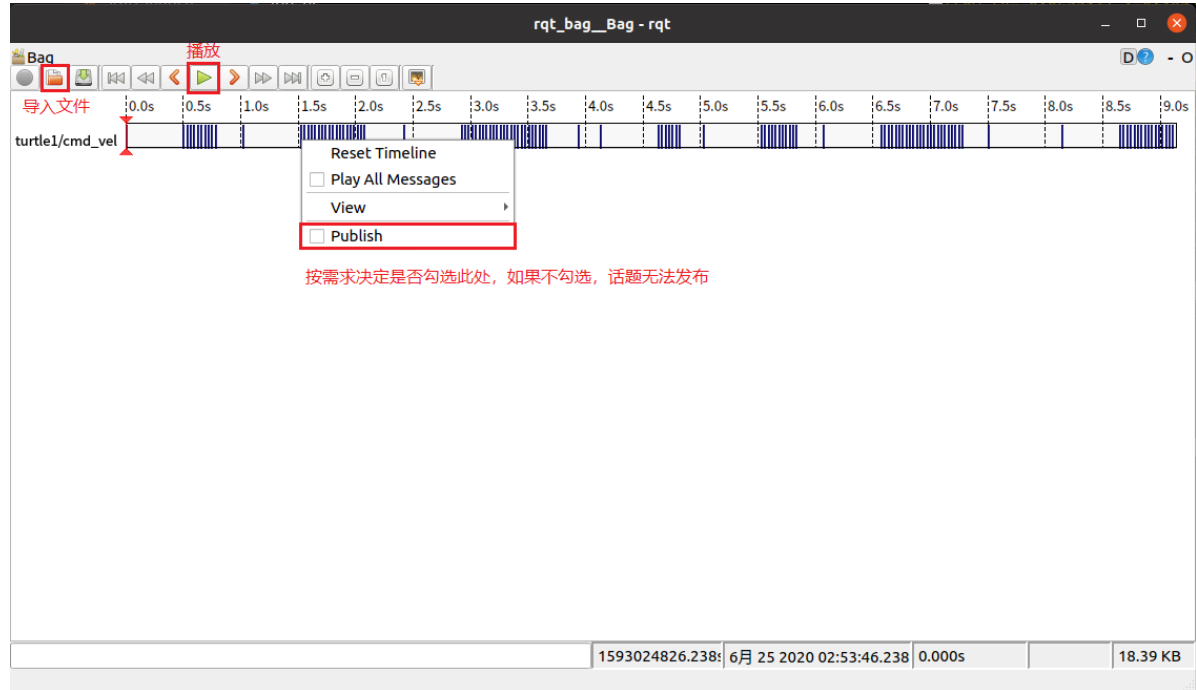
准备:启动 turtlesim 乌龟节点与键盘控制节点

启动:可以在 rqt 的 plugins 中添加, 或者使用 rqt_bag 启动

录制:



重放:



5.4 本章小结

本章主要介绍了ROS中的常用组件，内容如下：

- TF坐标变换(重点)
- rosbag 用于ros话题的录制与回放
- rqt工具箱，图形化方式调用组件，提高操作效率以及易用性

其中 TF坐标变换是重点，也是难点，需要大家熟练掌握坐标变换的应用场景以及代码实现。下一章开始将介绍机器人系统仿真，我们将在仿真环境下，创建机器人、控制机器人运动、搭建仿真环境，并以机器人的视角去感知世界。